



universidad
de león

Escuela de Ingenierías Industrial, Informática y
Aeroespacial
Universidad de León

Matemática Finita II

Grado en Ingeniería de Datos e Inteligencia Artificial

BLOQUE I

Unidad 2: Principios generales del diseño algorítmico

Dr. Miguel Angel Navarro Burgos

Contenido I

Introducción al diseño algorítmico

- Fundamentos Conceptuales de los Algoritmos

- Metodología del Diseño Algorítmico

- Concepto de Espacio de Búsqueda

 - Características Fundamentales del Espacio de Búsqueda

 - Problemas Ilustrativos y sus Espacios de Búsqueda

 - Espacio de Búsqueda y Paradigmas Algorítmicos

Herramientas de Formulación

- Diagramas y Pseudocódigo

Ejemplos de eficiencia algorítmica

- Máximo Común Divisor

- Exponenciación Eficiente

Paradigmas clásicos del diseño

- Análisis de Algoritmos Recursivos

- Divide y Conquista

Contenido II

- Programación Dinámica

- Algoritmos Geométricos

 - Ejemplos: Algoritmos Geométricos

- Algoritmos de aleatorización

 - Fundamentos

- Búsqueda y Optimización

 - Búsqueda Local y con Candidatos

 - Algoritmos Bioinspirados y Evolutivos

- Optimización avanzada

 - Optimización Combinatoria

 - Optimización Multiobjetivo

- Problemas de Complejidad NP

 - Backtracking y Branch & Bound

- Problemas de Análisis y Métodos Numéricos

Contenido III

- Grupo A: Ecuaciones y Optimización
- Grupo B: Aproximación y Representación
- Grupo C: Evolución y Dinámica
- Métodos Avanzados de Discretización
- Interconexión de Grupos

- Algoritmos Paralelos y Computación GPU
 - Fundamentos de Computación Paralela
 - Arquitectura CUDA
 - Patrones de Diseño Paralelo
 - Optimizaciones en CUDA
 - Comparativa y Consideraciones

- Análisis Comparativo
 - Tablas Comparativas

- Ejercicios propuestos

Contenido IV

NIVEL BÁSICO (30 Problemas)

NIVEL MEDIO (40 Problemas)

Bibliografía recomendada

Introducción al diseño algorítmico

Fundamentos Conceptuales de los Algoritmos

Definición Formal de Algoritmo I

Conceptualización Teórica

Un algoritmo constituye una secuencia finita de operaciones *discretas*, *unívocas* y *efectivas* destinadas a transformar un conjunto de entradas en una salida determinada. Desde una perspectiva formal, representa un procedimiento computacional bien definido que garantiza la resolución de una clase específica de problemas.

Definición Formal de Algoritmo II

Definición Matemática

Formalmente, puede definirse como una función computable $f : I \rightarrow O$, donde:

- ▶ I : dominio de entradas válidas (instancias del problema)
- ▶ O : codominio de salidas esperadas (soluciones)
- ▶ f : mapeo determinista que transforma la entrada en la salida, preservando la corrección.

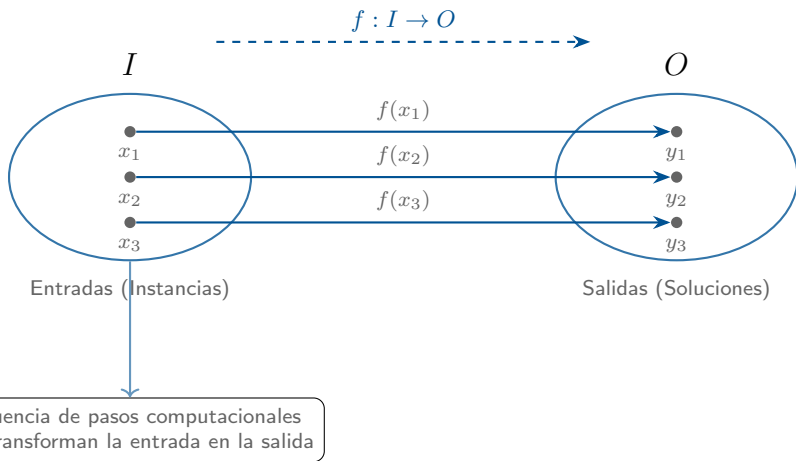
El algoritmo es, por tanto, la secuencia de pasos computacionales que materializa la función f .

Definición Formal de Algoritmo III

Características Esenciales

- ▶ **Discretitud:** Las operaciones son elementales, distinguibles y ejecutables en pasos definidos.
- ▶ **Univocidad:** Cada paso está definido sin ambigüedad operacional, garantizando reproducibilidad.
- ▶ **Efectividad:** Cada operación es implementable mediante primitivas computacionales básicas en tiempo finito.
- ▶ **Determinismo:** Para una entrada dada, el algoritmo produce siempre la misma salida.
- ▶ **Finitud:** El procedimiento debe concluir tras un número finito de pasos para toda entrada válida.

Definición Formal de Algoritmo



Propiedades Fundamentales de los Algoritmos I

Propiedades Axiomáticas

- ▶ **Finitud:** El procedimiento debe concluir tras un número finito de operaciones. Esta propiedad distingue a los algoritmos de los procesos computacionales infinitos.
- ▶ **Determinismo:** Cada estado del proceso determina unívocamente el estado subsiguiente, eliminando la ambigüedad en la ejecución.
- ▶ **Efectividad Computacional:** Cada operación debe ser implementable mediante primitivas computacionales básicas ejecutables en tiempo acotado.
- ▶ **Carácter General:** Capacidad para resolver una clase de problemas, no únicamente instancias particulares, garantizando reutilización.
- ▶ **Precisión:** Los pasos deben estar definidos sin ambigüedad operacional, facilitando la implementación y verificación.

Propiedades Fundamentales de los Algoritmos II

Implicaciones Prácticas

Estas propiedades constituyen criterios de validación para cualquier procedimiento que pretenda ser clasificado como algoritmo. Su verificación es fundamental en las etapas de especificación y diseño del desarrollo de software.

Corrección y Tipos de Algoritmos I

Planteamiento del Problema

Dado un problema computacional y un algoritmo propuesto para resolverlo, nos preguntamos: ¿el algoritmo produce siempre la salida correcta para toda entrada válida y termina en todos los casos?

Algoritmos Correctos

Se dice que un algoritmo es **correcto** si, para *toda* instancia de entrada, el algoritmo termina (*halts*) produciendo la salida correcta. Un algoritmo correcto **resuelve** el problema computacional para el que fue diseñado.

Corrección y Tipos de Algoritmos II

Algoritmos Incorrectos

Un algoritmo incorrecto puede no terminar para algunas instancias de entrada, o puede terminar produciendo una respuesta incorrecta.

- ▶ **Utilidad controlada:** Excepcionalmente, pueden ser útiles si su tasa de error es controlable (ej. algoritmos probabilísticos para tests de primalidad).
- ▶ En este curso, nos centraremos principalmente en el estudio de algoritmos correctos.

El Problema del Ordenamiento I

Planteamiento del Problema

Dada una secuencia desordenada de n números, se desea obtener una permutación de la misma en la que los elementos aparezcan en orden no decreciente (ascendente).

Definición Formal

Entrada: Una secuencia de n números $\langle a_1, a_2, \dots, a_n \rangle$.

Salida: Una permutación (reordenación) $\langle a'_1, a'_2, \dots, a'_n \rangle$ de la secuencia de entrada tal que $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

Ejemplo de Instancia

Dada la entrada $\langle 31, 41, 59, 26, 41, 58 \rangle$, un algoritmo de ordenamiento correcto devuelve como salida la secuencia $\langle 26, 31, 41, 41, 58, 59 \rangle$.

El Problema del Ordenamiento II

Importancia Fundamental y Multiplicidad de Soluciones

- ▶ El ordenamiento es una operación fundamental, usada como paso intermedio por innumerables programas.
- ▶ Existen múltiples algoritmos (inserción, mezcla, rápido, etc.).
- ▶ La elección del algoritmo óptimo depende del contexto: número de elementos (n), grado de orden previo, restricciones en los valores, arquitectura y jerarquía de memoria.

Ilustraciones Paradigmáticas I

Ejemplos Fundamentales

- ▶ **Algoritmo de Euclides:** Cálculo del máximo común divisor (MCD). Uno de los algoritmos más antiguos documentados (circa 300 a.C.).
- ▶ **Estrategias de Ordenamiento Comparativo:** Inserción (incremental) y Mezcla (divide y vencerás). Ilustran cómo diferentes estrategias llevan a diferentes complejidades temporales.
- ▶ **Búsqueda Estructurada:** Búsqueda binaria. Ejemplo de cómo explotar la estructura de los datos (orden) para lograr una eficiencia exponencialmente superior a la búsqueda lineal.
- ▶ **Recorrido de Grafos:** BFS (Breadth-First Search). Esencial para problemas de conectividad y rutas más cortas en grafos no ponderados.

Ilustraciones Paradigmáticas II

Relevancia del Estudio

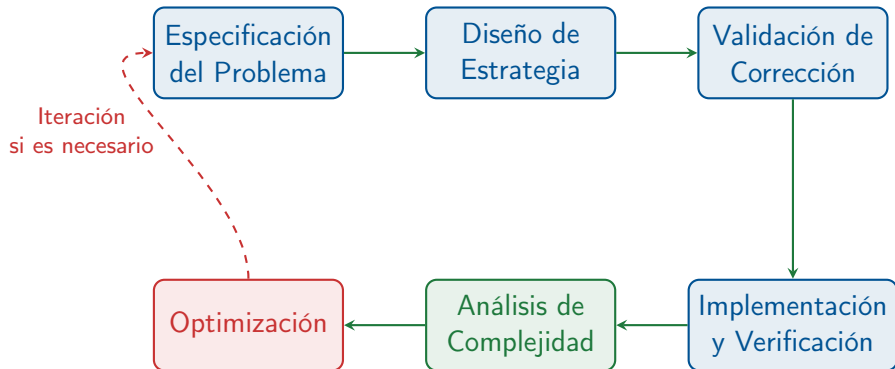
Estos algoritmos no son meros ejemplos aislados. Constituyen los fundamentos sobre los que se construyen soluciones computacionales complejas. Su estudio:

- ▶ Establece los principios del análisis de eficiencia.
- ▶ Proporciona patrones de diseño reutilizables.
- ▶ Prepara el terreno para entender problemas más avanzados (NP-completeness, algoritmos paralelos, etc.).

Metodología del Diseño Algorítmico

Ciclo de Vida del Diseño Algorítmico

Ciclo de Vida del Diseño Algorítmico



Ciclo de Vida del Diseño Algorítmico

Interpretación del Ciclo

El diseño algorítmico es un proceso iterativo donde el análisis puede revelar la necesidad de revisar la especificación o cambiar de paradigma. La optimización cierra el ciclo permitiendo refinamientos sucesivos.

Etapas del Proceso de Desarrollo I

Fase 1: Especificación del Problema

Formalización precisa de los requisitos de entrada y salida, así como la relación funcional entre ellas. Define el "qué" antes del "cómo". Incluye la definición de:

- ▶ **Instancia:** La entrada que satisface las restricciones del problema.
- ▶ **Solución:** La salida esperada para una instancia dada.

Una especificación clara (como la del problema de ordenamiento) es el primer paso crítico.

Etapas del Proceso de Desarrollo II

Fase 2: Diseño de la Estrategia

Selección del paradigma algorítmico. Las decisiones tomadas aquí determinan en gran medida la eficiencia futura. Ejemplos de enfoques:

- ▶ **Incremental:** Construir la solución paso a paso (ej. insertion sort).
- ▶ **Divide y Vencerás:** Descomponer el problema en subproblemas más pequeños y combinarlos (ej. merge sort).

Etapas del Proceso de Desarrollo III

Fase 3: Validación de Corrección

Demostración de que el algoritmo resuelve el problema para **todas** las entradas válidas. Técnicas comunes:

- ▶ Invariantes de bucle.
- ▶ Inducción matemática.

Un algoritmo incorrecto puede no terminar o dar una respuesta equivocada.

Etapas del Proceso de Desarrollo IV

Fase 4: Implementación y Verificación

Traducción del diseño a un lenguaje de programación concreto. La verificación empírica mediante pruebas (unitarias, de integración) busca errores de implementación y confirma el comportamiento esperado en un conjunto representativo de casos.

Etapas del Proceso de Desarrollo V

Fase 5: Análisis de Complejidad

Evaluación teórica de los recursos que consumirá el algoritmo, principalmente tiempo y espacio, en función del tamaño de la entrada. Utiliza la notación asintótica (ej. $O(n^2)$, $O(n \log n)$) para proporcionar una medida de eficiencia independiente de la máquina y el lenguaje. Este análisis es fundamental para comparar algoritmos y predecir su escalabilidad.

Concepto de Espacio de Búsqueda

Definición Formal del Espacio de Búsqueda I

Conceptualización Matemática

El *espacio de búsqueda* se define como el conjunto $\mathcal{S} = \{s_1, s_2, \dots, s_n\}$ de todas las posibles soluciones candidatas o configuraciones válidas para un problema computacional dado. Representa el dominio sobre el cual el algoritmo debe operar para encontrar una solución óptima o satisfactoria.

Definición Formal del Espacio de Búsqueda II

Formalización

Formalmente, $\mathcal{S} \subseteq \mathcal{U}$, donde:

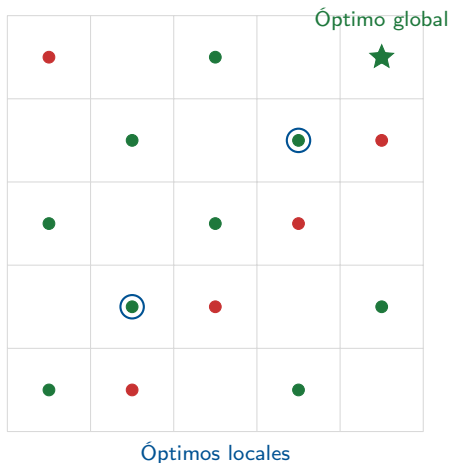
- ▶ $\mathcal{U}$: universo de todas las configuraciones posibles
- ▶ $\mathcal{S}$: subconjunto que satisface las restricciones del problema
- ▶ La cardinalidad $|\mathcal{S}|$ determina la complejidad de exploración exhaustiva

Definición Formal del Espacio de Búsqueda III

Función Objetivo

La **función objetivo** $f : \mathcal{S} \rightarrow \mathbb{R}$ evalúa la calidad de cada solución, permitiendo establecer criterios de optimalidad y comparación entre soluciones candidatas. En problemas de optimización, se busca $s^* \in \mathcal{S}$ tal que $f(s^*)$ sea máximo o mínimo según corresponda.

Ilustración: Espacio de Búsqueda Combinatorio



- Soluciones factibles
- Soluciones no factibles
- ★ Óptimo global
- Óptimos locales

Ilustración: Espacio de Búsqueda Combinatorio

Interpretación del Diagrama

El espacio de búsqueda contiene soluciones factibles (verdes) y no factibles (rojas). El objetivo del algoritmo es navegar eficientemente hacia el óptimo global (estrella) evitando quedar atrapado en óptimos locales (círculos).

Características Fundamentales del Espacio de Búsqueda

Características Fundamentales del Espacio de Búsqueda I

Dimensionalidad

Número de parámetros que definen cada solución candidata. Espacios de alta dimensionalidad presentan el fenómeno de *maldición de la dimensionalidad*, donde el volumen del espacio crece exponencialmente, dificultando la exploración exhaustiva.

Características Fundamentales del Espacio de Búsqueda II

Tamaño y Cardinalidad

Cardinalidad del conjunto $|\mathcal{S}|$, que determina la complejidad de exploración exhaustiva. Puede ser polinomial (explorable en tiempo razonable), exponencial (requiere estrategias de poda) o infinito (optimización continua).

Características Fundamentales del Espacio de Búsqueda III

Estructura Topológica

Organización del espacio (discreta, continua, combinatoria, mixta) que influye en la selección del paradigma algorítmico apropiado. La estructura determina qué operadores de exploración son válidos y eficientes.

Características Fundamentales del Espacio de Búsqueda IV

Métrica de Distancia

Función de distancia $d : \mathcal{S} \times \mathcal{S} \rightarrow \mathbb{R}^+$ que permite comparar soluciones y definir vecindarios para búsqueda local. Una métrica adecuada facilita la navegación eficiente del espacio de búsqueda.

Clasificación por Complejidad del Espacio I

Espacios Polinomiales

$|\mathcal{S}| = O(p(n))$ para algún polinomio p . Explorables en tiempo polinomial mediante algoritmos deterministas. Ejemplos: búsqueda lineal, ordenamiento básico, problemas en clase P.

Clasificación por Complejidad del Espacio II

Espacios Exponenciales

$|\mathcal{S}| = O(2^{p(n)})$. Requieren estrategias de poda, heurísticas o aproximación para ser manejables en la práctica. Ejemplos: problemas NP-completeness, optimización combinatoria, satisfacción de restricciones.

Clasificación por Complejidad del Espacio III

Espacios Infinitos

$|\mathcal{S}| = \infty$ (problemas de optimización continua). Requieren discretización o métodos de gradiente. Ejemplos: optimización de funciones reales, aprendizaje automático, ajuste de parámetros continuos.

Relevancia en Diseño Algorítmico

La caracterización del espacio de búsqueda determina la viabilidad de diferentes paradigmas algorítmicos y establece límites fundamentales sobre la eficiencia alcanzable. Constituye el primer paso en el análisis de cualquier problema computacional.

Problemas Ilustrativos y sus Espacios de Búsqueda

Problema: Ruta más Corta en un Mapa I

Problema

Dado un mapa de carreteras con distancias entre intersecciones, encontrar la ruta más corta entre dos puntos dados.

- ▶ **Espacio de búsqueda:** Todas las rutas posibles que conectan ambos puntos.
- ▶ **Dimensión del espacio:** Enorme, incluso descartando rutas que se cruzan sobre sí mismas.
- ▶ **Modelado:** El mapa se modela como un grafo, donde las intersecciones son vértices y las carreteras aristas con peso (distancia).
- ▶ **Solución eficiente:** Algoritmos como Dijkstra o BFS (para grafos no ponderados) exploran el espacio de forma inteligente sin evaluar todas las rutas.

Problema: Subsecuencia Común Más Larga (LCS) I

Problema

Dadas dos secuencias de símbolos, X (longitud m) e Y (longitud n), encontrar la subsecuencia más larga que aparece en ambas (en el mismo orden, no necesariamente consecutiva). Es una medida de similitud entre secuencias (ej. ADN).

- ▶ **Espacio de búsqueda:** El conjunto de todas las subsecuencias de X y de Y .
- ▶ **Dimensión del espacio:** X tiene 2^m subsecuencias posibles; Y tiene 2^n subsecuencias posibles.
- ▶ **Inviabilidad:** Generar y comparar todas las subsecuencias es prohibitivo para m, n moderados.
- ▶ **Solución eficiente:** Programación dinámica ($O(mn)$) explota la subestructura óptima del problema.

Problema: Ordenamiento de Piezas (Topológico) I

Problema

Dado un diseño con n piezas donde cada pieza puede usar otras (dependencias), listar las piezas en un orden tal que cada pieza aparezca antes que cualquier pieza que dependa de ella.

- ▶ **Espacio de búsqueda:** Todos los órdenes posibles de las n piezas.
- ▶ **Dimensión del espacio:** $n!$ (factorial de n) órdenes posibles. El factorial crece más rápido que una exponencial.
- ▶ **Inviabilidad:** Para $n = 20$, $20! \approx 2.43 \times 10^{18}$ órdenes.
- ▶ **Solución eficiente:** Ordenamiento topológico ($O(V + E)$) explota la estructura de grafo acíclico dirigido (DAG) del problema.

Problema: Envolverte Convexa (Convex Hull) I

Problema

Dados n puntos en el plano, encontrar el polígono convexo más pequeño que los contiene a todos (como una goma elástica que rodea todos los puntos).

- ▶ **Espacio de búsqueda:** Cualquier subconjunto de los n puntos podría ser, en principio, el conjunto de vértices de la envolvente convexa.
- ▶ **Dimensión del espacio:** Existen 2^n subconjuntos posibles de puntos.
- ▶ **Complejidad adicional:** No basta con seleccionar qué puntos son vértices; también es necesario determinar el **orden** en que aparecen alrededor del polígono.
- ▶ **Solución eficiente:** Algoritmos como Graham Scan o Jarvis March ($O(n \log n)$) ordenan los puntos y realizan un recorrido inteligente.

Espacio de Búsqueda y Paradigmas Algorítmicos

Espacio de Búsqueda y Paradigmas Algorítmicos I

Relación con Estrategias de Diseño

- ▶ **Divide y Vencerás:** Partición recursiva del espacio en subespacios independientes. Efectivo cuando los subproblemas son similares al original.
- ▶ **Programación Dinámica:** Exploración sistemática con **memoización** de subespacios solapados. Óptimo cuando hay subestructura óptima.
- ▶ **Algoritmos Voraces:** Selección local dentro del espacio sin backtracking. Eficiente cuando la elección local conduce al óptimo global.
- ▶ **Backtracking:** Exploración exhaustiva con poda de ramas inviables. Adecuado para espacios discretos con restricciones.
- ▶ **Branch and Bound:** Exploración dirigida por cotas en espacios de optimización. Combina exhaustividad con poda inteligente.

Espacio de Búsqueda y Paradigmas Algorítmicos II

Estrategias de Reducción del Espacio

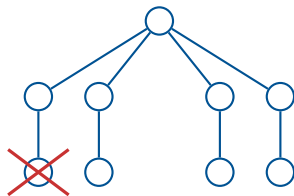
- ▶ **Poda:** Eliminación de regiones del espacio que no contienen soluciones óptimas.
- ▶ **Acotamiento:** Establecimiento de límites que permiten descartar subespacios.
- ▶ **Heurísticas:** Guías para exploración preferencial de regiones prometedoras.
- ▶ **Aproximación:** Reducción a subespacio de soluciones casi óptimas.

Espacio de Búsqueda y Paradigmas Algorítmicos III

Importancia Conceptual

El análisis del espacio de búsqueda precede a la selección del paradigma algorítmico y constituye una etapa crítica en el diseño de soluciones eficientes. Una caracterización adecuada puede reducir la complejidad de exponencial a polinomial.

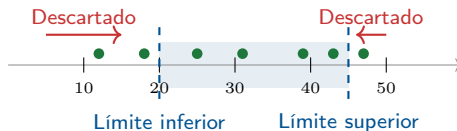
Estrategias de Reducción del Espacio: Poda



Podado

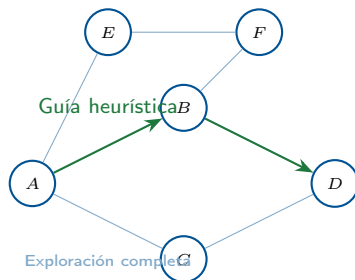
Eliminación de regiones que no contienen soluciones óptimas.

Estrategias de Reducción del Espacio: Acotamiento



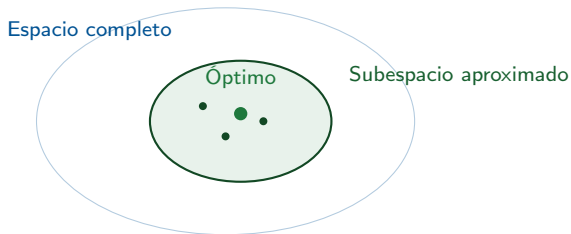
Establecimiento de límites para descartar subespacios no prometedores.

Estrategias de Reducción del Espacio: Heurísticas



Uso de reglas empíricas para priorizar regiones prometedoras.

Estrategias de Reducción del Espacio: Aproximación



Reducción a subespacio de soluciones casi óptimas.

Herramientas de Formulación

Diagramas y Pseudocódigo

Diagramas de Flujo en el Diseño Algorítmico I

Conceptualización

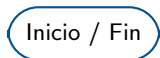
Los diagramas de flujo constituyen una representación gráfica estandarizada del flujo de control algorítmico mediante notación semiótica normalizada. Permiten visualizar la secuencia lógica de operaciones antes de la implementación en código.

Diagramas de Flujo en el Diseño Algorítmico II

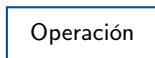
Ventajas Metodológicas

- ▶ **Abstracción visual:** Facilitan la comprensión de la estructura lógica global del algoritmo.
- ▶ **Detección temprana:** Permiten identificar anomalías en el flujo, como bucles infinitos o caminos inalcanzables.
- ▶ **Documentación técnica:** Constituyen una referencia de diseño valiosa para mantenimiento y transferencia de conocimiento.
- ▶ **Comunicación interdisciplinar:** Sirven como lenguaje común entre analistas, desarrolladores y stakeholders no técnicos.

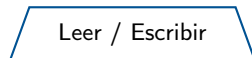
Simbología Estandarizada de Diagramas de Flujo



Terminador



Proceso



Entrada/Salida



Decisión



Conector



Elementos Fundamentales

- ▶ **Terminador:** Indica el inicio y fin del proceso (óvalos).
- ▶ **Proceso:** Representa operaciones o transformaciones de datos (rectángulos).
- ▶ **Decisión:** Bifurcación condicional con múltiples salidas (rombos).
- ▶ **Entrada/Salida:** Operaciones de lectura o escritura de datos (paralelogramos).
- ▶ **Conector:** Enlaces entre partes del diagrama (círculos o etiquetas).
- ▶ **Flujo de control:** Dirección del proceso mediante flechas.

Pseudocódigo como Herramienta de Especificación I

Definición y Propósito

El pseudocódigo es una notación semi-formal que describe algoritmos utilizando una mezcla de lenguaje natural, convenciones matemáticas y estructuras de control de programación, pero independiente de cualquier lenguaje de implementación específico.

Pseudocódigo como Herramienta de Especificación II

Características Fundamentales

- ▶ **Independencia de lenguaje:** No está atado a sintaxis particular de C++, Java o Python.
- ▶ **Legibilidad:** Prioriza la claridad sobre los detalles de implementación.
- ▶ **Nivel de abstracción:** Permite describir la lógica sin preocuparse por detalles de bajo nivel.
- ▶ **Equilibrio óptimo:** Combina *expresividad formal* con *accesibilidad cognitiva* para humanos.

Pseudocódigo como Herramienta de Especificación III

Convenciones de Notación

- ▶ **Indentación estricta:** Delimita ámbitos de control (similar a Python).
- ▶ **Palabras clave:** Si-Entonces-SiNo, Mientras, Repetir-Hasta, Devolver.
- ▶ **Operadores:** Uso de símbolos matemáticos estándar (+, −, ×, /, ∧, ∨).
- ▶ **Comentarios:** Explicaciones adicionales para claridad, generalmente entre llaves o con dos barras.

Ejemplo de Pseudocódigo I

Ejemplificación: Algoritmo de Suma de Elementos

Algoritmo 1 SumaElementos($A[1..n]$)

Require: A : arreglo de n elementos numéricos

Ensure: Suma total de los elementos de A

```
1: total  $\leftarrow$  0
2: for  $i \leftarrow 1$  to  $n$  do
3:   total  $\leftarrow$  total +  $A[i]$ 
4: end for
5: return total
```

Ejemplo de Pseudocódigo II

Análisis de Componentes

- ▶ **Línea 1-2 (Precondiciones):** Define los requisitos de entrada.
- ▶ **Línea 3:** Inicialización de variable acumuladora.
- ▶ **Líneas 4-6:** Iteración definida con ámbito delimitado por indentación.
- ▶ **Línea 5:** Operación fundamental del algoritmo (acumulación).
- ▶ **Línea 7:** Postcondición con retorno del resultado.

Pseudocódigo vs Lenguajes de Programación I

Característica	Pseudocódigo	Lenguaje de Programación
Sintaxis	Flexible, orientada a legibilidad	Estricta, definida por el compilador
Ejecutable	No, solo especificación	Sí, requiere compilación/interpretación
Nivel de abstracción	Alto, enfocado en lógica	Variable, incluye detalles de implementación
Propósito	Comunicación humana, diseño	Ejecución por máquina
Modificaciones	Rápidas, sin restricciones técnicas	Requieren compilación y pruebas

Pseudocódigo vs Lenguajes de Programación II

Ventaja Clave

El pseudocódigo permite centrarse en **qué** hace el algoritmo, no en **cómo** se implementa en un lenguaje específico, facilitando la traducción a cualquier plataforma posteriormente.

Constructores Fundamentales del Flujo de Control

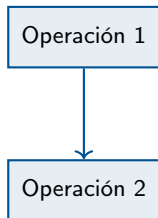
Todo algoritmo puede construirse mediante la combinación de tres estructuras básicas de control:

- ▶ **Secuencia:** Ejecución lineal de operaciones, una tras otra.
- ▶ **Selección (Condicional):** Bifurcación del flujo basada en la evaluación de predicados lógicos.
- ▶ **Iteración (Repetición):** Ejecución repetida de un bloque de operaciones.

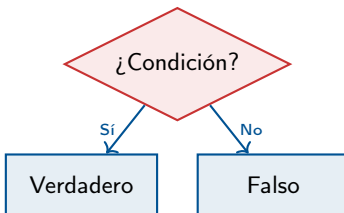
Estas estructuras, junto con la capacidad de modularización (subrutinas), constituyen la base de la programación estructurada.

Representación Gráfica de Estructuras de Control

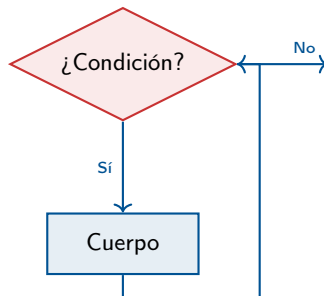
Secuencia



Selección



Iteración



Interpretación

- ▶ **Secuencia:** Flujo lineal sin bifurcaciones.
- ▶ **Selección:** Camino alternativo basado en condición (if-then-else).
- ▶ **Iteración:** Repetición mientras se cumple condición (while).

Patrones Iterativos Canónicos I

Iteración Definida (for)

Se utiliza cuando el número de repeticiones es conocido de antemano. Ideal para recorrer arreglos, matrices o secuencias de tamaño fijo.

Ejemplo: Recorrer todos los elementos de un arreglo de n posiciones.

Iteración Condicional Pre-test (while)

La condición se evalúa *antes* de cada iteración. Puede ejecutarse cero veces si la condición inicial es falsa. Adecuado cuando no se conoce el número de repeticiones.

Ejemplo: Búsqueda de un elemento hasta encontrarlo o alcanzar el final.

Patrones Iterativos Canónicos II

Iteración Condicional Post-test (do-while)

La condición se evalúa *después* de cada iteración, garantizando al menos una ejecución del cuerpo. Útil cuando se requiere procesar antes de verificar condición de salida.

Ejemplo: Validación de entrada de usuario (al menos una vez).

Patrones Iterativos Canónicos III

Selección Múltiple

Además de la selección binaria (if-then-else), existe la selección múltiple que permite elegir entre varias alternativas basadas en el valor de una expresión.

Ejemplo: Menús de opciones, procesamiento de comandos.

Comparativa de Patrones Iterativos I

Tipo	Pseudocódigo	Características
Definida	1: for $i = 1$ to n do 2: PROCESAR 3: end for	Número conocido de iteraciones
Pre-test	1: while <i>condición</i> do 2: PROCESAR 3: end while	Puede ejecutarse 0 veces
Post-test	1: repeat 2: PROCESAR 3: until <i>condición</i>	Ejecuta al menos 1 vez

Comparativa de Patrones Iterativos II

- ▶ Usar `for` cuando se conoce el número exacto de iteraciones.
- ▶ Usar `while` cuando la condición puede ser falsa inicialmente.
- ▶ Usar `do-while` cuando se necesita garantizar al menos una ejecución.

Buenas Prácticas en la Elaboración de Pseudocódigo I

Principios Fundamentales

- ▶ **Claridad sobre concisión:** Priorizar la legibilidad humana sobre la brevedad.
- ▶ **Consistencia:** Mantener mismo estilo y nomenclatura en todo el algoritmo.
- ▶ **Comentarios significativos:** Explicar partes no triviales, no obviedades.
- ▶ **Indentación:** Reflejar visualmente la estructura de control anidada.

Buenas Prácticas en la Elaboración de Pseudocódigo II

Errores Comunes a Evitar

- ▶ Mezclar múltiples lenguajes en el mismo pseudocódigo.
- ▶ Incluir detalles de implementación irrelevantes (punteros, gestión de memoria).
- ▶ Omitir casos borde o condiciones de parada.
- ▶ Utilizar estructuras de control ambiguas o mal definidas.

Transición a Implementación

Un buen pseudocódigo debe poder traducirse directamente a cualquier lenguaje de programación con cambios mínimos, principalmente sintácticos y no estructurales.

Ejemplos de eficiencia algorítmica

Máximo Común Divisor

MCD - Planteamiento del Problema I

Contexto del Problema

El cálculo del máximo común divisor (MCD) de dos números enteros positivos es un problema fundamental en teoría de números con aplicaciones en criptografía (RSA), simplificación de fracciones y algoritmos de sincronización.

Planteamiento del Problema

Dados dos enteros no negativos a y b , no ambos cero, encontrar el entero positivo más grande d tal que d divide exactamente tanto a a como a b .

MCD - Planteamiento del Problema II

Enfoques Comparados

- ▶ **Algoritmo de Euclides (restas):** Versión original basada en restas sucesivas.
- ▶ **Algoritmo de Euclides (módulo):** Versión optimizada usando operación módulo.
- ▶ **Fuerza bruta:** Verificación exhaustiva de todos los divisores posibles desde $\min(a, b)$ hasta 1.

MCD - Planteamiento del Problema III

Propiedad Fundamental

Para dos enteros positivos a y b con $a > b$:

$$\text{MCD}(a, b) = \text{MCD}(b, a \bmod b)$$

Esta propiedad permite reducir el problema recursivamente hasta alcanzar el caso base $\text{MCD}(d, 0) = d$.

MCD - Algoritmo de Euclides (Iterativo)

Algoritmo 2 MCD Euclides - Versión Iterativa

Require: a, b : enteros positivos

Ensure: $\text{MCD}(a, b)$

```
1: procedure MCD_EUCLIDES( $a, b$ )  
2:   while  $b \neq 0$  do  
3:      $r \leftarrow a \bmod b$   
4:      $a \leftarrow b$   
5:      $b \leftarrow r$   
6:   end while  
7:   return  $a$   
8: end procedure
```

Complejidad Temporal

Operación	Costo
Iteración del bucle	$O(1)$
Número de iteraciones	$O(\log(\min(a, b)))$
Total	$O(\log(\min(a, b)))$

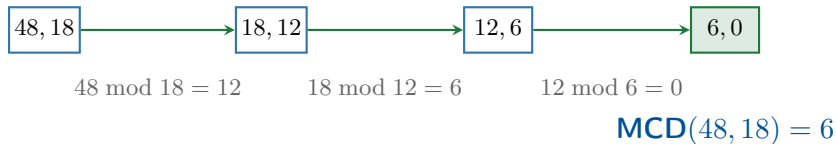
Complejidad Espacial

Componente	Espacio
Variables (a, b, r)	$O(1)$
Total	$O(1)$

Teorema de Lamé 1884

Sean a y b enteros positivos con $a > b$. El número de divisiones requeridas por el Algoritmo de Euclides para calcular $\text{MCD}(a, b)$ es a lo sumo $5k$, donde k es el número de dígitos decimales de b (el menor de los dos números).

MCD - Ilustración del Proceso



Comparativa de Complejidades

Resumen de Complejidades

Algoritmo	Temporal	Espacial
MCD Euclides (Iterativa)	$O(\log(\min(a, b)))$	$O(1)$
MCD Euclides (Recursiva)	$O(\log(\min(a, b)))$	$O(\log(\min(a, b)))$

Exponenciación Eficiente

Potencia de Números - Planteamiento I

Contexto del Problema

Calcular a^n (a elevado a la n) es una operación fundamental en matemáticas computacionales, criptografía (RSA, Diffie-Hellman) y algoritmos numéricos.

Planteamiento del Problema

Dados un número real a y un entero no negativo n , calcular el resultado de multiplicar a por sí mismo n veces: $a^n = a \times a \times \cdots \times a$ (n veces), con $a^0 = 1$.

Potencia de Números - Planteamiento II

Enfoques Comparados

- ▶ **Exponenciación ingenua:** Multiplicación repetida $a \times a \times \cdots \times a$ (n veces).
- ▶ **Exponenciación rápida:** Divide y vencerás aprovechando propiedades de potencias.

Propiedad Fundamental

$$a^n = \begin{cases} 1 & \text{si } n = 0 \\ (a^{n/2})^2 & \text{si } n \text{ es par} \\ a \cdot (a^{(n-1)/2})^2 & \text{si } n \text{ es impar} \end{cases}$$

Potencia - Exponenciación Ingenua

Algoritmo 3 Potencia Ingenua (Naive)

Require: a : base, n : exponente (entero no negativo)

Ensure: a^n

```
1: procedure POTENCIA_INGENUA( $a, n$ )
2:   if  $n = 0$  then
3:     return 1
4:   end if
5:    $resultado \leftarrow 1$ 
6:   for  $i \leftarrow 1$  to  $n$  do
7:      $resultado \leftarrow resultado \times a$ 
8:   end for
9:   return  $resultado$ 
10: end procedure
```

Cálculo de Complejidad Temporal

Operaciones: Bucle de 1 a n .

Costo por iteración: $O(1)$ (multiplicación).

Resultado: $T(n) = O(n)$

Análisis de Consumo de Memoria (Modelo RAM)

- ▶ Variable a : base (1 unidad).
- ▶ Variable n : exponente (1 unidad).
- ▶ Variable *resultado*: acumulador (1 unidad).
- ▶ Variable i : contador (1 unidad).
- ▶ Código del algoritmo: memoria constante.

El consumo total es independiente de n :

$$M(n) = 4 + \text{constante} = O(1)$$

Potencia - Exponenciación Rápida (Recursiva)

Algoritmo 4 Potencia Rápida (Divide y Vencerás)

Require: a : base, n : exponente (entero no negativo)

Ensure: a^n

```
1: procedure POTENCIA_RAPIDA( $a, n$ )
2:   if  $n = 0$  then
3:     return 1    //Caso base
4:   end if
5:    $temp \leftarrow$  POTENCIA_RAPIDA( $a, \lfloor n/2 \rfloor$ )
6:   if  $n$  es par then
7:     return  $temp \times temp$ 
8:   else
9:     return  $a \times temp \times temp$ 
10:  end if
11: end procedure
```

Cálculo de Complejidad Temporal

Recurrencia: $T(n) = T(n/2) + O(1)$

Resolución (Teorema Maestro):

- ▶ $a = 1, b = 2, f(n) = O(1)$
- ▶ $n^{\log_b a} = n^0 = 1$
- ▶ Caso 2: $T(n) = O(\log n)$

Análisis de Consumo de Memoria (Modelo RAM)

- ▶ Variables por llamada: $a, n, temp$ (3 unidades).
- ▶ Profundidad de recursión: $\log n$.
- ▶ Código del algoritmo: memoria constante.

El consumo total depende de la profundidad:

$$M(n) = 3 \cdot \log n + \text{constante} = O(\log n)$$

Potencia - Versión Iterativa

Algoritmo 5 Potencia Rápida Iterativa

Require: a : base, n : exponente (entero no negativo)

Ensure: a^n

```
1: procedure POTENCIA_RAPIDA_ITER( $a, n$ )
2:    $resultado \leftarrow 1$ 
3:    $base \leftarrow a$ 
4:   while  $n > 0$  do
5:     if  $n$  es impar then
6:        $resultado \leftarrow resultado \times base$ 
7:     end if
8:      $base \leftarrow base \times base$ 
9:      $n \leftarrow \lfloor n/2 \rfloor$ 
10:  end while
11:  return  $resultado$ 
12: end procedure
```

Potencia Iterativa - Análisis de Complejidad I

Cálculo de Complejidad Temporal

Temporal: $T(n) = O(\log n)$ (mismos pasos que recursiva).

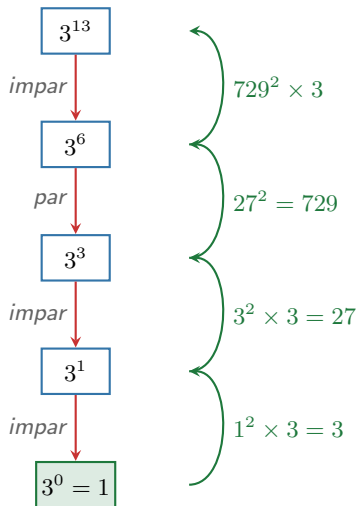
Análisis de Consumo de Memoria (Modelo RAM)

- ▶ Variable a : base original (1 unidad).
- ▶ Variable n : exponente (1 unidad).
- ▶ Variable $resultado$: acumulador (1 unidad).
- ▶ Variable $base$: base actual (1 unidad).
- ▶ Código del algoritmo: memoria constante.

El consumo total es independiente de n :

$$M(n) = 4 + \text{constante} = O(1)$$

Potencia - Ilustración del Proceso



Potencia - Comparación de Multiplicaciones I

Ejemplo: 3^{13}

Método	Multiplicaciones
Ingenuo	13
Rápido	5

Potencia - Comparación de Multiplicaciones II

Resumen de Complejidades

Algoritmo	Temporal	Espacial
Potencia (Ingenua)	$O(n)$	$O(1)$
Potencia Rápida (Recursiva)	$O(\log n)$	$O(\log n)$
Potencia Rápida (Iterativa)	$O(\log n)$	$O(1)$

Potencia - Comparación de Multiplicaciones III

Análisis Comparativo Asintótico

► Crecimiento asintótico:

$$\lim_{n \rightarrow \infty} \frac{\log n}{n} = 0 \Rightarrow \log n = O(n)$$

► Mejora exponenciación: Para $n = 10^6$:

- Ingenua: $\sim 10^6$ multiplicaciones.
- Rápida: ~ 20 multiplicaciones.
- **Factor de mejora:** $50,000\times$.

► Eficiencia espacial: Versiones iterativas usan $O(1)$ espacio.

Potencia - Comparación de Multiplicaciones IV

Conclusión

Las versiones optimizadas (Euclides con módulo, exponenciación rápida) demuestran superioridad asintótica al explotar propiedades matemáticas del problema, reduciendo complejidades de $O(n)$ a $O(\log n)$.

Principio Fundamental

La complejidad inherente de un problema puede ser reducida mediante el diseño de algoritmos que exploten inteligentemente la estructura matemática específica del problema, considerando tanto el tiempo como el espacio computacional.

Paradigmas clásicos del diseño

Análisis de Algoritmos Recursivos

Fundamentos de Recursividad I

Conceptualización

El estudio de los algoritmos recursivos constituye un aspecto central en el análisis de la eficiencia computacional. Este tipo de algoritmos se caracteriza por resolver un problema a partir de instancias más pequeñas del mismo problema, siguiendo el principio de *auto-referencia*.

Fundamentos de Recursividad II

Componentes de un Algoritmo Recursivo

- ▶ **Caso base:** Condición que termina la recursión, resolviendo directamente la instancia más pequeña.
- ▶ **Caso recursivo:** Llamada a la misma función con parámetros reducidos, acercándose al caso base.
- ▶ **Combinación:** Proceso que integra los resultados de las llamadas recursivas para formar la solución final.

Fundamentos de Recursividad III

Relaciones de Recurrencia (Modelo Divide y Vencerás)

Una relación de recurrencia modela el tiempo de ejecución $T(n)$ de un algoritmo recursivo al expresar $T(n)$ en función de $T(n')$ para $n' < n$, sumado al costo asociado a la división del problema y combinación de los subproblemas.

Para algoritmos que siguen el paradigma **divide y vencerás**, la forma general es:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

donde:

- ▶ a : número de subproblemas generados
- ▶ b : factor de reducción del tamaño
- ▶ $f(n)$: costo de dividir y combinar

Fundamentos de Recursividad IV

Relaciones de Recurrencia (Modelo Lineal)

Existen algoritmos recursivos donde la reducción se produce mediante **sustracción de constantes**. La forma general para estos casos es:

$$T(n) = \sum_{i=1}^k a_i T(n - c_i) + f(n)$$

donde:

- ▶ k : número de términos recursivos distintos
- ▶ a_i : coeficiente del i -ésimo subproblema
- ▶ c_i : constante de reducción del i -ésimo subproblema ($1 \leq c_i \leq n$)
- ▶ $f(n)$: costo de las operaciones no recursivas

Ejemplo 1: Factorial Recursivo - Planteamiento del Problema

Dado un número entero no negativo n , calcular el factorial $n!$, definido como el producto de todos los enteros positivos desde 1 hasta n , con $0! = 1$.

Ejemplo 1: Factorial Recursivo - Algoritmo

Algoritmo 6 Cálculo Recursivo de Factorial

Require: $n \geq 0$ entero

Ensure: $n!$ (factorial de n)

```
1: procedure FACTORIAL( $n$ )  
2:   if  $n = 0$  then  
3:     return 1    //Caso base  
4:   else  
5:     return  $n \times \text{factorial}(n - 1)$     //Caso recursivo  
6:   end if  
7: end procedure
```

Ejemplo 1: Factorial Recursivo - Análisis de Complejidad I

Recurrencia: $T(n) = T(n - 1) + O(1)$

Resolución: como

$$b^n p(n) = O(1) \Rightarrow b = 1 \wedge \deg(p) = 0.$$

Entonces

$$(r - 1)(r - 1)^{0+1} = (r - 1)^2 = 0 \Rightarrow T(n) = c_1 + c_2 n = O(n).$$

Resumen de Complejidades

Temporal	Espacial
$O(n)$	$O(n)$

Ejemplo 1: Factorial Recursivo - Análisis de Complejidad II

Observación

Aunque la versión recursiva es elegante, la versión iterativa logra la misma complejidad temporal con $O(1)$ espacio, siendo superior en la práctica.

Ejemplo 2: Fibonacci Recursivo - Planteamiento del Problema

Definición de la Sucesión de Fibonacci

La sucesión de Fibonacci se define mediante la relación de recurrencia:

$$F(0) = 0, \quad F(1) = 1, \quad F(n) = F(n-1) + F(n-2) \text{ para } n \geq 2.$$

Problema: Dado un número entero no negativo n , calcular el n -ésimo número de Fibonacci $F(n)$.

Ejemplo 2: Fibonacci Recursivo - Algoritmo

Algoritmo 7 Cálculo Recursivo de Fibonacci

Require: $n \geq 0$ entero

Ensure: $F(n)$ (n-ésimo número de Fibonacci)

```
1: procedure FIBONACCI( $n$ )
2:   if  $n \leq 1$  then
3:     return  $n$     //Casos base:  $F(0) = 0$ ,  $F(1) = 1$ 
4:   else
5:     return FIBONACCI( $n-1$ ) + FIBONACCI( $n-2$ )    //Caso recursivo
6:   end if
7: end procedure
```

Ejemplo 2: Fibonacci Recursivo - Análisis de Complejidad I

Recurrencia: $T(n) = T(n-1) + T(n-2) + O(1)$

Resolución: como

$$b^n p(n) = O(1) \Rightarrow b = 1 \wedge \deg(p) = 0.$$

Entonces

$$(r^2 - r - 1)(r - 1)^{0+1} = 0 \Rightarrow T(n) = c_1 \phi^n + c_2 \psi^n + c_3$$

con

$$\phi = \frac{1 + \sqrt{5}}{2} \wedge \psi = \frac{1 - \sqrt{5}}{2}.$$

Como $|\psi| < 1$, el término ψ^n tiende a 0 para n grande, por lo que:

$$T(n) = O(\phi^n)$$

Ejemplo 2: Fibonacci Recursivo - Análisis de Complejidad II

Resumen de Complejidades

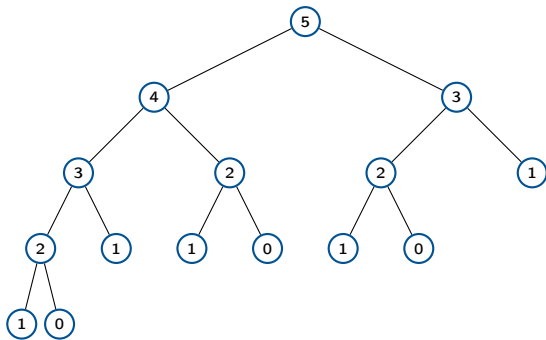
Temporal	Espacial
$O(\phi^n)$ con $\phi \approx 1.618$	$O(n)$

Ineficiencia del Enfoque

- ▶ Mismos subproblemas se calculan múltiples veces (redundancia exponencial)
- ▶ Árbol de recursión con profundidad n y número de nodos $\approx \phi^n$
- ▶ Para $n = 50$: $\phi^{50} \approx 2 \times 10^{10}$ llamadas
- ▶ Para $n = 100$: $\phi^{100} \approx 3.5 \times 10^{20}$ llamadas (inviabile)

Ejemplo 2: Fibonacci - Árbol de Llamadas I

Visualización del Árbol de Llamadas para $F(5)$



Ejemplo 2: Fibonacci - Árbol de Llamadas II

Observación

Nótese la enorme redundancia: $F(3)$ se calcula dos veces, $F(2)$ tres veces, $F(1)$ cinco veces, $F(0)$ tres veces. Esta repetición es la causa de la complejidad exponencial.

Ejemplo 3: Torres de Hanoi - Planteamiento del Problema

Problema de las Torres de Hanoi

Dadas tres varillas (origen, destino, auxiliar) y n discos de diferentes tamaños apilados en la varilla origen en orden decreciente de tamaño (el mayor abajo), mover todos los discos a la varilla destino respetando:

- ▶ Solo se puede mover un disco a la vez.
- ▶ Nunca se puede colocar un disco mayor sobre uno menor.

Ejemplo 3: Torres de Hanoi - Algoritmo

Algoritmo 8 Torres de Hanoi

Require: n : número de discos, varillas: *origen*, *destino*, *auxiliar*

Ensure: Secuencia de movimientos que resuelve el problema

```
1: procedure HANOI( $n$ , origen, destino, auxiliar)
2:   if  $n = 1$  then
3:     Mover disco de origen a destino    //Caso base
4:   else
5:     HANOI( $n - 1$ , origen, auxiliar, destino)
6:     Mover disco de origen a destino
7:     HANOI( $n - 1$ , auxiliar, destino, origen)
8:   end if
9: end procedure
```

Ejemplo 3: Torres de Hanoi - Análisis de Complejidad I

Recurrencia: Sea $M(n)$ el número de movimientos necesarios para n discos:

$$M(n) = 2M(n-1) + 1, \quad M(1) = 1$$

Resolución: como

$$b^n p(n) = 1 \Rightarrow b = 1 \wedge p(n) = 1 \Rightarrow \deg(p) = 0.$$

Entonces

$$(r-2)(r-1)^{0+1} = 0 \Rightarrow r_1 = 2 \wedge r_2 = 1,$$

y la solución es $M(n) = c_1 2^n + c_2$.

Usando $M(1) = 1$ y $M(1) = 2M(0) + 1 = 1 \Rightarrow M(0) = 0$:

$$c_1 + c_2 = 0 \wedge 2c_1 + c_2 = 1 \Rightarrow c_2 = 1 \wedge c_1 = -1$$

Luego:

$$M(n) = 2^n - 1 = O(2^n)$$

Ejemplo 3: Torres de Hanoi - Análisis de Complejidad II

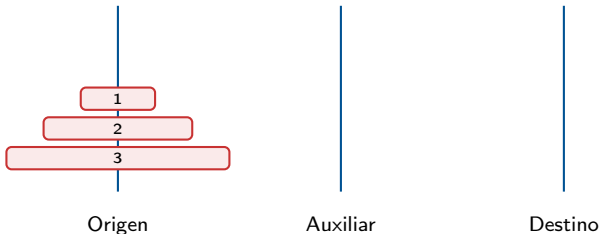
Resumen de Complejidades

Temporal	Espacial
$O(2^n)$	$O(n)$

Implicaciones Prácticas

- ▶ Para $n = 64$: $2^{64} - 1 \approx 1.84 \times 10^{19}$ movimientos
- ▶ A 1 movimiento por segundo: ≈ 584 mil millones de años
- ▶ A 1 millón de movimientos por segundo: ≈ 584 mil años
- ▶ Ejemplo clásico de complejidad exponencial que, a diferencia de Fibonacci, es **inherente al problema**: no existe solución más eficiente

Ejemplo 3: Torres de Hanoi - Ilustración para $n = 3$



Solución para $n = 3$ (7 movimientos)

1. Mover disco 1 de Origen a Destino
2. Mover disco 2 de Origen a Auxiliar
3. Mover disco 1 de Destino a Auxiliar
4. Mover disco 3 de Origen a Destino
5. Mover disco 1 de Auxiliar a Origen
6. Mover disco 2 de Auxiliar a Destino
7. Mover disco 1 de Origen a Destino

Ejemplo 4: Búsqueda Binaria Recursiva - Planteamiento del Problema

Problema de Búsqueda en un Arreglo Ordenado

Dado un arreglo ordenado de n elementos (en orden ascendente) y un valor objetivo *target*, determinar si *target* existe en el arreglo y, en caso afirmativo, devolver su índice.

Precondición: El arreglo está ordenado. **Postcondición:** Se devuelve un índice i tal que $arr[i] = target$, o -1 si *target* no está presente.

Ejemplo 4: Búsqueda Binaria Recursiva - Algoritmo

Algoritmo 9 Búsqueda Binaria Recursiva

Require: Arreglo ordenado arr , índices $left$, $right$, valor $target$

Ensure: Índice de $target$ en arr o -1 si no existe

```
1: procedure BINARY_SEARCH( $arr, left, right, target$ )
2:   if  $left > right$  then
3:     return  $-1$     //No encontrado
4:   end if
5:    $mid \leftarrow \lfloor (left + right)/2 \rfloor$ 
6:   if  $arr[mid] = target$  then
7:     return  $mid$     //Caso base: encontrado
8:   else if  $arr[mid] > target$  then
9:     return BINARY_SEARCH( $arr, left, mid - 1, target$ )
10:  else
11:    return BINARY_SEARCH( $arr, mid + 1, right, target$ )
12:  end if
13: end procedure
```

Ejemplo 4: Búsqueda Binaria - Análisis de Complejidad I

Recurrencia: Sea $T(n)$ el tiempo de ejecución para un arreglo de n elementos:

$$T(n) = T\left(\frac{n}{2}\right) + O(1)$$

Resolución: La ecuación característica asociada a $T(n) = T(n/2)$ no es lineal con coeficientes constantes en la forma estándar. Aplicamos el Teorema Maestro:

$$a = 1, b = 2, f(n) = O(1) = \Theta(n^0)$$

$$\log_b a = \log_2 1 = 0$$

Como $f(n) = \Theta(n^{\log_b a}) = \Theta(1)$, estamos en el Caso 2 del Teorema Maestro:

$$T(n) = \Theta(n^{\log_b a} \log n) = \Theta(\log n)$$

Ejemplo 4: Búsqueda Binaria - Análisis de Complejidad II

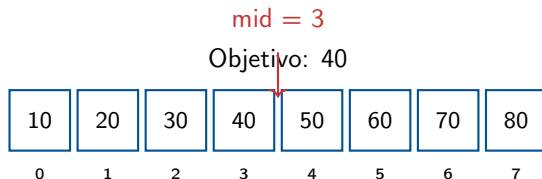
Resumen de Complejidades

Temporal	Espacial
$O(\log n)$	$O(\log n)$

Eficiencia del Enfoque

- ▶ Para $n = 10^9$ (mil millones), solo ≈ 30 comparaciones
- ▶ Para $n = 10^{12}$ (un billón), solo ≈ 40 comparaciones
- ▶ Óptimo para búsqueda en arreglos ordenados (demostrable por cota inferior $\Omega(\log n)$)
- ▶ La versión iterativa mejora la complejidad espacial a $O(1)$

Ejemplo 4: Búsqueda Binaria - Visualización



Ejemplo de Ejecución

Búsqueda del valor 40 en un arreglo ordenado de 8 elementos:

1. left = 0, right = 7, mid = 3
2. arr[3] = 40 → ¡encontrado en el primer intento!

En el peor caso, para 8 elementos se requieren $\log_2 8 = 3$ comparaciones.

Ejemplo 5: Multiplicación de Strassen - Planteamiento del Problema

Problema de la Multiplicación de Matrices

Dadas dos matrices cuadradas A y B de tamaño $n \times n$ (con n potencia de 2, para simplificar), calcular su producto $C = A \times B$ definido por:

$$C[i][j] = \sum_{k=1}^n A[i][k] \cdot B[k][j]$$

La multiplicación matricial es fundamental en computación científica, gráficos, inteligencia artificial, procesamiento de imágenes, etc.

Ejemplo 5: Multiplicación de Strassen - Motivación I

Limitaciones del Algoritmo Clásico

El algoritmo clásico de multiplicación de matrices (triple bucle anidado) tiene complejidad $O(n^3)$.

Para $n = 1000$:

- ▶ Número de multiplicaciones: 10^9 (mil millones)
- ▶ En un procesador de 3 GHz, asumiendo 1 multiplicación por ciclo: ≈ 0.33 segundos
- ▶ Pero en la práctica, con accesos a memoria y overhead, puede ser mucho mayor
- ▶ Para $n = 10^4$: 10^{12} operaciones $\rightarrow$ horas o días

Ejemplo 5: Multiplicación de Strassen - Motivación II

Objetivo de Strassen (1969)

Demostrar que $O(n^3)$ **no** es óptimo, encontrando un algoritmo asintóticamente más rápido mediante un enfoque recursivo divide y vencerás.

Ejemplo 5: Multiplicación de Strassen - Idea I

1. Dividir cada matriz en 4 submatrices de tamaño $n/2 \times n/2$:

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}$$

2. Calcular 7 productos de matrices (en lugar de 8):

$$M_1 = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$M_2 = (A_{21} + A_{22})B_{11}$$

$$M_3 = A_{11}(B_{12} - B_{22})$$

$$M_4 = A_{22}(B_{21} - B_{11})$$

$$M_5 = (A_{11} + A_{12})B_{22}$$

$$M_6 = (A_{21} - A_{11})(B_{11} + B_{12})$$

$$M_7 = (A_{12} - A_{22})(B_{21} + B_{22})$$

Ejemplo 5: Multiplicación de Strassen - Idea II

3. Combinar mediante sumas y restas para obtener las 4 submatrices resultado.

Ejemplo 5: Multiplicación de Strassen - Algoritmo

Algoritmo 10 Multiplicación de Strassen

Require: Matrices A, B de tamaño $n \times n$, donde n es potencia de 2

Ensure: Matriz $C = A \times B$

- ```
1: if $n = 1$ then
2: return $A \cdot B$ //Caso base: multiplicación escalar
3: end if
4: Dividir A en 4 submatrices de tamaño $n/2 \times n/2$:
```

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}$$

- ```
5: Dividir  $B$  en 4 submatrices de tamaño  $n/2 \times n/2$ :
```

$$B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}$$

- ```
6: Calcular 7 productos de matrices:
7: $M_1 \leftarrow \text{STRASSEN}(A_{11} + A_{22}, B_{11} + B_{22})$
8: $M_2 \leftarrow \text{STRASSEN}(A_{21} + A_{22}, B_{11})$
9: $M_3 \leftarrow \text{STRASSEN}(A_{11}, B_{12} - B_{22})$
10: $M_4 \leftarrow \text{STRASSEN}(A_{22}, B_{21} - B_{11})$
11: $M_5 \leftarrow \text{STRASSEN}(A_{11} + A_{12}, B_{22})$
12: $M_6 \leftarrow \text{STRASSEN}(A_{21} - A_{11}, B_{11} + B_{12})$
13: $M_7 \leftarrow \text{STRASSEN}(A_{12} - A_{22}, B_{21} + B_{22})$
14: Combinar resultados mediante sumas y restas:
15: $C_{11} \leftarrow M_1 + M_4 - M_5 + M_7$
16: $C_{12} \leftarrow M_3 + M_5$
17: $C_{21} \leftarrow M_2 + M_4$
18: $C_{22} \leftarrow M_1 - M_2 + M_3 + M_6$
19: return $C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$
```
-



## Ejemplo 5: Multiplicación de Strassen - Análisis de Complejidad I

**Recurrencia:** Sea  $T(n)$  el tiempo de ejecución para matrices  $n \times n$ :

$$T(n) = 7T\left(\frac{n}{2}\right) + \Theta(n^2)$$

**Resolución (Teorema Maestro):**

$$a = 7, b = 2, f(n) = \Theta(n^2)$$

$$\log_b a = \log_2 7 \approx 2.807$$

$$n^{\log_b a} = n^{\log_2 7}$$

Comparando  $f(n) = n^2$  con  $n^{\log_2 7}$ : como  $\log_2 7 > 2$ , tenemos  $f(n) = O(n^{\log_2 7 - \varepsilon})$  para  $\varepsilon = \log_2 7 - 2 > 0$ . Estamos en el Caso 1:

$$T(n) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$$

## Ejemplo 5: Multiplicación de Strassen - Análisis de Complejidad II

### Resumen de Complejidades

| Algoritmo | Complejidad Temporal |
|-----------|----------------------|
| Clásico   | $O(n^3)$             |
| Strassen  | $O(n^{2.807})$       |

### Comparación para $n = 1000$

- ▶ Clásico:  $\approx 10^9$  operaciones
- ▶ Strassen:  $\approx 10^{8.42} \approx 2.6 \times 10^8$  operaciones
- ▶ Factor de mejora:  $\approx 3.8\times$

## Ejemplo 5: Multiplicación de Strassen - Significado

### Importancia Histórica y Teórica

- ▶ Primera demostración de que  $O(n^3)$  **no** es el límite inferior para multiplicación de matrices
- ▶ Reduce el número de multiplicaciones de 8 a 7 por nivel recursivo
- ▶ Aunque las constantes ocultas son grandes, asintóticamente supera al algoritmo clásico para  $n$  suficientemente grande
- ▶ Inspiró una línea de investigación que ha producido algoritmos cada vez más rápidos:
  - ▶ Coppersmith-Winograd (1990):  $O(n^{2.376})$
  - ▶ Algoritmos modernos (2020):  $O(n^{2.372})$
- ▶ Ilustra cómo la reducción del número de subproblemas ( $a$ ) mejora drásticamente la complejidad

# Divide y Conquista

# Divide y Conquista - Fundamentos I

## Descripción del Paradigma

El paradigma de *divide y conquista* constituye una técnica de diseño algorítmico fundamental. Permite transformar problemas complejos en instancias más manejables mediante descomposición recursiva.

# Divide y Conquista - Fundamentos II

## Estructura del Paradigma

- ▶ **Dividir:** Descomponer el problema en subproblemas de menor tamaño e independientes (o lo más independientes posible).
- ▶ **Conquistar:** Resolver cada subproblema recursivamente. Cuando los subproblemas alcanzan un tamaño suficientemente pequeño (caso base), se resuelven directamente.
- ▶ **Combinar:** Reconstruir la solución global a partir de las soluciones de los subproblemas.

# Divide y Conquista - Fundamentos III

## Características Clave

- ▶ **Recursividad inherente:** La estructura del algoritmo refleja la descomposición del problema.
- ▶ **Independencia:** Los subproblemas deben ser resolubles de forma independiente para permitir paralelización.
- ▶ **Equilibrio:** Idealmente, los subproblemas tienen tamaño similar para optimizar la complejidad.

# Divide y Conquista - Ejemplos Aplicados I

## Aplicaciones Notables

- ▶ **Ordenamiento:** MergeSort, QuickSort ( $O(n \log n)$ ).
- ▶ **Búsqueda:** Búsqueda binaria ( $O(\log n)$ ).
- ▶ **Matrices:** Multiplicación de Strassen ( $O(n^{2.807})$ ).
- ▶ **Geometría:** Par de puntos más cercano ( $O(n \log n)$ ).
- ▶ **Transformadas:** FFT (Transformada Rápida de Fourier,  $O(n \log n)$ ).
- ▶ **Árboles:** Recorridos, construcción de árboles balanceados.



# Divide y Conquista - Ejemplos Aplicados II

## Ventajas del Paradigma

- ▶ **Paralelización natural:** Los subproblemas independientes pueden ejecutarse en paralelo.
- ▶ **Eficiencia:** Permite alcanzar cotas óptimas en muchos problemas (ej. ordenamiento, búsqueda).
- ▶ **Análisis sistemático:** La complejidad se modela mediante el Teorema Maestro, facilitando el análisis.
- ▶ **Claridad conceptual:** La estructura divide-conquista-combina proporciona una organización clara del algoritmo.

# Ejemplo: Par de puntos más cercano - Planteamiento del Problema I

## Problema

Dado un conjunto de  $n$  puntos en el plano, determinar el par de puntos cuya distancia euclidiana es mínima.

## Definición Formal

**Entrada:** Conjunto  $P = \{p_1, p_2, \dots, p_n\}$  con  $p_i = (x_i, y_i) \in \mathbb{R}^2$ .

**Salida:** Par de puntos  $(p_i, p_j)$  con  $i \neq j$  que minimiza

$$d(p_i, p_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

y el valor de la distancia mínima  $\delta$ .

# Ejemplo: Par de puntos más cercano - Planteamiento del Problema II

## Aplicaciones

- ▶ Geometría computacional
- ▶ Sistemas de información geográfica (GIS)
- ▶ Análisis de proximidad en bases de datos espaciales
- ▶ Gráficos por computador (detección de colisiones)

## Ejemplo: Par de puntos más cercano - Estrategia Divide y Conquista I

1. **Preprocesamiento:** Ordenar los puntos por coordenada  $x$  (obteniendo  $P_x$ ) y por coordenada  $y$  (obteniendo  $P_y$ ).
2. **Dividir:** Dividir el conjunto en dos mitades iguales usando la mediana de las coordenadas  $x$ .
3. **Conquistar:** Resolver recursivamente el problema en cada mitad, obteniendo las distancias mínimas  $\delta_L$  y  $\delta_R$ .
4. **Combinar:**
  - ▶ Sea  $\delta = \min(\delta_L, \delta_R)$ .
  - ▶ Considerar solo los puntos en una franja vertical de ancho  $2\delta$  centrada en la división.
  - ▶ Ordenar estos puntos por  $y$  y verificar pares candidatos (a lo sumo 7 vecinos por punto).

## Ejemplo: Par de puntos más cercano - Estrategia Divide y Conquista II

### Justificación de la Franja

Cualquier par con distancia menor que  $\delta$  debe tener sus puntos en la franja central; fuera de ella, la diferencia en  $x$  ya supera  $\delta$ .

## Ejemplo: Par de puntos más cercano - Algoritmo

---

### Algoritmo 11 ParMásCercano( $P_x, P_y$ )

---

**Require:**  $P_x$ : puntos ordenados por coordenada  $x$ ;  $P_y$ : puntos ordenados por coordenada  $y$

**Ensure:** Distancia mínima  $\delta$  y el par de puntos correspondiente

```
1: if $|P_x| \leq 3$ then
2: return fuerza bruta sobre P_x //Caso base
3: end if
4: $Q_x, R_x \leftarrow$ división de P_x en mitades por la mediana de x
5: $Q_y, R_y \leftarrow$ partición de P_y según pertenencia a Q_x o R_x
6: $(\delta_L, \text{par}_L) \leftarrow \text{PARMÁSCERCANO}(Q_x, Q_y)$
7: $(\delta_R, \text{par}_R) \leftarrow \text{PARMÁSCERCANO}(R_x, R_y)$
8: $\delta \leftarrow \min(\delta_L, \delta_R)$
9: $S \leftarrow$ puntos en P_y con $|x - x_{\text{med}}| < \delta$ //Franja central
10: for $i \leftarrow 0$ to $|S| - 1$ do
11: for $j \leftarrow i + 1$ to $\min(i + 7, |S| - 1)$ do
12: Actualizar δ y el par si $d(S[i], S[j]) < \delta$
13: end for
14: end for
15: return δ y el par correspondiente
```

---

## Ejemplo: Par de puntos más cercano - Análisis de Complejidad I

**Recurrencia:** Sea  $T(n)$  el tiempo de ejecución para  $n$  puntos:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

**Resolución:** Aplicando el Teorema Maestro:

$$a = 2, b = 2, f(n) = O(n)$$

$$\log_b a = \log_2 2 = 1$$

$$n^{\log_b a} = n^1 = n$$

Como  $f(n) = \Theta(n) = \Theta(n^{\log_b a})$ , estamos en el Caso 2:

$$T(n) = \Theta(n \log n)$$

## Ejemplo: Par de puntos más cercano - Análisis de Complejidad II

### Resumen de Complejidades

| Temporal      | Espacial |
|---------------|----------|
| $O(n \log n)$ | $O(n)$   |

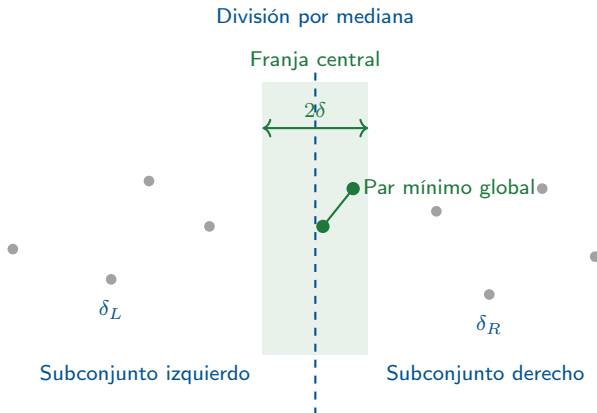


# Ejemplo: Par de puntos más cercano - Análisis de Complejidad III

## Análisis Detallado

- ▶ Ordenación inicial:  $O(n \log n)$  (una vez, fuera de la recurrencia)
- ▶ Cada nivel recursivo:  $O(n)$  para particionar  $P_y$  y procesar la franja
- ▶ Número de niveles:  $O(\log n)$
- ▶ En la franja, cada punto se compara con a lo sumo 7 vecinos (demostrable geométricamente)

# Ejemplo: Par de puntos más cercano - Ilustración



# Ejemplo: Par de puntos más cercano - Observaciones Clave I

## Interpretación - Ilustración

La solución global se construye combinando los óptimos locales de cada mitad ( $\delta_L$  y  $\delta_R$ ) y verificando solo los puntos dentro de la franja central de ancho  $2\delta$ , donde  $\delta = \min(\delta_L, \delta_R)$ . La comparación se realiza ordenando estos puntos por  $y$  y examinando un número constante de vecinos (máximo 7).

## Por qué funciona

- ▶ La recurrencia  $T(n) = 2T(n/2) + O(n)$  es característica de divide y conquista balanceado.
- ▶ La poda de la franja central reduce drásticamente el número de comparaciones en la fase de combinación.
- ▶ La cota de 7 vecinos por punto en la franja es una propiedad geométrica clave que garantiza la linealidad de la combinación.

# Ejemplo: Par de puntos más cercano - Observaciones Clave II

## Comparación con Fuerza Bruta

| Algoritmo          | Complejidad   | $n = 10^6$                          |
|--------------------|---------------|-------------------------------------|
| Fuerza bruta       | $O(n^2)$      | $10^{12}$ operaciones               |
| Divide y conquista | $O(n \log n)$ | $\approx 2 \times 10^7$ operaciones |

**Mejora:** Factor de  $\approx 50,000\times$  para  $n = 10^6$ .

## Variantes y Extensiones

- ▶ Par de puntos más cercano en 3D (complejidad  $O(n \log n)$  con técnicas similares)
- ▶ Problemas relacionados: diámetro de un conjunto, par más lejano, envolvente convexa

# Programación Dinámica

## Descripción del Paradigma

La *programación dinámica* constituye una técnica fundamental en la resolución de problemas de optimización. Permite abordar eficientemente problemas que presentan subproblemas solapados y cumplen el principio de optimalidad, evitando la recomputación mediante almacenamiento de resultados intermedios.

## Principio de Optimalidad de Bellman

Una solución óptima de un problema contiene en su interior soluciones óptimas para los subproblemas que la constituyen. Formalmente, cualquier subpolítica de una política óptima debe ser también óptima para el subproblema correspondiente.

Este principio permite construir soluciones globales a partir de soluciones óptimas locales, garantizando la corrección del enfoque.

## Características Clave

- ▶ **Subproblemas solapados:** Los mismos subproblemas aparecen repetidamente en la recursión.
- ▶ **Estructura óptima:** La solución óptima global se construye a partir de soluciones óptimas de subproblemas.
- ▶ **Memoización:** Almacenamiento de resultados para evitar recomputación.



## Enfoques de Implementación

### ▶ **Top-down (Memoización):**

- ▶ Enfoque recursivo con almacenamiento en tabla
- ▶ Solo resuelve los subproblemas necesarios
- ▶ Más intuitivo, pero puede tener overhead de recursión

### ▶ **Bottom-up (Tabular):**

- ▶ Enfoque iterativo que construye la tabla desde casos base
- ▶ Resuelve todos los subproblemas hasta el deseado
- ▶ Generalmente más eficiente en espacio y tiempo

# Ejemplo 1: Fibonacci - Planteamiento del Problema I

## Problema: Cálculo de números de Fibonacci

La sucesión de Fibonacci se define mediante la relación de recurrencia:

$$F(0) = 0, \quad F(1) = 1, \quad F(n) = F(n-1) + F(n-2) \text{ para } n \geq 2.$$

**Objetivo:** Dado un número entero no negativo  $n$ , calcular el  $n$ -ésimo número de Fibonacci  $F(n)$  de manera eficiente.

# Ejemplo 1: Fibonacci - Planteamiento del Problema II

## Limitación del Enfoque Recursivo Puro

El algoritmo recursivo puro tiene complejidad exponencial  $O(\phi^n)$  debido a la recomputación masiva de los mismos subproblemas. La programación dinámica elimina esta redundancia.

# Ejemplo 1: Fibonacci - Algoritmo con Memoización

---

## Algoritmo 12 Fibonacci con Memoización (Top-down)

---

**Require:**  $n \geq 0$  entero

**Ensure:**  $F(n)$  (n-ésimo número de Fibonacci)

```
1: procedure FIBONACCI(n)
2: $memo \leftarrow$ arreglo de tamaño $n + 1$ inicializado con -1
3: return FIB_MEMO($n, memo$)
4: end procedure
5: procedure FIB_MEMO($n, memo$)
6: if $n \leq 1$ then
7: return n //Casos base
8: end if
9: if $memo[n] \neq -1$ then
10: return $memo[n]$ //Resultado ya calculado
11: end if
12: $memo[n] \leftarrow$ FIB_MEMO($n - 1, memo$) + FIB_MEMO($n - 2, memo$)
13: return $memo[n]$
14: end procedure
```

---

## Ejemplo 1: Fibonacci con Memoización - Análisis de Complejidad I

### Análisis de Complejidad

Cada subproblema  $F(i)$ , con  $0 \leq i \leq n$ , se resuelve exactamente una vez. El costo de resolver cada subproblema (una vez que sus dependencias están resueltas) es  $O(1)$ .

$$T(n) = O(n), \quad M(n) = O(n)$$

La complejidad espacial incluye el arreglo de memoización de tamaño  $n + 1$  y la pila de recursión (profundidad  $n$  en el peor caso).

# Ejemplo 1: Fibonacci con Memoización - Análisis de Complejidad II

## Resumen de Complejidades

| Temporal | Espacial |
|----------|----------|
| $O(n)$   | $O(n)$   |

# Ejemplo 1: Fibonacci con Memoización - Análisis de Complejidad III

## Mejora Respecto al Enfoque Recursivo Puro

- ▶ **Recursivo puro:**  $O(\phi^n)$  exponencial
- ▶ **Con memoización:**  $O(n)$  lineal
- ▶ **Factor de mejora para  $n = 50$ :** de  $\approx 2 \times 10^{10}$  a 50 operaciones
- ▶ **Trade-off:** Se sacrifica espacio ( $O(n)$ ) para ganar eficiencia temporal

## Ejemplo 2: Coeficiente Binomial - Planteamiento del Problema I

### Problema: Cálculo del coeficiente binomial

El coeficiente binomial  $\binom{n}{k}$  representa el número de formas de elegir  $k$  elementos de un conjunto de  $n$  elementos, sin importar el orden. Se define como:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}, \quad 0 \leq k \leq n$$

**Objetivo:** Dados  $n$  y  $k$  enteros no negativos con  $k \leq n$ , calcular  $\binom{n}{k}$  de manera eficiente.



## Ejemplo 2: Coeficiente Binomial - Planteamiento del Problema II

### Aplicaciones

- ▶ Combinatoria y probabilidad
- ▶ Desarrollo del binomio de Newton
- ▶ Algoritmos de conteo
- ▶ Triángulo de Pascal

## Ejemplo 2: Coeficiente Binomial - Estrategia de Programación Dinámica I

### Identidad Recursiva Fundamental

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$$

con casos base:

$$\binom{n}{0} = \binom{n}{n} = 1$$

## Ejemplo 2: Coeficiente Binomial - Estrategia de Programación Dinámica II

### Principio de Optimalidad

El número de formas de elegir  $k$  elementos de  $n$  puede descomponerse en:

- ▶ Casos que incluyen un elemento fijo:  $\binom{n-1}{k-1}$
- ▶ Casos que excluyen ese elemento:  $\binom{n-1}{k}$

Esta descomposición conduce a una solución de programación dinámica construyendo el triángulo de Pascal.

## Ejemplo 2: Coeficiente Binomial - Algoritmo

---

### Algoritmo 13 Cálculo de Coeficiente Binomial (Programación Dinámica)

---

**Require:** Enteros  $n, k$  con  $0 \leq k \leq n$

**Ensure:**  $\binom{n}{k}$

```
1: procedure BINOMIAL_COEFF(n, k)
2: $C \leftarrow$ matriz de tamaño $(n + 1) \times (k + 1)$ inicializada con 0
3: for $i = 0$ to n do
4: $C[i][0] \leftarrow 1$ // $\binom{i}{0} = 1$
5: end for
6: for $i = 0$ to k do
7: $C[i][i] \leftarrow 1$ // $\binom{i}{i} = 1$ para $i \leq k$
8: end for
9: for $i = 2$ to n do
10: for $j = 1$ to $\min(i - 1, k)$ do
11: $C[i][j] \leftarrow C[i - 1][j - 1] + C[i - 1][j]$
12: end for
13: end for
14: return $C[n][k]$
15: end procedure
```

---

## Ejemplo 2: Coeficiente Binomial - Análisis de Complejidad I

### Análisis de Complejidad

El algoritmo construye una tabla de tamaño  $(n + 1) \times (k + 1)$ . Cada entrada se calcula en tiempo constante  $O(1)$ .

$$T(n, k) = O(n \cdot k), \quad M(n, k) = O(n \cdot k)$$

En el peor caso, cuando  $k \approx n/2$ , la complejidad es  $O(n^2)$ .

## Ejemplo 2: Coeficiente Binomial - Análisis de Complejidad II

### Resumen de Complejidades

| Temporal | Espacial |
|----------|----------|
| $O(nk)$  | $O(nk)$  |

## Ejemplo 2: Coeficiente Binomial - Análisis de Complejidad III

### Propiedades Adicionales (Acotaciones)

- ▶ Si  $2k \leq n$ :

$$2^k \binom{n-k}{0} \leq \binom{n}{k} \leq 2^k \binom{n-k}{k}$$

- ▶ Si  $2k > n$ :

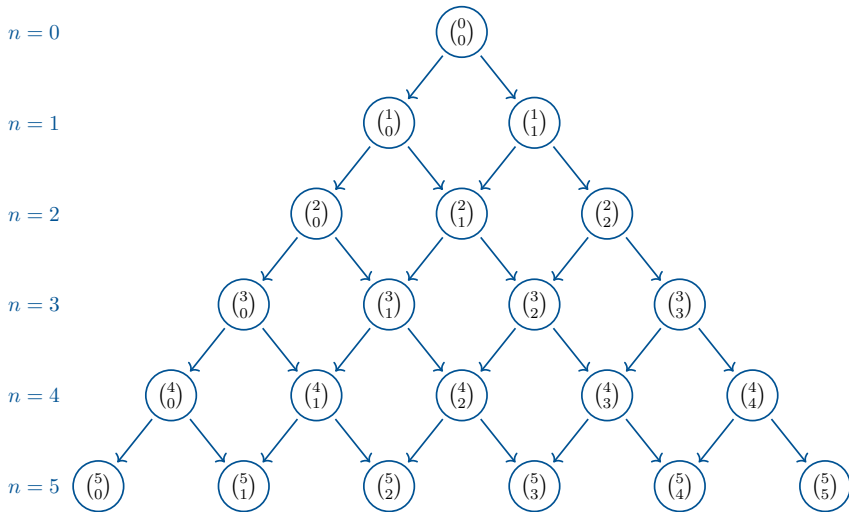
$$2^k \binom{n-k}{k} \leq \binom{n}{k} \leq 2^k \binom{n-k}{0}$$

- ▶ Para  $k = \lfloor n/2 \rfloor$ :

$$\binom{n}{k} = \Theta\left(2^{n/2}\right)$$

Estas acotaciones muestran el crecimiento exponencial de los coeficientes binomiales centrales.

## Ejemplo 2: Coeficiente Binomial - Triángulo de Pascal





## Ejemplo 3: Camino Mínimo en Grilla - Planteamiento del Problema I

### Problema: Camino de costo mínimo en una grilla

Dada una grilla  $m \times n$  con costos no negativos en cada celda, encontrar el costo mínimo para ir desde la esquina superior izquierda  $(0, 0)$  hasta la esquina inferior derecha  $(m-1, n-1)$ , con la restricción de que solo se permite moverse hacia la derecha o hacia abajo.

## Ejemplo 3: Camino Mínimo en Grilla - Planteamiento del Problema II

### Definición Formal

**Entrada:** Matriz *grid* de tamaño  $m \times n$  con  $grid[i][j] \geq 0$ .

**Salida:** El costo mínimo de un camino desde  $(0, 0)$  hasta  $(m - 1, n - 1)$  donde cada paso es  $(i, j) \rightarrow (i + 1, j)$  (abajo) o  $(i, j) \rightarrow (i, j + 1)$  (derecha).

**Costo del camino:** Suma de los valores de las celdas visitadas, incluyendo origen y destino.

## Ejemplo 3: Camino Mínimo en Grilla - Planteamiento del Problema III

### Aplicaciones

- ▶ Ruta óptima en mapas de costos
- ▶ Planificación de trayectorias
- ▶ Alineamiento de secuencias en bioinformática
- ▶ Problemas de optimización en redes

## Ejemplo 3: Camino Mínimo en Grilla - Estrategia de Programación Dinámica I

### Principio de Optimalidad

Cualquier subcamino de una solución óptima también es óptimo. Para llegar a la celda  $(i, j)$ , el camino óptimo debe provenir de:

- ▶ La celda superior  $(i - 1, j)$  (si existe)
- ▶ La celda izquierda  $(i, j - 1)$  (si existe)

## Ejemplo 3: Camino Mínimo en Grilla - Estrategia de Programación Dinámica II

### Ecuación de Recurrencia

Definimos  $dp[i][j]$  como el costo mínimo para llegar desde  $(0, 0)$  hasta  $(i, j)$ :

$$dp[i][j] = \begin{cases} grid[0][0] & \text{si } i = 0, j = 0 \\ dp[i-1][0] + grid[i][0] & \text{si } j = 0, i > 0 \\ dp[0][j-1] + grid[0][j] & \text{si } i = 0, j > 0 \\ \min(dp[i-1][j], dp[i][j-1]) + grid[i][j] & \text{si } i > 0, j > 0 \end{cases}$$

## Ejemplo 3: Camino Mínimo en Grilla - Algoritmo

---

### Algoritmo 14 Camino Mínimo en Grilla (Programación Dinámica)

---

**Require:** Matriz *grid* de tamaño  $m \times n$  con costos no negativos

**Ensure:** Costo mínimo desde  $(0, 0)$  hasta  $(m - 1, n - 1)$

```
1: procedure SHORTEST_PATH(grid)
2: $m \leftarrow$ número de filas
3: $n \leftarrow$ número de columnas
4: $dp \leftarrow$ matriz de tamaño $m \times n$ inicializada con 0
5: $dp[0][0] \leftarrow grid[0][0]$ //Caso base
6: for $i = 1$ to $m - 1$ do
7: $dp[i][0] \leftarrow dp[i - 1][0] + grid[i][0]$ //Primera columna
8: end for
9: for $j = 1$ to $n - 1$ do
10: $dp[0][j] \leftarrow dp[0][j - 1] + grid[0][j]$ //Primera fila
11: end for
12: for $i = 1$ to $m - 1$ do
13: for $j = 1$ to $n - 1$ do
14: $dp[i][j] \leftarrow \min(dp[i - 1][j], dp[i][j - 1]) + grid[i][j]$
15: end for
16: end for
17: return $dp[m - 1][n - 1]$
18: end procedure
```

---

## Ejemplo 3: Camino Mínimo en Grilla - Análisis de Complejidad I

### Análisis de Complejidad

El algoritmo procesa cada celda de la grilla exactamente una vez, realizando operaciones constantes por celda.

$$T(m, n) = O(m \cdot n), \quad M(m, n) = O(m \cdot n)$$

Donde  $m$  es el número de filas y  $n$  el número de columnas.

## Ejemplo 3: Camino Mínimo en Grilla - Análisis de Complejidad II

### Resumen de Complejidades

| Temporal | Espacial |
|----------|----------|
| $O(mn)$  | $O(mn)$  |



## Ejemplo 3: Optimización de Espacio

La complejidad espacial puede reducirse a  $O(n)$  (o  $O(m)$ ) y  $M(m, n) = O(n)$ , observando que solo se necesita la fila anterior para calcular la fila actual:

---

### Algoritmo 15 Optimización Espacial

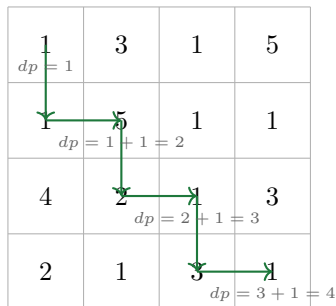
---

```
1: prev $\leftarrow$ arreglo de tamaño n inicializado con 0
2: prev[0] $\leftarrow$ grid[0][0]
3: for $j = 1$ to $n - 1$ do
4: prev[j] $\leftarrow$ prev[$j - 1$] + grid[0][j]
5: end for
6: for $i = 1$ to $m - 1$ do
7: curr $\leftarrow$ arreglo de tamaño n
8: curr[0] $\leftarrow$ prev[0] + grid[i][0]
9: for $j = 1$ to $n - 1$ do
10: curr[j] $\leftarrow$ min(prev[j], curr[$j - 1$]) + grid[i][j]
11: end for
12: prev $\leftarrow$ curr
13: end for
14: return prev[$n - 1$]
```

---

## Ejemplo 3: Camino Mínimo en Grilla - Ilustración

Inicio



Fin

## Ejemplo 3: Camino Mínimo en Grilla - Ilustración I

### Interpretación

El camino resaltado en **verde** muestra la ruta de costo mínimo. Los valores  $dp[i][j]$  representan el costo acumulado mínimo para llegar a cada celda. En este ejemplo, el costo total es  $1 + 1 + 1 + 1 + 1 = 4$ .

# Algoritmos Geométricos

# Algoritmos Geométricos - Fundamentos I

## Descripción General

Los algoritmos geométricos se utilizan para resolver problemas que involucran objetos geométricos como puntos, líneas, polígonos y poliedros. Requieren consideraciones especiales para manejar la precisión numérica y la complejidad de las estructuras geométricas.

# Algoritmos Geométricos - Fundamentos II

## Problemas Geométricos Clásicos

- ▶ **Par de puntos más cercano:** Encontrar los dos puntos más cercanos en un conjunto.
- ▶ **Envolvente convexa:** Determinar el polígono convexo más pequeño que contiene todos los puntos.
- ▶ **Intersección de segmentos:** Detectar si dos segmentos de recta se intersectan.
- ▶ **Diagrama de Voronoi:** Particionar el plano en regiones según la proximidad a puntos dados.
- ▶ **Localización de puntos:** Determinar si un punto está dentro de un polígono.
- ▶ **Área de polígonos:** Calcular el área de un polígono simple.

# Consideraciones de Precisión Numérica I

## Problemas de Precisión

- ▶ **Errores de redondeo:** En operaciones con números reales en punto flotante.
- ▶ **Comparaciones exactas:** Dificultad al comparar distancias o posiciones debido a la precisión finita.
- ▶ **Casos degenerados:** Puntos colineales, segmentos superpuestos, puntos coincidentes.
- ▶ **Inestabilidad numérica:** Pequeñas variaciones pueden cambiar el resultado geométrico.

# Consideraciones de Precisión Numérica II

## Soluciones Comunes

- ▶ Uso de aritmética de precisión arbitraria para cálculos críticos.
- ▶ Tolerancias numéricas bien definidas (épsilon) en comparaciones.
- ▶ Algoritmos robustos que manejen explícitamente casos degenerados.
- ▶ Empleo de números racionales cuando sea posible.



# Ejemplos: Algoritmos Geométricos

# Ejemplo 1: Área de Polígono - Planteamiento del Problema I

## Problema

Dado un polígono simple definido por una secuencia ordenada de  $n$  vértices  $(x_0, y_0), (x_1, y_1), \dots, (x_{n-1}, y_{n-1})$ , calcular su área.

## Definición Formal

**Entrada:** Lista de vértices  $V = [(x_0, y_0), (x_1, y_1), \dots, (x_{n-1}, y_{n-1})]$  en orden (horario o antihorario).

**Salida:** El área del polígono, un número real no negativo.

## Restricciones

- ▶ El polígono debe ser **simple** (sin autointersecciones).
- ▶ Los vértices deben estar en **orden** (horario o antihorario).
- ▶ No se requiere convexidad.

## Ejemplo 1: Área de Polígono - Planteamiento del Problema II

### Objetivo

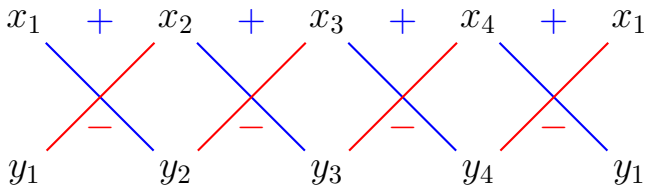
Obtener el área exacta mediante una fórmula eficiente que evite triangulación explícita.

Para calcular el área de un polígono simple utilizamos la **Fórmula de Shoelace**, también conocida como *fórmula del cordón* o *fórmula de Gauss*. Este algoritmo calcula el área a partir de los pares de coordenadas de sus vértices.

$$A = \frac{1}{2} \left| \sum_{i=0}^{n-1} (x_i y_{i+1} - x_{i+1} y_i) \right|, \quad \text{donde } (x_n, y_n) = (x_0, y_0).$$

## Ejemplo 1: Área de Polígono - Planteamiento del Problema III

La razón del nombre "cordón" se entiende mejor observando el siguiente diagrama para  $n = 4$ :



Donde las líneas azules representan los productos positivos ( $x_i y_{i+1}$ ) y las rojas los productos negativos ( $y_i x_{i+1}$ ).

## Ejemplo 1: Área de Polígono - Algoritmo (Fórmula de Shoelace)

---

### Algoritmo 16 Área de Polígono - Fórmula de Shoelace

---

**Require:** *vertices*: lista de vértices  $(x_i, y_i)$  en orden

**Ensure:** Área del polígono

```
1: procedure POLYGON_AREA(vertices)
2: $n \leftarrow |vertices|$
3: $area \leftarrow 0$
4: for $i = 0$ to $n - 1$ do
5: $j \leftarrow (i + 1) \bmod n$ //Índice del siguiente vértice
6: $area \leftarrow area + vertices[i].x \cdot vertices[j].y$
7: $area \leftarrow area - vertices[j].x \cdot vertices[i].y$
8: end for
9: return $|area|/2$
10: end procedure
```

---

# Ejemplo 1: Área de Polígono - Análisis de Complejidad I

## Complejidad Temporal

- ▶ **Temporal:**  $T(n) = O(n)$  (un único recorrido de vértices).
- ▶ Una iteración por cada vértice del polígono.
- ▶ Operaciones por iteración:  $O(1)$  (multiplicaciones y sumas).

## Resumen de Complejidades

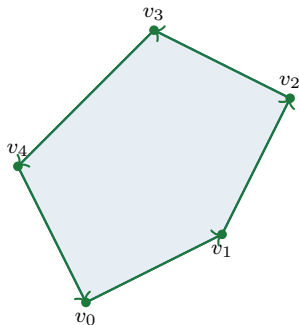
| Temporal | Espacial |
|----------|----------|
| $O(n)$   | $O(1)$   |

# Ejemplo 1: Área de Polígono - Análisis de Complejidad II

## Características

- ▶ **Espacial:**  $M(n) = O(1)$  (variables constantes).
- ▶ No requiere estructuras de datos auxiliares.
- ▶ Solo variables para acumular el área.
- ▶ Funciona para polígonos cóncavos y convexos.
- ▶ El signo del resultado indica la orientación (positivo = antihorario).

## Ejemplo 1: Área de Polígono - Ilustración



Recorrido en orden antihorario

$$\text{Área} = \frac{1}{2} \left| \sum (x_i y_{i+1} - x_{i+1} y_i) \right|$$



## Ejemplo 2: Punto Dentro de Polígono - Planteamiento del Problema I

### Problema

Dado un polígono simple  $P$  y un punto  $q$  en el plano, determinar si  $q$  está en el interior, exterior o sobre la frontera de  $P$ .

### Definición Formal

#### Entrada:

- ▶ Lista de vértices del polígono  $P = [v_0, v_1, \dots, v_{n-1}]$  en orden.
- ▶ Coordenadas del punto  $q = (x, y)$ .

#### Salida:

- ▶ true si  $q$  está dentro o sobre  $P$ .
- ▶ false si  $q$  está estrictamente fuera.

## Ejemplo 2: Punto Dentro de Polígono - Planteamiento del Problema II

### Aplicaciones

- ▶ Selección de objetos en interfaces gráficas.
- ▶ Sistemas de información geográfica (GIS).
- ▶ Detección de colisiones en juegos.
- ▶ Procesamiento de imágenes.

## Ejemplo 2: Punto Dentro de Polígono - Método del Rayo I

### Método del Rayo (Ray Casting)

Se traza un rayo horizontal hacia la derecha desde el punto y se cuenta cuántas veces cruza las aristas del polígono. Si el número de cruces es impar, el punto está dentro; si es par, está fuera.

### Fundamento Teórico

- ▶ Basado en el teorema de la curva de Jordan: una curva cerrada simple divide el plano en dos regiones (interior y exterior).
- ▶ Cada cruce cambia el estado (dentro/fuera).
- ▶ Estado inicial: fuera del polígono.

## Ejemplo 2: Punto Dentro de Polígono - Método del Rayo II

### Manejo de Casos Especiales

- ▶ Si el rayo pasa exactamente por un vértice: se considera que el cruce ocurre si el vértice es un mínimo o máximo local.
- ▶ Si el punto está sobre una arista: se considera dentro.

## Ejemplo 2: Punto Dentro de Polígono - Análisis de Complejidad I

### Complejidad

El algoritmo itera una vez por cada arista del polígono para verificar intersecciones con el rayo.

$$T(n) = O(n), \quad M(n) = O(1)$$

### Resumen de Complejidades

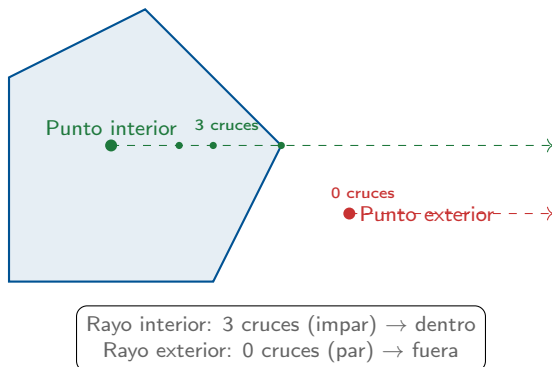
| Temporal | Espacial |
|----------|----------|
| $O(n)$   | $O(1)$   |

## Ejemplo 2: Punto Dentro de Polígono - Análisis de Complejidad II

### Consideraciones de Implementación

- ▶ Es importante manejar correctamente los casos degenerados (rayo pasa por vértices).
- ▶ La precisión numérica puede afectar la decisión cerca de la frontera.
- ▶ Existen variantes que usan rayos en otras direcciones o algoritmos basados en winding number.

## Ejemplo 2: Punto Dentro de Polígono - Ilustración



## Ejemplo 3: Envolverte Convexa - Planteamiento del Problema I

### Problema

Dado un conjunto de  $n$  puntos en el plano, encontrar el polígono convexo de menor área que contiene a todos los puntos.

### Definición Formal

**Entrada:** Conjunto  $P = \{p_1, p_2, \dots, p_n\}$  con  $p_i = (x_i, y_i) \in \mathbb{R}^2$ .

**Salida:** Lista de puntos  $H \subseteq P$  que son los vértices de la envolvente convexa, ordenados en sentido antihorario.



## Ejemplo 3: Envolverte Convexa - Planteamiento del Problema II

### Características de la solución

- ▶ Todos los vértices pertenecen al conjunto original.
- ▶ Los vértices están ordenados en sentido antihorario.
- ▶ No hay tres vértices colineales consecutivos (salvo casos degenerados).
- ▶ La envolverte es única (salvo colinealidades).

### Aplicaciones directas

- ▶ Colisión en juegos y simulaciones.
- ▶ Análisis de formas en visión por computador.
- ▶ Optimización geométrica.
- ▶ Reconocimiento de patrones.

# Ejemplo 3: Envoltente Convexa - Algoritmo (Graham Scan)

---

## Algoritmo 17 Graham Scan - Envoltente Convexa

---

**Require:** *points*: conjunto de puntos en el plano

**Ensure:** Vértices de la envoltente convexa en orden antihorario

```
1: procedure CROSS(a, b, c)
2: return (b.x - a.x)(c.y - a.y) - (b.y - a.y)(c.x - a.x)
3: end procedure

4: procedure CONVEX_HULL(points)
5: start $\leftarrow$ punto con coordenada y mínima (y menor x en caso de empate)
6: Ordenar los demás puntos por ángulo polar respecto a start
7: hull $\leftarrow$ pila vacía
8: push(hull, start)
9: for cada point en puntos ordenados do
10: while |hull| ≥ 2 and CROSS(hull[-2], hull[-1], point) ≤ 0 do
11: pop(hull) //Descartar giro a derecha o colineal
12: end while
13: push(hull, point)
14: end for
15: return hull
16: end procedure
```

---

## Ejemplo 3: Envolverte Convexa - Análisis de Complejidad I

### Complejidad Temporal

- ▶ **Encontrar punto inicial:**  $O(n)$ .
- ▶ **Ordenamiento por ángulo polar:**  $O(n \log n)$ .
- ▶ **Escaneo (Graham Scan):**  $O(n)$  (cada punto se apila y desapila a lo sumo una vez).
- ▶ **Total:**  $T(n) = O(n \log n)$ .

### Resumen de Complejidades

| Temporal      | Espacial |
|---------------|----------|
| $O(n \log n)$ | $O(n)$   |

## Ejemplo 3: Envolverte Convexa - Análisis de Complejidad II

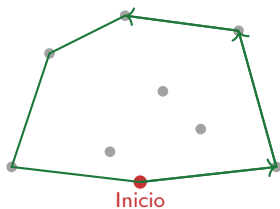
### Complejidad Espacial

- ▶ **Espacial:**  $M(n) = O(n)$ .
- ▶ Almacenamiento de la envolvente convexa (hasta  $n$  puntos).
- ▶ Estructura auxiliar para ordenamiento (pila o arreglo).

### Otros Algoritmos

- ▶ **Jarvis March (Gift Wrapping):**  $O(nh)$ , donde  $h$  es el número de puntos en la envolvente.
- ▶ **QuickHull:**  $O(n \log n)$  caso promedio,  $O(n^2)$  peor caso.
- ▶ **Algoritmo de Chan:**  $O(n \log h)$  óptimo.

### Ejemplo 3: Envolvente Convexa - Ilustración



Puntos en gris: interiores  
Borde en verde: envolvente convexa  
Rojo: punto inicial (mínimo  $y$ )

## Ejemplo 4: Intersección de Segmentos - Planteamiento del Problema I

### Problema

Dados dos segmentos de recta  $s_1 = (p_1, q_1)$  y  $s_2 = (p_2, q_2)$ , determinar si se intersectan (incluyendo intersecciones en extremos o superposición parcial).

### Definición Formal

**Entrada:** Cuatro puntos  $p_1, q_1, p_2, q_2 \in \mathbb{R}^2$  que definen los segmentos  $s_1$  y  $s_2$ .

**Salida:** true si los segmentos comparten al menos un punto, false en caso contrario.

## Ejemplo 4: Intersección de Segmentos - Planteamiento del Problema II

### Tipos de intersección

- ▶ **Propia:** Los segmentos se cruzan en un punto interior a ambos.
- ▶ **Impropia:** Comparten un extremo o uno está contenido en el otro.
- ▶ **Ninguna:** Son disjuntos.

### Requisitos

- ▶ Manejo robusto de casos degenerados (colinealidad, puntos coincidentes).
- ▶ Precisión numérica controlada para evitar falsos positivos/negativos.

# Ejemplo 4: Intersección de Segmentos - Algoritmo

---

## Algoritmo 18 Intersección de Segmentos (Basado en Orientación)

---

**Require:**  $p_1, q_1$ : extremos del segmento  $s_1$

1:  $p_2, q_2$ : extremos del segmento  $s_2$

**Ensure:** true si  $s_1$  y  $s_2$  se intersecan, false en otro caso

2: **function** ORIENTACION( $a, b, c$ )

3:     **return**  $(b_x - a_x)(c_y - a_y) - (b_y - a_y)(c_x - a_x)$

4: **end function**

5: **function** ENSEGMENTO( $a, b, c$ )

6:     **return**  $\min(a_x, b_x) \leq c_x \leq \max(a_x, b_x)$  **and**

7:

8:      $\min(a_y, b_y) \leq c_y \leq \max(a_y, b_y)$

9: **end function**

10: **function** SEGMENTOSINTERSECTAN( $p_1, q_1, p_2, q_2$ )

11:      $o_1 \leftarrow \text{orientacion}(p_1, q_1, p_2)$

12:      $o_2 \leftarrow \text{orientacion}(p_1, q_1, q_2)$

13:      $o_3 \leftarrow \text{orientacion}(p_2, q_2, p_1)$

14:      $o_4 \leftarrow \text{orientacion}(p_2, q_2, q_1)$

15:     **if**  $o_1 \neq 0$  **or**  $o_2 \neq 0$  **or**  $o_3 \neq 0$  **or**  $o_4 \neq 0$  **then**

16:         **if**  $o_1 \cdot o_2 < 0$  **and**  $o_3 \cdot o_4 < 0$  **then**

17:             **return** true   //Intersección propia

18:         **end if**

19:     **else**   //Todos colineales

20:         **if** ENSEGMENTO( $p_1, q_1, p_2$ ) **or** ENSEGMENTO( $p_1, q_1, q_2$ ) **then**

21:             **return** true

22:         **end if**

23:         **if** ENSEGMENTO( $p_2, q_2, p_1$ ) **or** ENSEGMENTO( $p_2, q_2, q_1$ ) **then**

24:             **return** true

25:         **end if**

26:     **end if**

27:     **return** false

28: **end function**

---



## Ejemplo 4: Intersección de Segmentos - Análisis I

### Método de Orientación

Se utiliza la noción de orientación de tres puntos (horaria, antihoraria o colineal). Dos segmentos se intersectan si sus orientaciones relativas son distintas o si hay colinealidad con superposición.

### Casos de Intersección

- ▶ **Intersección propia:** Los segmentos se cruzan en un punto interior (orientaciones opuestas).
- ▶ **Intersección impropia:** Los segmentos comparten un extremo o son colineales y se superponen.
- ▶ **Sin intersección:** Los segmentos son disjuntos.

## Ejemplo 4: Intersección de Segmentos - Análisis II

### Resumen de Complejidades

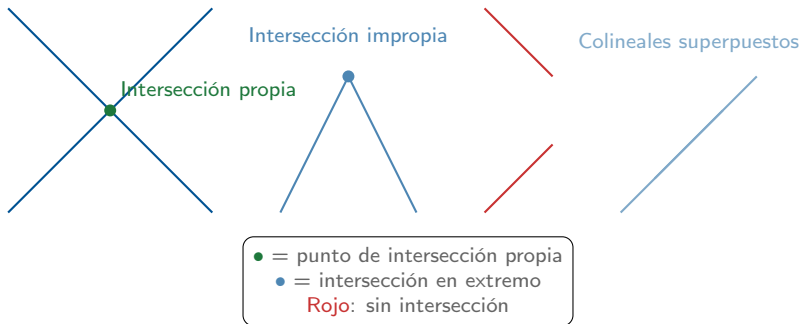
| Temporal | Espacial |
|----------|----------|
| $O(1)$   | $O(1)$   |

### Complejidad

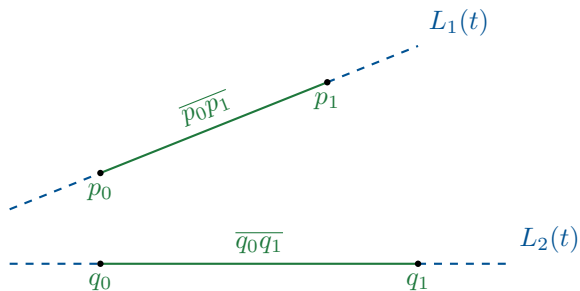
El algoritmo realiza un número constante de llamadas a funciones auxiliares, cada una con operaciones aritméticas básicas.

$$T(1) = O(1), \quad M(1) = O(1)$$

## Ejemplo 4: Intersección de Segmentos - Ilustración



## Ejemplo 5: Intersección de Segmentos - Ilustración Geométrica $\mathbb{R}^n$



## Ejemplo 5: Intersección de Segmentos - Análisis Matemático

I

### Planteamiento del Problema

Dados cuatro puntos  $p_0, p_1, q_0, q_1 \in \mathbb{R}^n$ , determinar si los segmentos  $\overline{p_0p_1}$  y  $\overline{q_0q_1}$  se intersecan.

**Objetivo:** Encontrar si existe un punto común a ambos segmentos, no solo a las rectas que los contienen.

## Ejemplo 5: Intersección de Segmentos - Análisis Matemático II

### Rectas Paramétricas (Extensiones infinitas)

$$L_1(t) = p_0 + td_1, \quad \text{donde } d_1 = p_1 - p_0 \neq 0$$

$$L_2(s) = q_0 + sd_2, \quad \text{donde } d_2 = q_1 - q_0 \neq 0$$

con  $t, s \in \mathbb{R}$ .

**Observación:** Los segmentos corresponden a  $t, s \in [0, 1]$ .

## Ejemplo 5: Intersección de Segmentos - Análisis Matemático III

### Distancia Mínima entre Rectas

Definimos la distancia entre las dos rectas como:

$$d(L_1, L_2) = \min_{t,s \in \mathbb{R}} \|L_1(t) - L_2(s)\|$$

Este problema de optimización nos permitirá encontrar los parámetros  $(t_0, s_0)$  que minimizan la distancia.

## Ejemplo 5: Intersección de Segmentos - Análisis Matemático IV

### Producto Interno y Ángulo

El producto interno entre los vectores directores:

$$\langle d_1, d_2 \rangle = \|d_1\| \|d_2\| \cos \theta$$

donde  $\theta$  es el ángulo entre  $d_1$  y  $d_2$ .



# Ejemplo 5: Intersección de Segmentos - Análisis Matemático

## V

### Casos Especiales (Análisis Previo)

- **Rectas paralelas o coincidentes** ( $\cos \theta = \pm 1$ ):

$$d(L_1, L_2) = \begin{cases} 0 & \text{si } p_0 \in L_2 \text{ (coincidentes)} \\ \|p_0 - L_2(s_0)\| & \text{si } p_0 \notin L_2 \text{ (paralelas distintas)} \end{cases}$$

donde  $s_0$  es la proyección ortogonal de  $p_0$  sobre  $L_2$ .

- **Rectas perpendiculares** ( $\cos \theta = 0$ ):

$$d(L_1, L_2) = 0 \quad (\text{siempre se cortan en algún punto})$$

En todos estos casos, la distancia mínima puede ser cero o positiva.

## Ejemplo 5: Intersección de Segmentos - Análisis Matemático VI

### Caso General

Definimos la función objetivo como el cuadrado de la distancia:

$$\begin{aligned} f(t, s) &= \|L_1(t) - L_2(s)\|^2 = \|p_0 - q_0 + td_1 - sd_2\|^2 \\ &= \|td_1 - sd_2\|^2 + 2\langle p_0 - q_0, td_1 - sd_2 \rangle + \|p_0 - q_0\|^2. \end{aligned}$$

Además

$$\|td_1 - sd_2\|^2 = \|d_1\|^2 t^2 + \|d_2\|^2 s^2 - 2\langle d_1, d_2 \rangle ts.$$

## Ejemplo 5: Intersección de Segmentos - Análisis Matemático VII

### Forma cuadrática

Notemos que  $f$  admite una forma cuadrática

$$f(t, s) = \frac{1}{2} \begin{pmatrix} t & s \end{pmatrix} Q \begin{pmatrix} t \\ s \end{pmatrix} + b^T \begin{pmatrix} t \\ s \end{pmatrix} + c,$$

donde

$$Q = \begin{pmatrix} 2\|d_1\|^2 & -2\langle d_1, d_2 \rangle \\ -2\langle d_1, d_2 \rangle & 2\|d_2\|^2 \end{pmatrix}, \quad b = \begin{pmatrix} 2\langle p_0 - q_0, d_1 \rangle \\ -2\langle p_0 - q_0, d_2 \rangle \end{pmatrix} \quad \text{y} \quad c = \|p_0 - q_0\|^2.$$

## Ejemplo 5: Intersección de Segmentos - Análisis Matemático VIII

### Observación

La forma cuadrática asociada a la matriz  $Q$  puede expresarse como el cuadrado de la norma de la diferencia direccional:

$$\frac{1}{2} \begin{pmatrix} t & s \end{pmatrix} Q \begin{pmatrix} t \\ s \end{pmatrix} = \|t d_1 - s d_2\|^2.$$

Esta identidad revela que  $Q$  codifica geoméricamente la separación entre las direcciones  $d_1$  y  $d_2$  escaladas por  $t$  y  $s$  respectivamente.

## Ejemplo 5: Intersección de Segmentos - Análisis Matemático IX

### Derivadas Parciales (Condiciones de Optimalidad)

$$\nabla f(t_0, s_0) = 0 \Rightarrow Q \begin{pmatrix} t_0 \\ s_0 \end{pmatrix} + b = 0$$

$$\nabla^2 f(t, s) = H_f(t, s) \Rightarrow Q = Q^T = H_f(0, 0)$$

## Ejemplo 5: Intersección de Segmentos - Análisis Matemático

X

### Sistema Lineal para $(t_0, s_0)$

Si existe  $Q^{-1} = H_f(0, 0)^{-1}$ , es decir,  $\det(H_f(0, 0)) \neq 0$ , se tiene

$$H_f(0, 0) \begin{pmatrix} t_0 \\ s_0 \end{pmatrix} = -b \Rightarrow \begin{pmatrix} t_0 \\ s_0 \end{pmatrix} = -H_f(0, 0)^{-1}b$$

### Determinante de la Matriz

$$\begin{aligned} \det(H_f(0, 0)) &= 4\|d_1\|^2\|d_2\|^2 - 4\langle d_1, d_2 \rangle^2 \\ &= 4\|d_1\|^2\|d_2\|^2(1 - \cos^2 \theta) \\ &= 4\|d_1\|^2\|d_2\|^2 \sin^2 \theta \neq 0 \quad (\text{ya que } \sin \theta \neq 0) \end{aligned}$$

## Ejemplo 5: Intersección de Segmentos - Análisis Matemático XI

### Solución del Sistema (Fórmulas Explícitas)

La matriz inversa es:

$$H_f(0,0)^{-1} = \frac{1}{\det(H_f(t,s))} \begin{pmatrix} 2\|d_2\|^2 & 2\langle d_1, d_2 \rangle \\ 2\langle d_1, d_2 \rangle & 2\|d_1\|^2 \end{pmatrix}$$

Por tanto:

$$t_0 = \frac{-\langle p_0 - q_0, d_1 \rangle \|d_2\|^2 + \langle p_0 - q_0, d_2 \rangle \langle d_1, d_2 \rangle}{\|d_1\|^2 \|d_2\|^2 - \langle d_1, d_2 \rangle^2}$$

$$s_0 = \frac{-\langle p_0 - q_0, d_1 \rangle \langle d_1, d_2 \rangle + \langle p_0 - q_0, d_2 \rangle \|d_1\|^2}{\|d_1\|^2 \|d_2\|^2 - \langle d_1, d_2 \rangle^2}$$

## Ejemplo 5: Intersección de Segmentos - Análisis Matemático XII

### Valor mínimo

Como

$$\begin{pmatrix} t_0 \\ s_0 \end{pmatrix} = -H_f(0, 0)^{-1}b.$$

Entonces

$$f(t_0, s_0) = c - \frac{1}{2}b^T H_f(0, 0)^{-1}b,$$

donde

$$b = \begin{pmatrix} 2\langle p_0 - q_0, d_1 \rangle \\ -2\langle p_0 - q_0, d_2 \rangle \end{pmatrix} \text{ y } c = \|p_0 - q_0\|^2.$$



## Ejemplo 5: Intersección de Segmentos - Análisis Matemático XIII

Demostración:  $(t_0, s_0)$  es Mínimo Global

Sea  $(x, y) \in \mathbb{R}^2$  arbitrario. Evaluando la forma cuadrática asociada a  $Q$ :

$$\begin{aligned}\frac{1}{2} (x \ y) Q \begin{pmatrix} x \\ y \end{pmatrix} &= \frac{1}{2} (x \ y) H_f(0,0) \begin{pmatrix} x \\ y \end{pmatrix} \\ &= \|x d_1 - y d_2\|^2 \geq 0.\end{aligned}$$

Como la forma cuadrática es no negativa para todo  $(x, y)$ , concluimos que  $H_f(0,0)$  es **semidefinida positiva**. Por tanto,  $f$  es convexa y  $(t_0, s_0)$  es un **mínimo global**.

## Ejemplo 5: Intersección de Segmentos - Análisis Matemático XIV

### Distancia Mínima entre Rectas

La distancia mínima entre las rectas es:

$$d_{\min} = \sqrt{f(t_0, s_0)} = \|L_1(t_0) - L_2(s_0)\|$$

### Condición de Intersección de Rectas

Las rectas se cortan si y solo si:

$$d_{\min} = 0 \quad \Longleftrightarrow \quad L_1(t_0) = L_2(s_0)$$

## Ejemplo 5: Intersección de Segmentos - Análisis Matemático XV

### Interpretación Geométrica para Segmentos

Cuando  $d_{\min} = 0$  (las rectas se cortan en un punto  $P$ ):

- ▶ Si  $t_0 \in [0, 1]$  y  $s_0 \in [0, 1]$ : El punto  $P$  pertenece a **ambos segmentos**  $\Rightarrow$  **se intersectan**.
- ▶ Si  $t_0 \in [0, 1]$  pero  $s_0 \notin [0, 1]$ : El punto  $P$  está en  $\overline{p_0p_1}$  pero no en  $\overline{q_0q_1}$ .
- ▶ Si  $s_0 \in [0, 1]$  pero  $t_0 \notin [0, 1]$ : El punto  $P$  está en  $\overline{q_0q_1}$  pero no en  $\overline{p_0p_1}$ .
- ▶ Si  $t_0 \notin [0, 1]$  y  $s_0 \notin [0, 1]$ : El punto  $P$  no pertenece a ninguno de los segmentos.

## Ejemplo 5: Intersección de Segmentos - Análisis Matemático XVI

### Complejidad

El algoritmo resultante requiere un número constante de operaciones aritméticas:

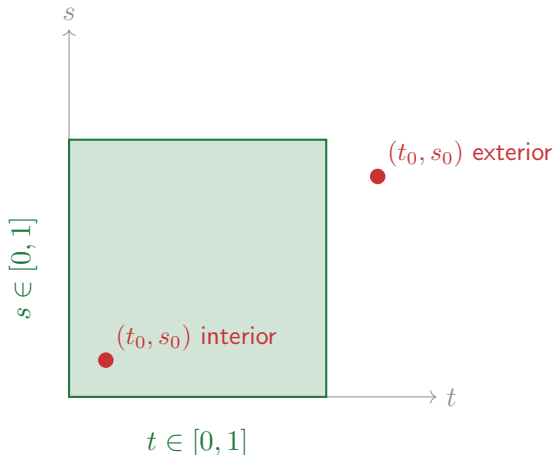
$$T(1) = O(1), \quad M(1) = O(1)$$

## Ejemplo 5: Intersección de Segmentos - Ilustración Geométrica

### Interpretación de la Gráfica

- ▶ Las líneas sólidas representan los **segmentos**  $\overline{p_0p_1}$  y  $\overline{q_0q_1}$  (corresponden a  $t, s \in [0, 1]$ ).
- ▶ Las líneas punteadas representan las **extensiones infinitas** de las rectas  $L_1(t)$  y  $L_2(s)$  ( $t, s \in \mathbb{R}$ ).
- ▶ El problema consiste en determinar si existe algún punto que pertenezca simultáneamente a ambas líneas sólidas.

## Ejemplo 5: Intersección de Segmentos - Ilustración Geométrica



Los puntos  $(t_0, s_0) \in [0, 1] \times [0, 1]$  indican intersección de segmentos.

# Ejemplo 5: Intersección de Segmentos - Algoritmo basado en Optimización

---

## Algoritmo 19 Intersección de Segmentos (Basado en Optimización)

---

**Require:**  $p_0, p_1, q_0, q_1 \in \mathbb{R}^n$  con  $p_0 \neq p_1, q_0 \neq q_1$

**Ensure:** true si los segmentos se intersectan

```
1: procedure SEGMENTOSINTERSECTAN(p_0, p_1, q_0, q_1)
2: $d_1 \leftarrow p_1 - p_0$
3: $d_2 \leftarrow q_1 - q_0$
4: $w \leftarrow p_0 - q_0$
5: $a \leftarrow \|d_1\|^2$
6: $b \leftarrow \langle d_1, d_2 \rangle$
7: $c \leftarrow \|d_2\|^2$
8: $d \leftarrow \langle w, d_1 \rangle$
9: $e \leftarrow \langle w, d_2 \rangle$
10: $\text{det} \leftarrow a \cdot c - b \cdot b$
11: if $|\text{det}| < \epsilon$ then
12: return MANEJARPARALELAS(p_0, p_1, q_0, q_1)
13: else
14: $t_0 \leftarrow (-d \cdot c + e \cdot b) / \text{det}$
15: $s_0 \leftarrow (-d \cdot b + e \cdot a) / \text{det}$
16: return $(0 \leq t_0 \leq 1)$ and $(0 \leq s_0 \leq 1)$
17: end if
18: end procedure
```

---

# Algoritmos de aleatorización



# Fundamentos

# Fundamentos de Aleatorización I

## Definición

Un algoritmo aleatorizado es aquel que utiliza números aleatorios durante su ejecución para tomar decisiones. Su comportamiento no está determinado únicamente por la entrada, sino también por los valores generados por un generador de números aleatorios.

# Fundamentos de Aleatorización II

## Tipos de Algoritmos Aleatorizados

### ▶ Algoritmos de Monte Carlo:

- ▶ Pueden producir respuestas incorrectas con cierta probabilidad (acotada y controlable).
- ▶ El tiempo de ejecución es determinista.
- ▶ Útiles cuando una pequeña probabilidad de error es aceptable.
- ▶ Ejemplo: Test de primalidad de Miller-Rabin.

### ▶ Algoritmos de Las Vegas:

- ▶ Siempre producen respuestas correctas.
- ▶ El tiempo de ejecución es una variable aleatoria.
- ▶ Útiles cuando el tiempo de ejecución promedio es bueno aunque el peor caso sea malo.
- ▶ Ejemplo: Quicksort aleatorizado.

# Fundamentos de Aleatorización III

## Ventajas de la Aleatorización

- ▶ **Robustez:** Evita casos patológicos que afectan a algoritmos deterministas (ej. Quicksort con entrada ordenada).
- ▶ **Simplicidad:** Permite implementaciones más simples que sus contrapartes deterministas.
- ▶ **Exploración del espacio de búsqueda:** La aleatoriedad permite escapar de óptimos locales en problemas de optimización.
- ▶ **Balance garantizado:** En promedio, el rendimiento es bueno para cualquier entrada.

# Fundamentos de Aleatorización IV

## Aplicaciones Típicas

- ▶ **Criptografía:** Generación de números primos grandes, claves seguras.
- ▶ **Optimización:** Escape de óptimos locales, búsqueda global.
- ▶ **Simulación:** Modelado de sistemas complejos con componentes estocásticos.
- ▶ **Aprendizaje Automático:** Inicialización de parámetros, regularización (dropout).
- ▶ **Algoritmos Numéricos:** Integración Monte Carlo, métodos de muestreo.
- ▶ **Estructuras de Datos:** Skip lists, hashing universal.

# Ejemplo 1: Integración Monte Carlo - Planteamiento del Problema I

## Problema

Dada una función  $f : D \subset \mathbb{R}^d \rightarrow \mathbb{R}$  definida en un dominio  $D$  de volumen conocido, calcular la integral:

$$I = \int_D f(\mathbf{x}) d\mathbf{x}$$

cuando la integral es difícil de calcular analíticamente o por métodos deterministas.

# Ejemplo 1: Integración Monte Carlo - Planteamiento del Problema II

## Aplicaciones

- ▶ Cálculo de áreas y volúmenes de formas complejas.
- ▶ Física estadística (cálculo de valores esperados).
- ▶ Finanzas computacionales (valoración de opciones).
- ▶ Gráficos por computador (iluminación global).

## Ejemplo 1: Integración Monte Carlo - Algoritmo

---

### Algoritmo 20 Integración Monte Carlo

---

**Require:**  $f$ : función a integrar,  $D$ : dominio,  $n$ : número de muestras

**Ensure:** Aproximación de la integral  $I = \int_D f(\mathbf{x})d\mathbf{x}$

```
1: procedure MONTE_CARLO_INTEGRATE(f, D, n)
2: $sum \leftarrow 0$
3: $V \leftarrow \text{VOLUMEN}(D)$ //Volumen del dominio
4: for $i \leftarrow 1$ to n do
5: $\mathbf{x}_i \leftarrow \text{MUESTREAR_UNIFORME}(D)$ //Muestra aleatoria uniforme
6: $sum \leftarrow sum + f(\mathbf{x}_i)$
7: end for
8: return $V \cdot \frac{sum}{n}$
9: end procedure
```

---



# Ejemplo 1: Integración Monte Carlo - Análisis I

## Fundamento Probabilístico

Sea  $\mathbf{X} \sim U(D)$  una variable aleatoria uniformemente distribuida en el dominio  $D$ . Entonces:

$$I = \int_D f(\mathbf{x}) d\mathbf{x} = V(D) \cdot \mathbb{E}[f(\mathbf{X})]$$

donde  $V(D)$  es el volumen del dominio.

## Estimador

$$\hat{I}_n = V(D) \cdot \frac{1}{n} \sum_{i=1}^n f(\mathbf{X}_i)$$

donde  $\mathbf{X}_1, \dots, \mathbf{X}_n$  son muestras independientes e idénticamente distribuidas.

# Ejemplo 1: Integración Monte Carlo - Análisis II

## Propiedades del Estimador

- ▶ **Insesgado:**  $\mathbb{E}[\hat{I}_n] = I$
- ▶ **Consistente:**  $\hat{I}_n \xrightarrow[n \rightarrow \infty]{\text{c.s.}} I$  (Ley Fuerte de los Grandes Números)
- ▶ **Varianza:**  $\text{Var}(\hat{I}_n) = \frac{V(D)^2}{n} \text{Var}(f(\mathbf{X}))$

# Ejemplo 1: Integración Monte Carlo - Análisis III

## Convergencia y Error

Por el Teorema Central del Límite:

$$\sqrt{n}(\hat{I}_n - I) \xrightarrow{d} \mathcal{N}(0, \sigma^2)$$

donde  $\sigma^2 = V(D)^2 \text{Var}(f(\mathbf{X}))$ .

**Error estándar:**  $\text{Error} \approx \frac{\sigma}{\sqrt{n}}$

**Tasa de convergencia:**  $O_p\left(\frac{1}{\sqrt{n}}\right)$  — independiente de la dimensión  $d$ .

## Precisión

La precisión mejora con  $n$ , independientemente de la dimensión del dominio (ventaja sobre métodos deterministas).

# Ejemplo 1: Integración Monte Carlo - Análisis IV

## Resumen de Complejidades

| Temporal | Espacial | Error           |
|----------|----------|-----------------|
| $O(n)$   | $O(1)$   | $O(1/\sqrt{n})$ |

## Ventaja Dimensional

A diferencia de los métodos deterministas (cuadratura), cuya complejidad crece exponencialmente con la dimensión ( $O(m^d)$ ), Monte Carlo mantiene tasa  $O(1/\sqrt{n})$  independiente de  $d$ . Esto lo hace especialmente útil para problemas en alta dimensión.

## Ejemplo 2: Aproximación de $\pi$ - Planteamiento del Problema I

### Problema

Aproximar el valor de la constante matemática  $\pi$  utilizando métodos probabilísticos.

## Ejemplo 2: Aproximación de $\pi$ - Planteamiento del Problema II

### Idea Geométrica

Consideramos un cuadrado de lado 2 centrado en el origen (con vértices en  $(-1, -1)$  y  $(1, 1)$ ) y un círculo inscrito de radio 1 centrado en el origen.

- ▶ Área del cuadrado:  $A_{\text{cuadrado}} = 4$
- ▶ Área del círculo:  $A_{\text{círculo}} = \pi \cdot 1^2 = \pi$

La proporción de puntos aleatorios uniformes en el cuadrado que caen dentro del círculo es aproximadamente:

$$\frac{\text{puntos dentro del círculo}}{\text{puntos totales}} \approx \frac{A_{\text{círculo}}}{A_{\text{cuadrado}}} = \frac{\pi}{4}$$

## Ejemplo 2: Aproximación de $\pi$ - Algoritmo

---

### Algoritmo 21 Aproximación de $\pi$ por Monte Carlo

---

**Require:**  $n$ : número de muestras

**Ensure:** Aproximación de  $\pi$

```
1: procedure APPROXIMATE_PI(n)
2: $inside \leftarrow 0$
3: for $i \leftarrow 1$ to n do
4: $x \leftarrow \text{random}(-1, 1)$ //Coordenada x uniforme en $[-1, 1]$
5: $y \leftarrow \text{random}(-1, 1)$ //Coordenada y uniforme en $[-1, 1]$
6: if $x^2 + y^2 \leq 1$ then //Dentro del círculo
7: $inside \leftarrow inside + 1$
8: end if
9: end for
10: return $4 \cdot \frac{inside}{n}$
11: end procedure
```

---

## Ejemplo 2: Aproximación de $\pi$ - Análisis I

### Fundamento Probabilístico

Sea  $(X, Y)$  un punto uniformemente distribuido en el cuadrado  $[-1, 1] \times [-1, 1]$ . Definimos la variable indicadora:

$$Z = \begin{cases} 1 & \text{si } X^2 + Y^2 \leq 1 \\ 0 & \text{en otro caso} \end{cases}$$

Entonces:

$$\mathbb{E}[Z] = P(X^2 + Y^2 \leq 1) = \frac{\pi}{4}$$



## Ejemplo 2: Aproximación de $\pi$ - Análisis II

### Estimador

$$\hat{\pi}_n = 4 \cdot \frac{1}{n} \sum_{i=1}^n Z_i$$

donde  $Z_i$  son independientes con la misma distribución que  $Z$ .

### Propiedades

- ▶ **Insesgado:**  $\mathbb{E}[\hat{\pi}_n] = \pi$
- ▶ **Varianza:**  $\text{Var}(\hat{\pi}_n) = \frac{16}{n} \cdot \frac{\pi}{4} \left(1 - \frac{\pi}{4}\right) = \frac{4\pi - \pi^2}{n}$
- ▶ **Error estándar:**  $\sigma_n = \sqrt{\frac{4\pi - \pi^2}{n}} \approx \frac{2.85}{\sqrt{n}}$

## Ejemplo 2: Aproximación de $\pi$ - Análisis III

### Estimación

La estimación converge a  $\pi$  cuando  $n \rightarrow \infty$  por la Ley de los Grandes Números.

## Ejemplo 2: Aproximación de $\pi$ - Análisis IV

### Resumen de Complejidades

| Temporal | Espacial | Error           |
|----------|----------|-----------------|
| $O(n)$   | $O(1)$   | $O(1/\sqrt{n})$ |

## Ejemplo 2: Aproximación de $\pi$ - Análisis V

### Limitaciones Prácticas

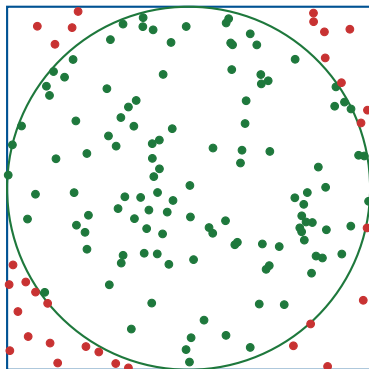
- ▶ **Convergencia lenta:**  $O(1/\sqrt{n})$  significa que para mejorar la precisión en un factor 10, necesitamos 100 veces más muestras.
- ▶ Para obtener 6 dígitos decimales correctos ( $\epsilon \approx 10^{-6}$ ), se necesitan:

$$n \approx \left( \frac{2.85}{10^{-6}} \right)^2 \approx 8 \times 10^{12} \text{ muestras}$$

- ▶ Útil para demostración conceptual, no para cálculo de alta precisión.
- ▶ Existen métodos deterministas mucho más eficientes para calcular  $\pi$ .

## Ejemplo 2: Aproximación de $\pi$ - Ilustración

Cuadrado  $[-1, 1] \times [-1, 1]$



- Dentro del círculo
- Fuera del círculo

$$\pi \approx 4 \times \frac{\text{Puntos dentro del círculo}}{\text{Total de puntos}}$$

## Ejemplo 3: Búsqueda Local Aleatorizada - Planteamiento del Problema I

### Problema

Dada una función  $f : \mathcal{S} \rightarrow \mathbb{R}$  definida sobre un espacio de búsqueda  $\mathcal{S}$  (posiblemente continuo, discreto o combinatorio), encontrar el máximo global de  $f$ .

### Dificultad

- ▶ La función puede tener múltiples óptimos locales.
- ▶ Los algoritmos deterministas de ascenso por gradiente o búsqueda local quedan atrapados en el primer óptimo local encontrado.
- ▶ El espacio de búsqueda puede ser demasiado grande para exploración exhaustiva.

## Ejemplo 3: Búsqueda Local Aleatorizada - Planteamiento del Problema II

### Aplicaciones

- ▶ Optimización de funciones multimodales.
- ▶ Ajuste de hiperparámetros en aprendizaje automático.
- ▶ Problemas de scheduling y asignación.
- ▶ Diseño de ingeniería.

## Ejemplo 3: Búsqueda Local Aleatorizada con Reinicio

---

**Algoritmo 22** Búsqueda Aleatorizada con Reinicio (Randomized Local Search)

---

**Require:**  $f$ : función objetivo a maximizar,  $k$ : número de reinicios

**Ensure:** Mejor valor encontrado

```
1: procedure RANDOMIZED_LOCAL_SEARCH(f, k)
2: $best_value \leftarrow -\infty$
3: $best_point \leftarrow \text{null}$
4: for $r \leftarrow 1$ to k do
5: $current \leftarrow \text{PUNTO_ALEATORIO}$ //Reinicio aleatorio
6: $current_value \leftarrow f(current)$
7: $improved \leftarrow \text{true}$
8: while $improved$ do
9: $improved \leftarrow \text{false}$
10: $neighbor \leftarrow \text{VECINO_ALEATORIO}(current)$ //Exploración
11: if $f(neighbor) > current_value$ then
12: $current \leftarrow neighbor$
13: $current_value \leftarrow f(current)$
14: $improved \leftarrow \text{true}$
15: end if
16: end while
17: if $current_value > best_value$ then
18: $best_value \leftarrow current_value$
19: $best_point \leftarrow current$
20: end if
21: end for
22: return $best_value$
23: end procedure
```

---



## Ejemplo 3: Búsqueda Local Aleatorizada - Análisis I

### Estrategia

La búsqueda local aleatorizada con reinicio combina dos fases:

- ▶ **Exploración global:** Los reinicios aleatorios permiten muestrear diferentes regiones del espacio de búsqueda.
- ▶ **Explotación local:** Una vez en una región, se realiza ascenso (greedy) hacia el óptimo local de esa región.

### Balance Exploración/Explotación

El parámetro  $k$  (número de reinicios) controla este balance:

- ▶  $k$  pequeño: Menos exploración, riesgo de no encontrar el óptimo global.
- ▶  $k$  grande: Más exploración, mayor probabilidad de éxito pero mayor costo computacional.

## Ejemplo 3: Búsqueda Local Aleatorizada - Análisis II

### Resumen de Complejidades

| Temporal       | Espacial | Parámetro clave           |
|----------------|----------|---------------------------|
| $O(k \cdot m)$ | $O(1)$   | $k$ : número de reinicios |

donde  $m$  es el número promedio de mejoras por reinicio.

## Ejemplo 3: Búsqueda Local Aleatorizada - Análisis III

### Probabilidad de Éxito

Si la probabilidad de que un reinicio aleatorio caiga en la cuenca de atracción del óptimo global es  $p$ , entonces la probabilidad de no encontrar el óptimo después de  $k$  reinicios es  $(1 - p)^k$ .

Por tanto, la probabilidad de éxito es:

$$P_{\text{éxito}} = 1 - (1 - p)^k$$

Para lograr una probabilidad de éxito de al menos  $1 - \delta$ , necesitamos:

$$k \geq \frac{\log(1/\delta)}{\log(1/(1 - p))}$$

## Ejemplo 3: Búsqueda Local Aleatorizada - Análisis IV

### Ventajas sobre Búsqueda Determinista

- ▶ Escapa de óptimos locales mediante reinicios.
- ▶ Explora múltiples regiones del espacio.
- ▶ Balance controlable entre exploración y explotación.
- ▶ No requiere información de gradiente (solo evaluaciones de  $f$ ).
- ▶ Puede aplicarse a espacios discretos o continuos.

## Ejemplo 4: Quicksort Aleatorizado - Planteamiento del Problema I

### Problema

Ordenar un arreglo de  $n$  elementos utilizando el algoritmo Quicksort, evitando el mal rendimiento en el peor caso ( $O(n^2)$ ) que ocurre cuando el pivote se elige de forma determinista (ej. siempre el primer o último elemento) y la entrada ya está ordenada o invertida.

### Solución

Introducir aleatoriedad en la selección del pivote para garantizar que, en promedio, el algoritmo tenga buen rendimiento para cualquier entrada.

## Ejemplo 4: Quicksort Aleatorizado - Algoritmo

---

### Algoritmo 23 Quicksort Aleatorizado

---

**Require:**  $A$ : arreglo,  $left, right$ : índices (inclusive)

**Ensure:**  $A$  ordenado in-place

```
1: procedure RANDOMIZED_QUICKSORT($A, left, right$)
2: if $left < right$ then
3: $pivot_index \leftarrow \text{random}(left, right)$ //Pivote aleatorio
4: $\text{SWAP}(A[pivot_index], A[right])$ //Mover pivote al final
5: $pivot \leftarrow \text{PARTITION}(A, left, right)$ //Partición estándar
6: $\text{RANDOMIZED_QUICKSORT}(A, left, pivot - 1)$
7: $\text{RANDOMIZED_QUICKSORT}(A, pivot + 1, right)$
8: end if
9: end procedure
```

---

La función `partition` es la misma que en Quicksort determinista: reorganiza el arreglo y devuelve la posición final del pivote.

## Ejemplo 4: Quicksort Aleatorizado - Análisis I

### Idea Clave

Antes de particionar, se elige un pivote aleatorio uniformemente entre los elementos del subarreglo y se intercambia con el último elemento (o el primero). Esto garantiza que, en promedio, el pivote divida el arreglo de manera balanceada, independientemente de la distribución de los datos de entrada.

### Complejidad Temporal

- ▶ **Caso esperado:**  $T(n) = O(n \log n)$
- ▶ **Peor caso:**  $T(n) = O(n^2)$  (cuando siempre se elige el peor pivote posible, pero esto ocurre con probabilidad extremadamente baja)
- ▶ **Probabilidad del peor caso:** La probabilidad de que el algoritmo tome tiempo  $O(n^2)$  es menor que  $O(1/n!)$ , prácticamente nula para  $n$  moderado.

## Ejemplo 4: Quicksort Aleatorizado - Análisis II

### Demostración Intuitiva

En cada nivel de recursión, la probabilidad de que el pivote divida el arreglo en proporciones desbalanceadas (ej. 99%-1%) es baja. En promedio, la profundidad del árbol de recursión es  $O(\log n)$ .

### Resumen de Complejidades

| Temporal (esperado) | Temporal (peor caso)                 | Espacial    |
|---------------------|--------------------------------------|-------------|
| $O(n \log n)$       | $O(n^2)$ (probabilidad $\approx 0$ ) | $O(\log n)$ |



## Ejemplo 4: Quicksort Aleatorizado - Análisis III

### Ventajas sobre Quicksort Determinista

- ▶ **Elimina dependencia del orden de entrada:** El algoritmo tiene el mismo rendimiento esperado para cualquier entrada, incluso para arreglos ya ordenados o invertidos.
- ▶ **No requiere información previa:** A diferencia de otras variantes (como elegir la mediana de tres), la selección aleatoria es simple y efectiva.
- ▶ **Overhead mínimo:** Solo requiere una llamada al generador de números aleatorios por nivel de recursión.

## Ejemplo 4: Quicksort Aleatorizado - Análisis IV

### Análisis de Probabilidad de Buen Rendimiento

Se puede demostrar que el tiempo esperado de Quicksort aleatorizado satisface:

$$\mathbb{E}[T(n)] \leq 2n \ln n + O(n)$$

Además, la probabilidad de que el tiempo exceda  $c \cdot n \log n$  es exponencialmente pequeña en  $c$ .

# Búsqueda y Optimización

# Búsqueda Local y con Candidatos

# Búsqueda Local - Fundamentos I

## Definición

La búsqueda local es una familia de algoritmos de optimización que exploran el espacio de soluciones moviéndose iterativamente a soluciones vecinas, generalmente aceptando solo movimientos que mejoran la función objetivo.

## Características de la Búsqueda Local

- ▶ **Exploración local:** Se mueve a soluciones vecinas en el espacio de búsqueda definido por una estructura de vecindario.
- ▶ **Basada en mejora:** Siempre busca mejorar la solución actual (greedy).
- ▶ **Sensible a inicialización:** La calidad de la solución final depende críticamente del punto de partida.
- ▶ **Eficiente:** Generalmente de complejidad polinomial por iteración.
- ▶ **Convergencia:** Termina en un óptimo local (no necesariamente global).

## Características de la Búsqueda con Candidatos Múltiples

- ▶ **Exploración múltiple:** Evalúa varios candidatos simultáneamente en cada iteración.
- ▶ **Menos propensa a óptimos locales:** Mayor diversificación gracias a la población de soluciones.
- ▶ **Mayor costo computacional:** Requiere evaluar y mantener múltiples soluciones.
- ▶ **Balance exploración-explotación:** Combina búsqueda amplia (diversidad) con refinamiento local (convergencia).
- ▶ **Ejemplos:** Algoritmos genéticos, PSO, colonias de hormigas.

# Ejemplo 1: Hill Climbing - Planteamiento del Problema I

## Problema

Optimización de funciones en espacios de búsqueda donde se dispone de una estructura de vecindario, pero los métodos exhaustivos son inviables. Se busca una solución de buena calidad en tiempo razonable mediante una estrategia de mejora iterativa.

## Inspiración

El algoritmo de ascenso de colina (Hill Climbing) se inspira en la metáfora de un escalador que intenta alcanzar la cima más alta en un terreno montañoso con niebla espesa. El escalador solo puede percibir la pendiente local y se mueve siempre hacia donde el terreno asciende, sin visión global del paisaje.

# Ejemplo 1: Hill Climbing - Planteamiento del Problema II

## Analogía

- ▶ **Solución actual:** Posición del escalador en la montaña.
- ▶ **Vecindario:** Posiciones alcanzables con un paso desde la posición actual.
- ▶ **Función objetivo:** Altitud del terreno.
- ▶ **Mejor vecino:** Dirección de máximo ascenso perceptible localmente.



# Ejemplo 1: Hill Climbing - Algoritmo

---

## Algoritmo 24 Hill Climbing (Ascenso de Colina)

---

**Require:**  $f$ : función objetivo a maximizar,  $s_0$ : solución inicial,  $N$ : función de vecindario

**Ensure:** Mejor solución encontrada  $s_{\text{best}}$  (óptimo local)

```
1: procedure HILLCLIMBING(f, s_0, N)
2: $s \leftarrow s_0$ //solución actual
3: while true do
4: $\text{mejorado} \leftarrow \text{false}$
5: for $s_{\text{new}} \in N(s)$ do
6: if $f(s_{\text{new}}) > f(s)$ then
7: $s \leftarrow s_{\text{new}}$
8: $\text{mejorado} \leftarrow \text{true}$
9: break //first-choice: tomar primer vecino mejor
10: end if
11: end for
12: if not mejorado then
13: return s //no hay vecinos mejores: óptimo local
14: end if
15: end while
16: end procedure
```

---

Existen variantes: *steepest ascent* evalúa todos los vecinos y toma el mejor; *first-choice* toma el primer vecino que mejora.

# Ejemplo 1: Hill Climbing - Análisis I

## Idea Clave

El algoritmo se mueve iterativamente hacia soluciones vecinas de mayor calidad, deteniéndose cuando alcanza un punto donde ningún vecino ofrece mejora (óptimo local). La simplicidad es su principal ventaja, pero carece de mecanismos para escapar de óptimos locales.

## Complejidad Temporal

- ▶ **Tiempo por iteración:**  $O(|N(s)| \cdot C_f)$ , donde  $|N(s)|$  es el tamaño del vecindario y  $C_f$  el costo de evaluar  $f$ .
- ▶ **Número de iteraciones:**  $I$ , depende del paisaje y la distancia al óptimo local.
- ▶ **Complejidad total:**  $O(I \cdot |N(s)| \cdot C_f)$ .

# Ejemplo 1: Hill Climbing - Análisis II

## Complejidad Espacial

- **Espacial:**  $O(1)$  (solo almacena la solución actual y la mejor encontrada).

## Resumen de Complejidades

| Temporal                      | Espacial |
|-------------------------------|----------|
| $O(I \cdot  N(s)  \cdot C_f)$ | $O(1)$   |

# Ejemplo 1: Hill Climbing - Análisis III

## Parámetros Críticos

- ▶ **Solución inicial**  $s_0$ : Determina qué óptimo local se alcanzará. Múltiples reinicios aleatorios mejoran la probabilidad de encontrar el óptimo global.
- ▶ **Estructura de vecindario**  $N(s)$ : Define la conectividad del espacio de búsqueda. Vecindarios más grandes pueden encontrar mejores soluciones pero aumentan el costo por iteración.
- ▶ **Estrategia de exploración**: *Steepest ascent* (mejor vecino) vs *first-choice* (primer vecino mejor).

## Ejemplo 1: Hill Climbing - Análisis IV

### Ventajas

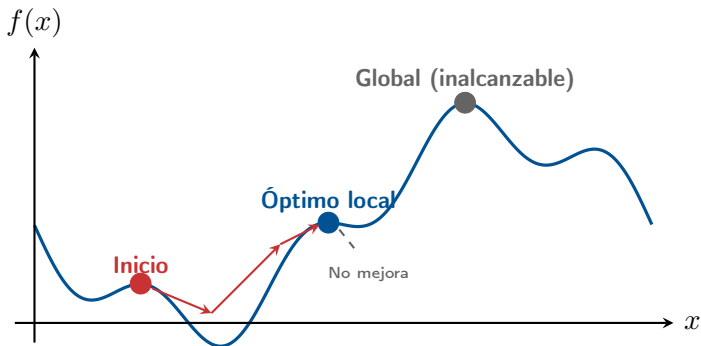
- ▶ **Simplicidad:** Fácil de implementar y entender.
- ▶ **Bajo consumo de memoria:** Solo requiere almacenar la solución actual.
- ▶ **Rápido en espacios suaves:** Converge rápidamente en funciones convexas o cerca de óptimos.

# Ejemplo 1: Hill Climbing - Análisis V

## Limitaciones

- ▶ **Óptimos locales:** No puede escapar de ellos, quedando atrapado.
- ▶ **Sensibilidad a la inicialización:** Diferentes puntos de partida pueden llevar a soluciones muy diferentes.
- ▶ **Dependencia del vecindario:** La definición del vecindario afecta críticamente el rendimiento.
- ▶ **Mesetas:** Si muchos vecinos tienen el mismo valor, el algoritmo puede deambular sin progreso.

## Ejemplo 1: Hill Climbing - Ilustración



- Inicio
- Óptimo local alcanzado
- Óptimo global inaccesible

## Ejemplo 1: Hill Climbing - Ilustración

### Interpretación

El algoritmo asciende greedily por la pendiente hasta alcanzar un pico desde donde ningún movimiento vecino mejora la solución. En este ejemplo, partiendo desde el punto indicado, Hill Climbing converge al óptimo local destacado en azul, sin poder alcanzar el óptimo global (en gris) porque requeriría descender primero (movimiento que el algoritmo no permite).



## Ejemplo 2: Recocido Simulado - Planteamiento del Problema I

### Problema

Optimización de funciones complejas en espacios de búsqueda multidimensionales, donde los algoritmos de búsqueda local clásicos (como Hill Climbing) pueden quedar atrapados en óptimos locales. Se busca una metaheurística que permita escapar de dichos óptimos mediante una exploración controlada.

### Inspiración Física

El recocido simulado (Simulated Annealing) se inspira en el proceso metalúrgico de recocido, donde un material se calienta a alta temperatura y luego se enfría lentamente para alcanzar un estado de mínima energía, eliminando defectos en su estructura cristalina.

## Ejemplo 2: Recocido Simulado - Planteamiento del Problema II

### Analogía

- ▶ **Temperatura alta:** Alta probabilidad de aceptar soluciones peores → exploración global.
- ▶ **Temperatura baja:** Baja probabilidad de aceptar soluciones peores → explotación local.
- ▶ **Enfriamiento gradual:** Transición suave de exploración a explotación, permitiendo escapar de óptimos locales al principio y converger al final.

## Ejemplo 2: Recocido Simulado - Algoritmo

---

### Algoritmo 25 Recocido Simulado (Simulated Annealing)

**Require:**  $f$ : función objetivo a minimizar,  $s_0$ : solución inicial,  $T_0$ : temperatura inicial,  $T_{\min}$ : temperatura mínima,  $\alpha$ : factor de enfriamiento ( $0 < \alpha < 1$ ),  $L$ : número de iteraciones por temperatura

**Ensure:** Mejor solución encontrada  $s_{\text{best}}$

```
1: procedure SIMULATEDANNEALING($f, s_0, T_0, T_{\min}, \alpha, L$)
2: $s \leftarrow s_0$ //solución actual
3: $s_{\text{best}} \leftarrow s$ //mejor solución encontrada
4: $T \leftarrow T_0$
5: while $T > T_{\min}$ do
6: for $i \leftarrow 1$ to L do
7: $s_{\text{new}} \leftarrow \text{GENERARVECINO}(s)$ //generar solución vecina
8: $\Delta E \leftarrow f(s_{\text{new}}) - f(s)$
9: if $\Delta E < 0$ then //mejor solución
10: $s \leftarrow s_{\text{new}}$
11: if $f(s) < f(s_{\text{best}})$ then
12: $s_{\text{best}} \leftarrow s$
13: end if
14: else //solución peor: aceptar con probabilidad, y sigue la distribución de Boltzmann
15: $p \leftarrow \exp(-\Delta E/T)$
16: if $\text{random}(0, 1) < p$ then
17: $s \leftarrow s_{\text{new}}$
18: end if
19: end if
20: end for
21: $T \leftarrow \alpha \cdot T$ //enfriamiento
22: end while
23: return s_{best}
24: end procedure
```

---

## Ejemplo 2: Recocido Simulado - Análisis I

### Idea Clave

La aceptación probabilística de soluciones peores permite escapar de óptimos locales. La temperatura controla la probabilidad: a alta temperatura, casi cualquier movimiento se acepta; a baja temperatura, solo mejoras.

### Complejidad Temporal

- ▶ **Tiempo por iteración:**  $O(C)$ , donde  $C$  es el costo de generar un vecino y evaluar  $f$ .
- ▶ **Número total de iteraciones:**  $L \cdot \lceil \log_{\alpha}(T_{\min}/T_0) \rceil$ .
- ▶ **Complejidad total:**  $O(L \cdot \log(T_0/T_{\min}) \cdot C)$ .

## Ejemplo 2: Recocido Simulado - Análisis II

### Complejidad Espacial

- **Espacial:**  $O(1)$  (solo se almacenan la solución actual, la mejor solución y algunos escalares).

### Resumen de Complejidades

| Temporal                                | Espacial |
|-----------------------------------------|----------|
| $O(L \cdot \log(T_0/T_{\min}) \cdot C)$ | $O(1)$   |

## Ejemplo 2: Recocido Simulado - Análisis III

### Parámetros Críticos

- ▶ **Temperatura inicial  $T_0$ :** Debe ser lo suficientemente alta para aceptar la mayoría de los movimientos iniciales (exploración). Típicamente se elige de modo que la probabilidad inicial de aceptación sea  $\approx 0.8$ .
- ▶ **Factor de enfriamiento  $\alpha$ :** Controla la rapidez del enfriamiento. Valores típicos:  $0.8 \leq \alpha \leq 0.99$ . Un  $\alpha$  más alto da mejor exploración pero más iteraciones.
- ▶ **Iteraciones por temperatura  $L$ :** Número de movimientos probados en cada nivel térmico. Suele ser proporcional al tamaño del vecindario o al problema.
- ▶ **Temperatura mínima  $T_{\min}$ :** Criterio de parada. Se puede fijar un valor pequeño o detener cuando no haya mejoras durante varias iteraciones.

## Ejemplo 2: Recocido Simulado - Análisis IV

### Ventajas sobre Hill Climbing

- ▶ **Escape de óptimos locales:** Acepta soluciones peores con probabilidad controlada.
- ▶ **Convergencia asintótica:** Bajo ciertas condiciones (enfriamiento logarítmico), converge al óptimo global con probabilidad 1.
- ▶ **Balance exploración-explotación:** La temperatura alta inicial permite exploración amplia; el enfriamiento gradual enfoca la búsqueda.

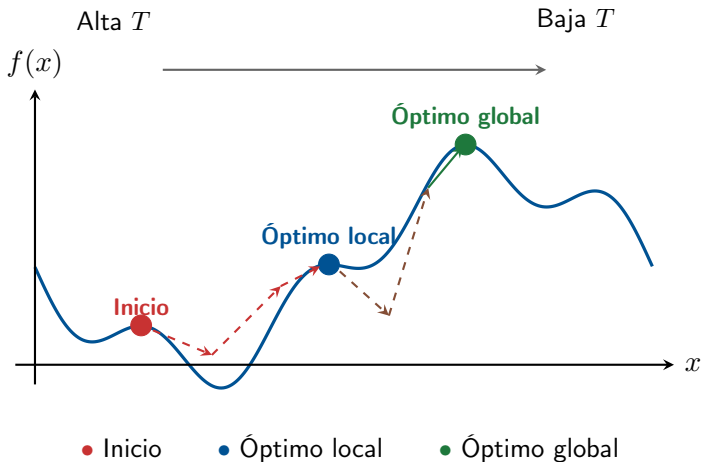
## Ejemplo 2: Recocido Simulado - Análisis V

### Limitaciones

- ▶ Sensibilidad a la elección de parámetros (temperatura inicial, factor de enfriamiento).
- ▶ Puede requerir muchas iteraciones para problemas grandes.
- ▶ No garantiza optimalidad global en la práctica (aunque sí en teoría con enfriamiento suficientemente lento).



## Ejemplo 2: Recocido Simulado - Ilustración



## Ejemplo 2: Recocido Simulado - Ilustración

### Interpretación

La temperatura alta inicial permite movimientos que empeoran la función, facilitando la salida del óptimo local. A medida que la temperatura disminuye, la búsqueda se restringe a mejoras locales, convergiendo finalmente al óptimo global.

## Ejemplo 3: Tabu Search - Planteamiento del Problema I

### Problema

Superar la limitación de los óptimos locales en la búsqueda local, permitiendo escapes mediante el uso de memoria que registra movimientos recientes para evitar ciclos y fomentar la exploración de nuevas regiones.

### Inspiración

La búsqueda tabú (Tabu Search) se inspira en estrategias de memoria humana a corto plazo. Así como una persona evita repetir acciones recientes que no condujeron al éxito, el algoritmo mantiene una "lista tabú" de movimientos prohibidos temporalmente para evitar retrocesos y ciclos.

## Ejemplo 3: Tabu Search - Planteamiento del Problema II

### Analogía

- ▶ **Lista tabú:** Memoria de corto plazo que registra movimientos recientes.
- ▶ **Criterio de aspiración:** Regla que permite violar la prohibición si el movimiento conduce a una solución excepcionalmente buena.
- ▶ **Diversificación:** Estrategias para explorar regiones no visitadas del espacio de búsqueda.
- ▶ **Intensificación:** Exploración detallada de regiones prometedoras.

## Ejemplo 3: Tabu Search - Algoritmo

---

### Algoritmo 26 Tabu Search (Búsqueda Tabú)

---

**Require:**  $f$ : función objetivo a minimizar,  $s_0$ : solución inicial,  $T$ : tamaño de lista tabú (tenure),  $maxIter$ : máximo de iteraciones

**Ensure:** Mejor solución encontrada  $s_{best}$

```
1: procedure TABUSEARCH($f, s_0, T, maxIter$)
2: $s \leftarrow s_0$ //solución actual
3: $s_{best} \leftarrow s$ //mejor solución encontrada
4: $tabuList \leftarrow$ cola vacía (capacidad T)
5: for $iter \leftarrow 1$ to $maxIter$ do
6: $vecinos \leftarrow$ GENERARVECINOS(s)
7: $mejorVecino \leftarrow$ SELECCIONARMEJORNO-
 TABU($vecinos, tabuList$)
8: if $mejorVecino$ es null then //todos los vecinos son tabú
9: $mejorVecino \leftarrow$ APLICARCRITERIOASPIRA-
 CION($vecinos, s_{best}$)
10: end if
11: $s \leftarrow mejorVecino$
12: ACTUALIZARTABULIST($tabuList, s, T$) //registrar movimiento
13: if $f(s) < f(s_{best})$ then
14: $s_{best} \leftarrow s$
15: end if
16: end for
17: return s_{best}
18: end procedure
```

---

## Ejemplo 3: Tabu Search - Análisis I

### Idea Clave

Tabu Search extiende la búsqueda local con memoria adaptativa. La lista tabú evita ciclos al prohibir temporalmente movimientos recientes, permitiendo escapar de óptimos locales mediante la aceptación de soluciones peores pero no tabú.

### Complejidad Temporal

- ▶ **Tiempo por iteración:**  $O(|N(s)| \cdot C_f + T)$ , donde  $|N(s)|$  es tamaño del vecindario y  $T$  el costo de verificar la lista tabú.
- ▶ **Número de iteraciones:**  $maxIter$ , fijo por configuración.
- ▶ **Complejidad total:**  $O(maxIter \cdot (|N(s)| \cdot C_f + T))$ .

## Ejemplo 3: Tabu Search - Análisis II

### Complejidad Espacial

- **Espacial:**  $O(T)$  para la lista tabú, más  $O(1)$  para soluciones actual y mejor.

### Resumen de Complejidades

| Temporal                                   | Espacial |
|--------------------------------------------|----------|
| $O(\maxIter \cdot ( N(s)  \cdot C_f + T))$ | $O(T)$   |

## Ejemplo 3: Tabu Search - Análisis III

### Parámetros Críticos

- ▶ **Tamaño de lista tabú  $T$  (tenure):** Controla la duración de la memoria. Valores pequeños pueden no evitar ciclos; valores grandes pueden restringir demasiado la búsqueda. Típicamente  $T \in [5, 20]$  o proporcional al tamaño del problema.
- ▶ **Criterio de aspiración:** Generalmente se acepta un movimiento tabú si conduce a una solución mejor que la mejor conocida ( $f(s) < f(s_{\text{best}})$ ).
- ▶ **Estrategias de diversificación:** Incentivan la exploración de regiones no visitadas, por ejemplo, penalizando movimientos frecuentes.
- ▶ **Estrategias de intensificación:** Refuerzan la búsqueda en regiones prometedoras, por ejemplo, reiniciando desde la mejor solución con parámetros ajustados.



## Ejemplo 3: Tabu Search - Análisis IV

### Ventajas sobre Hill Climbing

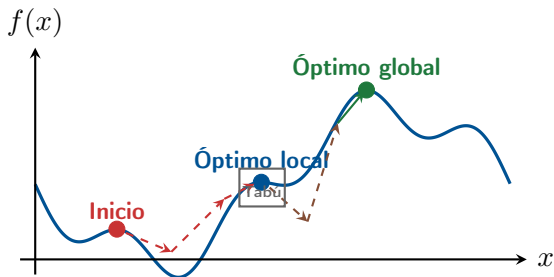
- ▶ **Escape de óptimos locales:** La lista tabú permite aceptar movimientos que empeoran la solución para explorar nuevas regiones.
- ▶ **Memoria adaptativa:** Aprende de la historia de la búsqueda para evitar ciclos y fomentar diversificación.
- ▶ **Flexibilidad:** Puede combinarse con otras metaheurísticas y adaptarse a diversos problemas.

## Ejemplo 3: Tabu Search - Análisis V

### Limitaciones

- ▶ **Configuración de parámetros:** El rendimiento es sensible al tamaño de lista tabú y criterios de aspiración.
- ▶ **Costo de memoria:** Para problemas grandes, almacenar soluciones completas puede ser costoso (se usan atributos).
- ▶ **Convergencia:** No garantiza optimalidad global y puede requerir muchas iteraciones.

## Ejemplo 3: Tabu Search - Ilustración



● Inicio

● Óptimo local

● Óptimo global

Rectángulo: Movimiento tabú (prohibido)

## Ejemplo 3: Tabu Search - Ilustración

### Interpretación

A diferencia de Hill Climbing, Tabu Search puede escapar del óptimo local (azul) aceptando un movimiento que empeora temporalmente la función (flechas discontinuas). La lista tabú impide regresar inmediatamente al óptimo local recién abandonado, permitiendo que la búsqueda continúe hacia regiones no exploradas y eventualmente alcance el óptimo global (verde).

# Algoritmos Bioinspirados y Evolutivos

# Fundamentos de Algoritmos Bioinspirados I

## Definición y Motivación

Los algoritmos bioinspirados constituyen una familia de metaheurísticas de optimización que emulan procesos naturales, biológicos o físicos para resolver problemas computacionales complejos. Se utilizan cuando los métodos exactos son inviables debido a la alta dimensionalidad o complejidad del problema.

# Fundamentos de Algoritmos Bioinspirados II

## Taxonomía

- ▶ **Basados en evolución:** Algoritmos Genéticos, Estrategias Evolutivas, Programación Genética.
- ▶ **Basados en enjambre:** Optimización por Enjambre de Partículas (PSO), Colonias de Hormigas (ACO).
- ▶ **Basados en ecología:** Colonias de Abejas (ABC), Optimización por Lobo Gris (GWO).
- ▶ **Basados en física:** Recocido Simulado, Algoritmo Gravitacional, Búsqueda de Armónicos.

# Fundamentos de Algoritmos Bioinspirados III

## Características Comunes

- ▶ **Búsqueda poblacional:** Mantienen múltiples soluciones simultáneamente, permitiendo exploración paralela del espacio.
- ▶ **Operadores estocásticos:** Utilizan aleatoriedad controlada para diversificar la búsqueda.
- ▶ **No requieren gradientes:** Funcionan con funciones no diferenciables, discontinuas o multimodales.
- ▶ **Robustez:** Menos sensibles a condiciones iniciales que los métodos deterministas.
- ▶ **Paralelización natural:** La evaluación de individuos puede distribuirse fácilmente.



## Descripción General

Un algoritmo genético es una metaheurística de optimización poblacional inspirada en la evolución natural que explora espacios de búsqueda complejos mediante operadores de selección, recombinación (cruce) y mutación.

## Analogía Biológica

- ▶ **Individuo:** Una solución candidata representada por un cromosoma.
- ▶ **Población:** Conjunto de soluciones actuales.
- ▶ **Aptitud (fitness):** Valor de la función objetivo.
- ▶ **Selección:** Los individuos más aptos tienen mayor probabilidad de reproducirse.
- ▶ **Cruce:** Combinación de material genético de dos padres.
- ▶ **Mutación:** Alteración aleatoria de un individuo.
- ▶ **Generación:** Iteración del proceso evolutivo.

## Ciclo Evolutivo Básico

1. **Inicialización:** Generar población aleatoria de  $P_{size}$  individuos.
2. **Evaluación:** Calcular aptitud (fitness) de cada individuo.
3. **Selección:** Elegir padres según aptitud (rueda de ruleta, torneo, ranking).
4. **Cruce:** Combinar información genética con probabilidad  $p_c$  para crear hijos.
5. **Mutación:** Introducir variaciones aleatorias con probabilidad  $p_m$  por gen.
6. **Reemplazo:** Formar nueva generación (reemplazo total o generacional).
7. **Repetir** hasta criterio de parada (generaciones, convergencia, tiempo).

## Representación de Soluciones

- ▶ **Binaria:** Cadenas de bits  $[0,1]^n$ . Útil para problemas de selección, mochila, etc.
- ▶ **Real:** Vectores de números reales  $\mathbb{R}^n$ . Para optimización continua.
- ▶ **Permutación:** Ordenamientos para problemas combinatorios (TSP, scheduling).
- ▶ **Estructuras complejas:** Árboles (programación genética), grafos, etc.

## Selección por Ruleta (Proporcional)

Cada individuo  $i$  tiene probabilidad de selección:

$$p_i = \frac{f_i}{\sum_{j=1}^{P_{size}} f_j}$$

donde  $f_i$  es la aptitud del individuo  $i$ .

**Ventaja:** Simple, intuitivo. **Desventaja:** Sensible a escala de aptitud; individuos muy buenos dominan rápidamente.

## Selección por Torneo

1. Seleccionar  $k$  individuos aleatoriamente (típicamente  $k = 2$  o  $3$ ).
2. Elegir el de mayor aptitud como padre.

**Ventaja:** Control de presión selectiva mediante tamaño de torneo  $k$ . **Complejidad:**  $O(k)$  por selección. **Balance:** Mayor  $k \rightarrow$  mayor presión selectiva  $\rightarrow$  convergencia más rápida (pero menos diversidad).

# Algoritmos Genéticos - Operadores de Selección III

## Selección por Ranking

Ordenar individuos por aptitud y asignar probabilidades basadas en el rango, no en el valor absoluto.

$$p_i = \frac{2 \cdot rank_i}{P_{size}(P_{size} + 1)} \quad (\text{lineal})$$

**Ventaja:** Elimina dependencia de la escala de aptitud; evita que superindividuos dominen prematuramente.

## Elitismo

Preservar los  $e$  mejores individuos de la generación actual para la siguiente, garantizando que la mejor solución no se pierda.

**Beneficio:** Convergencia monótona (no empeora). **Riesgo:** Exceso de elitismo puede causar convergencia prematura.

## Cruce en Un Punto (Binario)

Padre 1: [1 0 1 0 1 0]

Padre 2: [0 1 0 1 0 1]

Hijo: [1 0 1 1 0 1]

Se elige un punto de corte aleatorio. El hijo toma el prefijo del Padre 1 y el sufijo del Padre 2.



# Algoritmos Genéticos - Operadores de Cruce II

## Cruce en Dos Puntos

Padre 1: [1 | 0 1 0 | 1 0]

Padre 2: [0 | 1 0 1 | 0 1]

Hijo: [1 1 0 1 1 0]

Se eligen dos puntos de corte. El hijo toma los extremos del Padre 1 y la sección central del Padre 2.

# Algoritmos Genéticos - Operadores de Cruce III

## Cruce Uniforme

Se genera una máscara binaria aleatoria de la misma longitud que los cromosomas. Donde la máscara es 1, se toma del Padre 1; donde es 0, del Padre 2.

**Ventaja:** Mayor diversidad genética que cruce en puntos. **Desventaja:** Puede romper esquemas útiles (building blocks).

## Cruce para Representación Real

- ▶ **Cruce aritmético:**  $hijo = \alpha \cdot padre_1 + (1 - \alpha) \cdot padre_2$  con  $\alpha \in [0, 1]$ .
- ▶ **Cruce BLX- $\alpha$ :** Hijo en intervalo  $[min - \alpha \cdot d, max + \alpha \cdot d]$  donde  $d = max - min$ .

## Mutación Bit-Flip (Binaria)

Antes: [1, 0, 1, 0, 1]

Después: [1, 0, 0, 0, 1]

**Tasa típica:**  $p_m \in [0.001, 0.01]$  por bit.

Se selecciona un bit al azar y se invierte su valor (0→1 o 1→0).

## Mutación Gaussiana (Real)

Se añade ruido gaussiano  $\mathcal{N}(0, \sigma)$  a cada gen con probabilidad  $p_m$ :

Después:  $[x_1 + \mathcal{N}(0, \sigma), x_2 + \mathcal{N}(0, \sigma), \dots]$

Permite pequeñas variaciones continuas en representación real. **Varianza**  $\sigma^2$ :  
Controla la magnitud de la mutación.

## Mutación por Intercambio (Permutación)

Para problemas de ordenamiento (TSP), se intercambian dos posiciones aleatorias:

Antes:  $[A, B, C, D, E]$

Después:  $[A, D, C, B, E]$

## Balance Cruce-Mutación

- ▶ **Cruce:** Explotación (refina soluciones prometedoras combinando características).
- ▶ **Mutación:** Exploración (introduce nueva diversidad, evita convergencia prematura).
- ▶ **Balance crítico:**
  - ▶ Demasiada explotación → convergencia prematura a óptimo local.
  - ▶ Demasiada exploración → el algoritmo no converge (búsqueda aleatoria).

# Algoritmo Genético Básico

---

## Algoritmo 27 Algoritmo Genético Básico (Generacional)

---

**Require:**  $P_{size}$ ,  $G$ ,  $p_c$ ,  $p_m$

**Ensure:** Mejor individuo encontrado

```
1: $P \leftarrow \text{INICIALIZAR_POBLACION}(P_{size})$
2: $\text{EVALUAR_APTITUD}(P)$
3: $mejor_global \leftarrow \text{MEJOR_INDIVIDUO}(P)$
4: for $gen \leftarrow 1$ to G do
5: $P_{nueva} \leftarrow \emptyset$
6: while $|P_{nueva}| < P_{size}$ do
7: $padre_1 \leftarrow \text{SELECCIONAR}(P)$
8: $padre_2 \leftarrow \text{SELECCIONAR}(P)$
9: if $\text{random}() < p_c$ then
10: $hijo_1, hijo_2 \leftarrow \text{CRUZAR}(padre_1, padre_2)$
11: else
12: $hijo_1 \leftarrow padre_1, hijo_2 \leftarrow padre_2$
13: end if
14: $hijo_1 \leftarrow \text{MUTAR}(hijo_1, p_m)$
15: $hijo_2 \leftarrow \text{MUTAR}(hijo_2, p_m)$
16: $P_{nueva} \leftarrow P_{nueva} \cup \{hijo_1, hijo_2\}$
17: end while
18: $P \leftarrow P_{nueva}$
19: $\text{EVALUAR_APTITUD}(P)$
20: $mejor_actual \leftarrow \text{MEJOR_INDIVIDUO}(P)$
21: if $\text{fitness}(mejor_actual) > \text{fitness}(mejor_global)$ then
22: $mejor_global \leftarrow mejor_actual$
23: end if
24: end for
25: return $mejor_global$
```

---

# Algoritmo Genético - Análisis de Complejidad I

## Complejidad Temporal

| Operación                         | Costo                                                 |
|-----------------------------------|-------------------------------------------------------|
| Evaluación de aptitud             | $O(P_{size} \cdot C_f)$                               |
| Selección ( $P_{size}$ padres)    | $O(P_{size} \cdot k)$ para torneo tamaño $k$          |
| Cruce ( $P_{size}/2$ pares)       | $O(P_{size} \cdot L)$ donde $L$ es longitud cromosoma |
| Mutación ( $P_{size}$ individuos) | $O(P_{size} \cdot L \cdot p_m)$                       |
| Total ( $G$ generaciones)         | $O(G \cdot P_{size} \cdot (C_f + L))$                 |



# Algoritmo Genético - Análisis de Complejidad II

## Resumen de Complejidades

| Temporal                              | Espacial              |
|---------------------------------------|-----------------------|
| $O(G \cdot P_{size} \cdot (C_f + L))$ | $O(P_{size} \cdot L)$ |

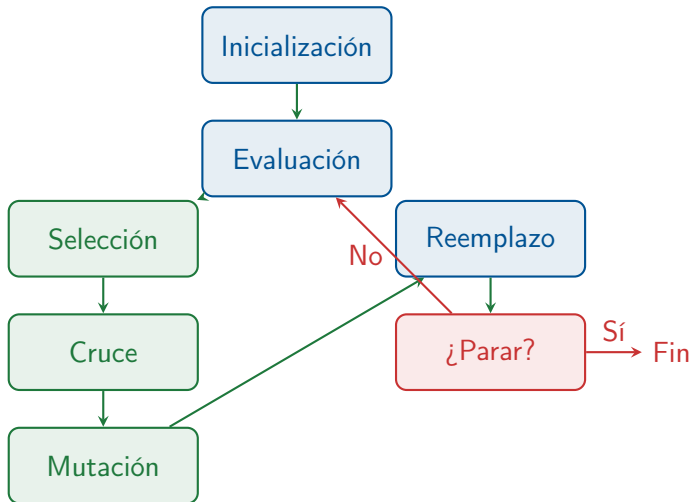
## Complejidad Espacial

| Componente                    | Espacio               |
|-------------------------------|-----------------------|
| Población actual              | $O(P_{size} \cdot L)$ |
| Población temporal (opcional) | $O(P_{size} \cdot L)$ |
| Vector de aptitud             | $O(P_{size})$         |
| <b>Total</b>                  | $O(P_{size} \cdot L)$ |

## Parámetros Críticos

- ▶  $P_{size}$ : Tamaño de población (20-100 típico, 100-1000 para problemas complejos).
- ▶  $G$ : Número de generaciones (criterio de parada).
- ▶  $p_c$ : Tasa de cruce (típicamente 0.6-0.9).
- ▶  $p_m$ : Tasa de mutación (típicamente 0.001-0.01 por gen).
- ▶  $L$ : Longitud del cromosoma (dimensión del problema).

# Algoritmo Genético - Ilustración del Ciclo Evolutivo



# Algoritmo Genético - Ilustración del Ciclo Evolutivo I

## Interpretación del Ciclo

El ciclo evolutivo se repite hasta alcanzar el criterio de parada (generaciones máximas, convergencia de la población, o tiempo límite). El elitismo (no mostrado) preserva los mejores individuos entre generaciones.

# Optimización por Enjambre de Partículas (PSO) - Fundamentos I

## Inspiración Biológica

PSO (Particle Swarm Optimization) emula el comportamiento social de bandadas de aves o cardúmenes de peces, donde individuos se mueven ajustando su trayectoria según:

- ▶ Su mejor posición personal encontrada (memoria individual).
- ▶ La mejor posición encontrada por el enjambre (memoria colectiva).

# Optimización por Enjambre de Partículas (PSO) - Fundamentos II

## Componentes del Algoritmo

- ▶ **Partícula:** Una solución candidata con posición  $\mathbf{x}_i$  y velocidad  $\mathbf{v}_i$ .
- ▶ **pbest:** Mejor posición personal histórica de la partícula.
- ▶ **gbest:** Mejor posición global del enjambre.
- ▶ **Topologías:** Estructura de comunicación entre partículas (global, local, anillo, etc.).

# Optimización por Enjambre de Partículas (PSO) - Fundamentos III

## Ecuaciones de Movimiento

### Actualización de velocidad:

$$\mathbf{v}_i^{t+1} = w \cdot \mathbf{v}_i^t + c_1 \cdot \mathbf{r}_1 \odot (\mathbf{pbest}_i - \mathbf{x}_i^t) + c_2 \cdot \mathbf{r}_2 \odot (\mathbf{gbest} - \mathbf{x}_i^t)$$

### Actualización de posición:

$$\mathbf{x}_i^{t+1} = \mathbf{x}_i^t + \mathbf{v}_i^{t+1}$$

donde  $\odot$  denota producto elemento a elemento.



# Optimización por Enjambre de Partículas (PSO) - Fundamentos IV

## Parámetros

- ▶  $w$ : Peso de inercia (controla influencia de velocidad previa).
- ▶  $c_1, c_2$ : Coeficientes cognitivo y social (aceleración).
- ▶  $\mathbf{r}_1, \mathbf{r}_2$ : Vectores aleatorios en  $[0, 1]^n$  (estocasticidad).

# Optimización por Enjambre de Partículas (PSO) - Algoritmo

---

## Algoritmo 28 Optimización por Enjambre de Partículas (PSO)

---

**Require:**  $N$ : número de partículas,  $max\_iter$ ,  $w$ ,  $c_1$ ,  $c_2$

**Ensure:** Mejor posición encontrada **gbest**

```
1: procedure PSO(N , max_iter , w , c_1 , c_2)
2: Inicializar posiciones $\mathbf{x}_i$ y velocidades $\mathbf{v}_i$ aleatoriamente
3: $\mathbf{pbest}_i \leftarrow \mathbf{x}_i$ para cada partícula
4: $\mathbf{gbest} \leftarrow \arg \min_i f(\mathbf{pbest}_i)$
5: for $t \leftarrow 1$ to max_iter do
6: for $i \leftarrow 1$ to N do
7: $\mathbf{r}_1 \leftarrow$ vector aleatorio $[0, 1]^n$
8: $\mathbf{r}_2 \leftarrow$ vector aleatorio $[0, 1]^n$
9: $\mathbf{v}_i \leftarrow w \cdot \mathbf{v}_i + c_1 \cdot \mathbf{r}_1 \odot (\mathbf{pbest}_i - \mathbf{x}_i) + c_2 \cdot \mathbf{r}_2 \odot (\mathbf{gbest} - \mathbf{x}_i)$
10: $\mathbf{x}_i \leftarrow \mathbf{x}_i + \mathbf{v}_i$
11: if $f(\mathbf{x}_i) < f(\mathbf{pbest}_i)$ then
12: $\mathbf{pbest}_i \leftarrow \mathbf{x}_i$
13: if $f(\mathbf{pbest}_i) < f(\mathbf{gbest})$ then
14: $\mathbf{gbest} \leftarrow \mathbf{pbest}_i$
15: end if
16: end if
17: end for
18: end for
19: return $\mathbf{gbest}$
20: end procedure
```

---

# PSO - Análisis de Complejidad I

## Complejidad Temporal

- ▶ Cada iteración procesa  $N$  partículas.
- ▶ Por partícula: actualización de velocidad ( $O(n)$ ), actualización de posición ( $O(n)$ ), evaluación de aptitud ( $O(C_f)$ ).

$$T(max\_iter, N, n) = O(max\_iter \cdot N \cdot (n + C_f))$$

## Resumen de Complejidades

| Temporal                               | Espacial       |
|----------------------------------------|----------------|
| $O(max\_iter \cdot N \cdot (n + C_f))$ | $O(N \cdot n)$ |

## Complejidad Espacial

- ▶ Posiciones:  $O(N \cdot n)$
- ▶ Velocidades:  $O(N \cdot n)$
- ▶ Mejores personales:  $O(N \cdot n)$
- ▶ Mejor global:  $O(n)$

$$M(N, n) = O(N \cdot n)$$

## Ventajas y Desventajas

### ▶ **Ventajas:**

- ▶ Fácil implementación, pocos parámetros.
- ▶ Convergencia rápida en problemas continuos.
- ▶ Memoria social y personal guían la búsqueda.

### ▶ **Desventajas:**

- ▶ Convergencia prematura a óptimos locales.
- ▶ Sensibilidad a parámetros  $(w, c_1, c_2)$ .
- ▶ No garantiza optimalidad global.

# Comparativa de Algoritmos Bioinspirados I

## Resumen Comparativo

| Algoritmo | Tipo      | Parámetros            | Fortalezas                              | Debilidades                           |
|-----------|-----------|-----------------------|-----------------------------------------|---------------------------------------|
| AG        | Evolutivo | $P, p_c, p_m$         | Versatilidad, representaciones variadas | Convergencia lenta, muchos parámetros |
| ACO       | Enjambre  | $\alpha, \beta, \rho$ | Excelente en problemas combinatorios    | Lento en espacios continuos           |
| PSO       | Enjambre  | $w, c_1, c_2$         | Rápido, pocos parámetros                | Convergencia prematura                |
| ABC       | Enjambre  | $limit, size$         | Balance exploración-explotación         | Sensible a parámetros                 |

# Comparativa de Algoritmos Bioinspirados II

## Criterios de Selección

- ▶ **Problemas discretos/combinatorios:** ACO o Algoritmos Genéticos (con representación adecuada).
- ▶ **Problemas continuos:** PSO o ABC (por su manejo natural de variables reales).
- ▶ **Espacios multimodales:** Algoritmos Genéticos con alta diversidad o niching.
- ▶ **Tiempo limitado:** PSO por su convergencia rápida (aunque puede ser prematura).
- ▶ **Problemas con restricciones:** AG con penalización o operadores específicos.

# Comparativa de Algoritmos Bioinspirados III

## Factores a Considerar

- ▶ Dimensionalidad del problema.
- ▶ Presupuesto computacional disponible.
- ▶ Precisión requerida.
- ▶ Existencia de gradientes o estructura especial.
- ▶ Facilidad de implementación.



## Ingeniería y Diseño

- ▶ **Diseño de antenas:** Optimización de forma y patrón de radiación mediante AG o PSO.
- ▶ **Diseño estructural:** Minimización de peso con restricciones de resistencia usando AG.
- ▶ **Planificación de rutas:** Logística, transporte, drones con ACO o AG.
- ▶ **Diseño de circuitos:** Optimización de topología y parámetros.

## Ciencia de Datos

- ▶ **Selección de características:** Reducir dimensionalidad manteniendo rendimiento (AG, PSO).
- ▶ **Ajuste de hiperparámetros:** Optimización de modelos de ML (AG, PSO, recocido simulado).
- ▶ **Clustering:** Agrupamiento no supervisado (AG con codificación de centros, ACO para routing).
- ▶ **Redes neuronales:** Optimización de pesos y arquitectura (AG, PSO).

# Aplicaciones Prácticas de Metaheurísticas III

## Bioinformática

- ▶ **Alineamiento de secuencias:** ADN, proteínas (AG con representación de permutaciones).
- ▶ **Predicción de estructura:** Plegamiento de proteínas (AG, recocido simulado).
- ▶ **Diseño de fármacos:** Docking molecular (AG, PSO).
- ▶ **Análisis filogenético:** Construcción de árboles evolutivos.

## Finanzas

- ▶ Optimización de carteras de inversión.
- ▶ Predicción de series temporales.
- ▶ Trading algorítmico.

# Optimización avanzada

# Optimización Combinatoria

## Descripción General

La optimización combinatoria se ocupa de encontrar la mejor solución en un conjunto finito de posibles soluciones, donde las soluciones son combinaciones discretas de elementos. Estos problemas a menudo son NP-completeness, lo que significa que no se conocen algoritmos eficientes (polinomiales) para encontrar soluciones óptimas en todos los casos.

## Características Fundamentales

- ▶ **Espacio discreto:** Las soluciones son combinaciones, permutaciones o subconjuntos de elementos discretos.
- ▶ **Función objetivo:** Cada solución tiene asociado un valor numérico que se desea maximizar o minimizar.
- ▶ **Restricciones:** Condiciones que definen el subconjunto de soluciones factibles dentro del espacio total.
- ▶ **Combinatoriedad:** El número de soluciones crece exponencialmente con el tamaño del problema (ej.  $n!$  para permutaciones,  $2^n$  para subconjuntos).

## Desafíos Computacionales

- ▶ **Explosión combinatoria:** Espacios de búsqueda exponencialmente grandes hacen inviable la exploración exhaustiva para instancias de tamaño moderado.
- ▶ **NP-completitud:** Para muchos problemas, no existen algoritmos polinomiales conocidos (y se cree que no pueden existir).
- ▶ **Compromiso optimalidad-eficiencia:** Se requiere un balance entre calidad de la solución y tiempo de cómputo.
- ▶ **Estrategias requeridas:** Poda (branch and bound), heurísticas, meta-heurísticas, algoritmos de aproximación.



## Aplicaciones

- ▶ Logística y transporte (rutas, asignación).
- ▶ Planificación y scheduling (producción, horarios).
- ▶ Diseño de redes (telecomunicaciones, distribución).
- ▶ Bioinformática (alineamiento, plegamiento).

# Problemas Clásicos de Optimización Combinatoria I

## Problema del Viajante (TSP - Traveling Salesman Problem)

- ▶ **Entrada:** Un conjunto de  $n$  ciudades y distancias  $d_{ij}$  entre cada par.
- ▶ **Salida:** Un recorrido (ciclo hamiltoniano) que visite cada ciudad exactamente una vez y regrese a la ciudad de origen, minimizando la distancia total recorrida.
- ▶ **Tamaño del espacio:**  $(n - 1)!/2$  recorridos distintos (considerando simetría y punto de partida fijo).
- ▶ **Complejidad:** NP-complete.

# Problemas Clásicos de Optimización Combinatoria II

## Problema de la Mochila (Knapsack Problem)

- ▶ **Entrada:**  $n$  objetos con peso  $w_i$  y valor  $v_i$ , y una mochila con capacidad  $W$ .
- ▶ **Salida:** Seleccionar un subconjunto de objetos que maximice el valor total sin exceder la capacidad.
- ▶ **Tamaño del espacio:**  $2^n$  subconjuntos posibles.
- ▶ **Complejidad:** NP-complete (versión 0-1).

# Problemas Clásicos de Optimización Combinatoria III

## Problema de Asignación (Assignment Problem)

- ▶ **Entrada:**  $n$  tareas y  $n$  agentes, con costo  $c_{ij}$  de asignar tarea  $i$  al agente  $j$ .
- ▶ **Salida:** Asignación biyectiva (cada tarea a un agente) que minimice el costo total.
- ▶ **Tamaño del espacio:**  $n!$  asignaciones posibles.
- ▶ **Complejidad:** Resoluble en  $O(n^3)$  con el algoritmo húngaro (es uno de los pocos problemas combinatorios con solución polinomial).

# Problemas Clásicos de Optimización Combinatoria IV

## Problema de Secuenciamiento (Scheduling)

- ▶ **Entrada:**  $n$  trabajos con tiempos de procesamiento, fechas de entrega, etc., en  $m$  máquinas.
- ▶ **Salida:** Orden de procesamiento que optimice una métrica (makespan, tardanza total, etc.).
- ▶ **Complejidad:** Muchas variantes son NP-completas.

# Problemas Clásicos de Optimización Combinatoria V

## Problema de Cubrimiento (Set Cover)

- ▶ **Entrada:** Un conjunto universo  $U$  y una familia de subconjuntos  $\mathcal{S}$  con costos  $c(S)$ .
- ▶ **Salida:** Subcolección de  $\mathcal{S}$  que cubra todos los elementos de  $U$  con mínimo costo total.
- ▶ **Complejidad:** NP-complete.

## Complejidad Computacional

- ▶ La mayoría de estos problemas son NP-completeness o NP-hardness .
- ▶ No existen algoritmos polinomiales conocidos (y se cree que no pueden existir).
- ▶ En la práctica, se emplean:
  - ▶ Algoritmos exactos (branch and bound) para instancias pequeñas.
  - ▶ Algoritmos de aproximación con garantías de calidad.
  - ▶ Metaheurísticas (AG, ACO, PSO, recocido simulado) para instancias grandes.

# Ejemplo: Problema del Viajante (TSP) - Algoritmo de Fuerza Bruta

---

## Algoritmo 29 TSP - Búsqueda Exhaustiva (Fuerza Bruta)

---

**Require:** Matriz de distancias  $dist[0..n-1][0..n-1]$ , ciudad inicial  $start$

**Ensure:** Recorrido óptimo y distancia mínima

```
1: procedure TSP_BRUTEFORCE($dist, start$)
2: $cities \leftarrow$ lista de ciudades excepto $start$
3: $min_distance \leftarrow \infty$
4: $best_tour \leftarrow \text{null}$
5: for cada permutación $perm$ de $cities$ do
6: $current_distance \leftarrow 0$
7: $current \leftarrow start$
8: for $i \leftarrow 0$ to $|perm| - 1$ do
9: $next \leftarrow perm[i]$
10: $current_distance \leftarrow current_distance +$
11: $dist[current][next]$
12: $current \leftarrow next$
13: end for
14: $current_distance \leftarrow current_distance + dist[current][start]$
15: if $current_distance < min_distance$ then
16: $min_distance \leftarrow current_distance$
17: $best_tour \leftarrow [start] + perm + [start]$
18: end if
19: end for
20: return $best_tour, min_distance$
21: end procedure
```

---

**Complejidad:**  $O((n-1)!)$  — inviable para  $n > 15$  aproximadamente.



# TSP - Análisis de Complejidad y Enfoques I

## Complejidad del TSP

El Problema del Viajante es uno de los problemas NP-completeness más estudiados:

- ▶ **Espacio de búsqueda:**  $(n - 1)!/2$  tours distintos (para TSP simétrico).
- ▶ **Fuerza bruta:**  $O(n!)$  — impráctico para  $n > 15$ .
- ▶ **Programación dinámica (Held-Karp):**  $O(n^2 2^n)$  — factible hasta  $n \approx 25$ .
- ▶ **No se conoce algoritmo polinomial:** Se cree que no existe ( $P \neq NP$ ).

# TSP - Análisis de Complejidad y Enfoques II

## Crecimiento del Espacio de Búsqueda

| $n$ | $(n - 1)!/2$          | Tiempo estimado (1 ns/perm) |
|-----|-----------------------|-----------------------------|
| 10  | 181,440               | 0.18 ms                     |
| 15  | $4.36 \times 10^{10}$ | 43.6 s                      |
| 20  | $6.08 \times 10^{16}$ | 1,928 años                  |
| 25  | $3.10 \times 10^{23}$ | $9.8 \times 10^6$ años      |

# TSP - Análisis de Complejidad y Enfoques III

## Enfoques para Resolver TSP

### ▶ Algoritmos exactos:

- ▶ Branch and bound con cotas (Held-Karp).
- ▶ Programación dinámica (Held-Karp,  $O(n^2 2^n)$ ).
- ▶ Útiles para instancias hasta  $n \approx 100$  con técnicas avanzadas.

### ▶ Algoritmos de aproximación:

- ▶ Heurística del vecino más cercano:  $O(n^2)$ , ratio 1.25 en promedio.
- ▶ Algoritmo de Christofides: garantía  $3/2$ -óptimo para TSP métrico.

### ▶ Metaheurísticas:

- ▶ Algoritmos genéticos con codificación de permutaciones.
- ▶ Colonias de hormigas (ACO) específicas para TSP.
- ▶ Recocido simulado, búsqueda tabú.

# Ejemplo: Problema de la Mochila - Algoritmo de Programación Dinámica

---

## Algoritmo 30 Mochila 0/1 - Programación Dinámica

---

**Require:**  $weights[0..n-1]$ ,  $values[0..n-1]$ , capacidad  $W$

**Ensure:** Valor máximo y objetos seleccionados

```
1: procedure KNAPSACK_DP($weights, values, W$)
2: $n \leftarrow |weights|$
3: $dp \leftarrow$ matriz $(n+1) \times (W+1)$ inicializada con 0
4: for $i \leftarrow 1$ to n do
5: for $w \leftarrow 1$ to W do
6: if $weights[i-1] \leq w$ then
7: $dp[i][w] \leftarrow \max(dp[i-1][w], dp[i-1][w - weights[i-1]] + values[i-1])$
8: else
9: $dp[i][w] \leftarrow dp[i-1][w]$
10: end if
11: end for
12: end for
13: return $dp[n][W]$
14: end procedure
```

---

**Complejidad:**  $O(nW)$  — pseudopolinomial (no polinomial en longitud de entrada).

## Complejidad del Problema de la Mochila 0/1

- ▶ **NP-complete:** No se conoce algoritmo polinomial en  $n$ .
- ▶ **Programación dinámica:**  $O(nW)$  tiempo y  $O(nW)$  espacio (pseudopolinomial).
- ▶ **Pseudopolinomial:** Polinomial en  $n$  y en el valor de  $W$ , pero no en el tamaño de la entrada (que es  $O(\log W)$ ).
- ▶ **Implicación:** Si  $W$  es grande (ej.  $2^n$ ), el algoritmo es exponencial.

## Resumen de Complejidades

| Algoritmo             | Temporal           | Espacial |
|-----------------------|--------------------|----------|
| Fuerza bruta          | $O(2^n)$           | $O(1)$   |
| Programación dinámica | $O(nW)$            | $O(nW)$  |
| Branch and bound      | $O(2^n)$ peor caso | $O(n)$   |

## Optimización de Espacio para Mochila DP

La matriz  $dp$  puede reducirse a un arreglo unidimensional si se itera  $w$  en orden descendente:

---

### Algoritmo 31 Mochila DP - Optimización Espacial

---

```
1: $dp \leftarrow$ arreglo $(W + 1)$ inicializado con 0
2: for $i \leftarrow 0$ to $n - 1$ do
3: for $w \leftarrow W$ downto $weights[i]$ do
4: $dp[w] \leftarrow \max(dp[w], dp[w - weights[i]] + values[i])$
5: end for
6: end for
```

---

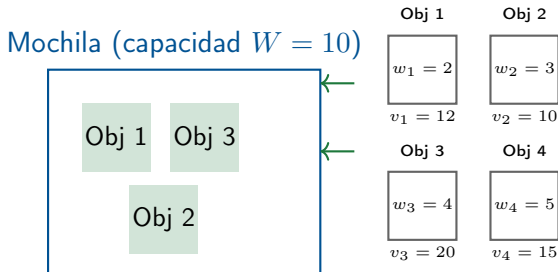
Complejidad espacial:  $O(W)$ .

## Variantes

- ▶ **Mochila fraccionaria:** Resoluble con algoritmo voraz ( $O(n \log n)$ ).
- ▶ **Mochila múltiple:** Múltiples mochilas (NP-complete).
- ▶ **Mochila con restricciones:** Dependencias entre objetos.



## Ejemplo: Problema de la Mochila - Ilustración



Valor total:  $12 + 20 + 10 = 42$

### Interpretación

Se seleccionan los objetos 1 (peso 2, valor 12), 2 (peso 3, valor 10) y 3 (peso 4, valor 20) con peso total  $2 + 3 + 4 = 9 \leq 10$ , maximizando el valor total 42. El objeto 4 (peso 5, valor 15) no cabe sin exceder la capacidad.

# Optimización Multiobjetivo

## Planteamiento del Problema

En numerosos problemas de decisión en contextos reales, las alternativas deben evaluarse simultáneamente bajo **múltiples criterios** que frecuentemente resultan **conflictivos**. Por ejemplo:

- ▶ Maximizar beneficios mientras se minimizan riesgos.
- ▶ Maximizar cobertura poblacional y minimizar costos logísticos.
- ▶ Minimizar peso y maximizar resistencia en diseño estructural.
- ▶ Maximizar rendimiento y minimizar consumo energético.

# Optimización Multiobjetivo - Fundamentos II

## Formalización Matemática

La **optimización multiobjetivo** (OMO) constituye la rama de la investigación operativa que aborda problemas de la forma:

$$\begin{array}{ll} \text{optimizar} & \mathbf{F}(\mathbf{x}) = (f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_k(\mathbf{x})) \\ \text{sujeto a} & \mathbf{x} \in \mathcal{X} \subseteq \mathbb{R}^n \end{array}$$

donde  $\mathcal{X}$  representa el conjunto de soluciones factibles, y cada función  $f_i : \mathcal{X} \rightarrow \mathbb{R}$  constituye un objetivo que puede requerir maximización o minimización.

# Optimización Multiobjetivo - Fundamentos III

## Diferencia con Optimización Mono-objetivo

- ▶ En optimización mono-objetivo, existe un orden total en el espacio de objetivos (porque es  $\mathbb{R}$ ).
- ▶ En optimización multiobjetivo, no existe generalmente un orden total, sino un orden parcial basado en dominancia.
- ▶ La mejora de un objetivo típicamente implica el deterioro de otro.
- ▶ Se busca identificar el conjunto de soluciones **eficientes** o **óptimas de Pareto**, no una única solución.

## Objetivos

1. Encontrar soluciones lo más cercanas posible al frente de Pareto.
2. Mantener diversidad en el conjunto de soluciones (buena distribución).
3. Cubrir todo el rango del frente de Pareto (exploración).

# Dominancia de Pareto - Definiciones Formales I

## Definición de Dominancia

Dadas dos soluciones factibles  $\mathbf{x}, \mathbf{y} \in \mathcal{X}$ , se dice que  $\mathbf{x}$  *domina* a  $\mathbf{y}$  (denotado  $\mathbf{x} \prec \mathbf{y}$ ) si se satisfacen simultáneamente las siguientes condiciones:

- ▶ Para todos los objetivos,  $\mathbf{x}$  no es peor que  $\mathbf{y}$ :

$$f_i(\mathbf{x}) \geq f_i(\mathbf{y}) \quad \forall i \text{ (si son maximización)}$$

$$f_j(\mathbf{x}) \leq f_j(\mathbf{y}) \quad \forall j \text{ (si son minimización)}$$

- ▶ Existe al menos un objetivo donde  $\mathbf{x}$  es estrictamente mejor que  $\mathbf{y}$ :

$$f_k(\mathbf{x}) > f_k(\mathbf{y}) \text{ para algún } k \text{ (maximización)}$$

$$f_k(\mathbf{x}) < f_k(\mathbf{y}) \text{ para algún } k \text{ (minimización)}$$

# Dominancia de Pareto - Definiciones Formales II

## Ejemplo (2 objetivos, ambos maximización)

- ▶  $\mathbf{a} = (5, 3)$ ,  $\mathbf{b} = (4, 2)$ :  $\mathbf{a} \prec \mathbf{b}$  (mejor en ambos).
- ▶  $\mathbf{a} = (5, 3)$ ,  $\mathbf{c} = (4, 4)$ : No hay dominancia (mejor en un objetivo, peor en otro).

# Dominancia de Pareto - Definiciones Formales III

## Solución Óptima de Pareto

Una solución  $\mathbf{x}^* \in \mathcal{X}$  se denomina **óptima de Pareto** (o eficiente) si no existe ninguna otra solución  $\mathbf{y} \in \mathcal{X}$  tal que  $\mathbf{y} \prec \mathbf{x}^*$ .

Es decir,  $\mathbf{x}^*$  es no dominada por ninguna otra solución factible.

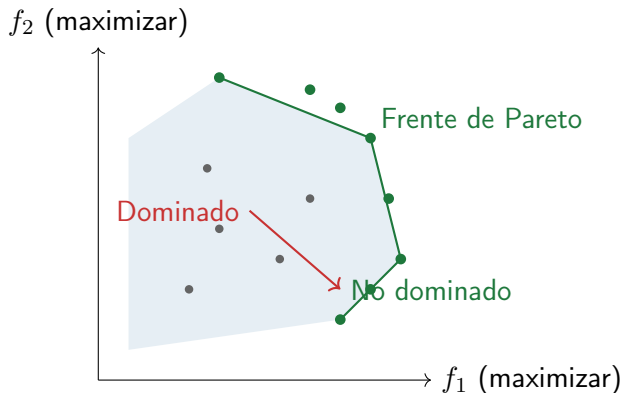
## Frente de Pareto

- ▶ **Conjunto eficiente:**  $\mathcal{X}^* = \{\mathbf{x} \in \mathcal{X} \mid \mathbf{x} \text{ es óptimo de Pareto}\}$
- ▶ **Frente de Pareto:**  $\mathcal{P} = \mathbf{F}(\mathcal{X}^*) = \{(f_1(\mathbf{x}), \dots, f_k(\mathbf{x})) \mid \mathbf{x} \in \mathcal{X}^*\}$

El frente de Pareto representa el trade-off óptimo entre los objetivos.



# Optimización Multiobjetivo - Ilustración del Frente de Pareto I



# Optimización Multiobjetivo - Ilustración del Frente de Pareto II

## Interpretación

Los puntos en **verde** constituyen el frente de Pareto (no dominados). Los puntos en **gris** son dominados por al menos un punto del frente. El objetivo es encontrar un conjunto de soluciones bien distribuidas a lo largo del frente.

# NSGA-II - Algoritmo Genético Multiobjetivo I

## Descripción General

NSGA-II (Non-dominated Sorting Genetic Algorithm II) es uno de los algoritmos genéticos multiobjetivo más populares y efectivos. Fue propuesto por Deb et al. en 2002 y se caracteriza por:

- ▶ Clasificación por frentes de no dominancia.
- ▶ Distancia de aglomeración (crowding distance) para preservar diversidad.
- ▶ Elitismo mediante comparación por torneo basado en rango y distancia.

## Componentes Clave

1. **Clasificación no dominada:** Ordena la población en frentes ( $F_1, F_2, \dots$ ) donde  $F_1$  contiene los no dominados,  $F_2$  los dominados solo por  $F_1$ , etc.
2. **Distancia de aglomeración:** Mide la densidad de soluciones alrededor de cada individuo en el espacio de objetivos.
3. **Selección por torneo:** Compara dos individuos; gana el de mejor rango, o si igual rango, el de mayor distancia.

# NSGA-II - Algoritmo Genético Multiobjetivo III

## Esquema del Algoritmo

1. Inicializar población  $P_0$  de tamaño  $N$ .
2. Evaluar objetivos para todos los individuos.
3. Para cada generación  $t$ :
  - 3.1 Crear población hijo  $Q_t$  mediante selección, cruce y mutación.
  - 3.2 Combinar  $R_t = P_t \cup Q_t$  (tamaño  $2N$ ).
  - 3.3 Clasificar  $R_t$  en frentes  $F_1, F_2, \dots$ .
  - 3.4 Construir  $P_{t+1}$ :
    - ▶ Añadir frentes completos mientras  $|P_{t+1}| + |F_i| \leq N$ .
    - ▶ Para el último frente, ordenar por distancia de aglomeración y añadir los más dispersos.
  - 3.5 Seleccionar padres de  $P_{t+1}$  para siguiente generación.

# NSGA-II - Pseudocódigo

---

## Algoritmo 32 NSGA-II (Non-dominated Sorting Genetic Algorithm II)

---

**Require:**  $N$ : tamaño población,  $G$ : número generaciones,  $k$ : número objetivos

**Ensure:** Conjunto de soluciones aproximadas al frente de Pareto

```
1: procedure NSGAII(N, G, k)
2: $P \leftarrow$ INICIALIZAR_POBLACIÓN(N)
3: EVALUAR_OBJETIVOS(P)
4: for $gen \leftarrow 1$ to G do
5: $Q \leftarrow$ CREAR_HIJOS(P) //Selección, cruce, mutación
6: EVALUAR_OBJETIVOS(Q)
7: $R \leftarrow P \cup Q$ //Población combinada tamaño $2N$
8: $frentes \leftarrow$ FAST_NON_DOMINATED_SORT(R)
9: $P_{nueva} \leftarrow \emptyset$
10: $i \leftarrow 1$
11: while $|P_{nueva}| + |frentes[i]| \leq N$ do
12: CALCULAR_CROWDING_DISTANCE($frentes[i]$)
13: $P_{nueva} \leftarrow P_{nueva} \cup frentes[i]$
14: $i \leftarrow i + 1$
15: end while
16: CALCULAR_CROWDING_DISTANCE($frentes[i]$)
17: ORDENAR_POR_CROWDING($frentes[i]$)
18: $P_{nueva} \leftarrow P_{nueva} \cup \{frentes[i][1..(N - |P_{nueva}|)]\}$
19: $P \leftarrow P_{nueva}$
20: end for
21: return P
22: end procedure
```

---

## Complejidad de la Clasificación No Dominada

El algoritmo `fast_non_dominated_sort` tiene complejidad  $O(M(2N)^2) = O(MN^2)$ , donde:

- ▶  $M$ : número de objetivos.
- ▶  $N$ : tamaño de la población.
- ▶ La población combinada  $R$  tiene tamaño  $2N$ .

## Complejidad de la Distancia de Aglomeración

Para cada frente, ordenar por cada objetivo y calcular distancias:

$$O(M \cdot |F| \log |F|)$$

En total, para todos los frentes:  $O(MN \log N)$ .

## Resumen de Complejidades por Generación

| Operación                    | Complejidad            |
|------------------------------|------------------------|
| Clasificación no dominada    | $O(MN^2)$              |
| Cálculo de crowding distance | $O(MN \log N)$         |
| Selección, cruce, mutación   | $O(NL)$                |
| Evaluación de objetivos      | $O(N \cdot C_f)$       |
| <b>Total</b>                 | $O(MN^2 + N(L + C_f))$ |



## Complejidad Espacial

- ▶ Población actual e hijos:  $O(NL)$ .
- ▶ Frentes de dominancia:  $O(N)$ .
- ▶ Distancias:  $O(N)$ .

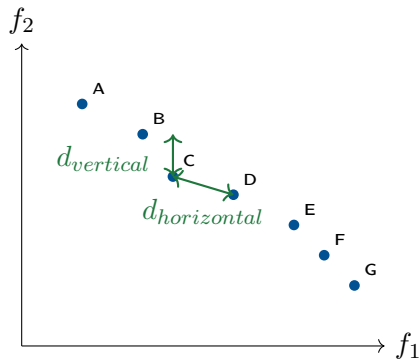
$$M(N, L) = O(NL)$$

donde  $L$  es la longitud del cromosoma (dimensión del problema).

## Observaciones

- ▶ NSGA-II es uno de los algoritmos multiobjetivo más eficientes ( $O(MN^2)$  es aceptable para  $N$  moderado).
- ▶ La complejidad dominante es la clasificación no dominada.
- ▶ Para problemas con muchos objetivos ( $M > 5$ ), pueden requerirse técnicas alternativas (NSGA-III, MOEA/D).

# NSGA-II - Ilustración de Crowding Distance I



$$d_C = d_{vertical} + d_{horizontal}$$

(suma de distancias a vecinos en cada objetivo)

# NSGA-II - Ilustración de Crowding Distance II

## Interpretación

La distancia de aglomeración (crowding distance) mide la densidad de soluciones alrededor de un punto. Puntos con mayor distancia están en regiones menos pobladas y se prefieren para mantener diversidad en el frente.

# Aplicaciones de Optimización Multiobjetivo I

## Ingeniería y Diseño

- ▶ **Diseño aerodinámico:** Minimizar resistencia y maximizar sustentación.
- ▶ **Diseño estructural:** Minimizar peso y maximizar rigidez/resistencia.
- ▶ **Diseño de circuitos:** Minimizar consumo y maximizar velocidad.
- ▶ **Diseño de redes:** Minimizar costo y maximizar cobertura/confiabilidad.

## Finanzas y Economía

- ▶ **Optimización de portafolios:** Maximizar retorno esperado y minimizar riesgo (Markowitz).
- ▶ **Planificación financiera:** Múltiples objetivos de inversión a largo plazo.
- ▶ **Asignación de recursos:** Balance entre beneficios, costos y riesgos.

# Aplicaciones de Optimización Multiobjetivo II

## Logística y Operaciones

- ▶ **Ruteo de vehículos:** Minimizar distancia y maximizar satisfacción de clientes.
- ▶ **Localización de instalaciones:** Minimizar costos y maximizar cobertura.
- ▶ **Planificación de producción:** Minimizar costos y maximizar calidad/utilización.

# Aplicaciones de Optimización Multiobjetivo III

## Ciencia de Datos e IA

- ▶ **Selección de características:** Maximizar precisión y minimizar número de características.
- ▶ **Ajuste de hiperparámetros:** Balance entre rendimiento y complejidad del modelo.
- ▶ **Clustering:** Minimizar distancia intra-cluster y maximizar distancia inter-cluster.

## Energía y Medio Ambiente

- ▶ **Gestión de recursos hídricos:** Balance entre consumo agrícola, urbano y ecológico.
- ▶ **Diseño de sistemas energéticos:** Minimizar costo y emisiones, maximizar eficiencia.

# Problemas de Complejidad NP



# Backtracking y Branch & Bound

## Clase NP-complete

La clase de complejidad **NP-complete** se caracteriza por la ausencia de algoritmos de tiempo polinomial deterministas que garanticen soluciones óptimas para todas las instancias. Esta categoría incluye problemas cuya verificabilidad es eficiente (en tiempo polinomial), mas no necesariamente su resolución.

Formalmente, un problema es NP-complete si:

1. Está en NP (una solución candidata puede verificarse en tiempo polinomial).
2. Es NP-hard (todo problema en NP puede reducirse a él en tiempo polinomial).

## Implicaciones Prácticas

- ▶ No se conocen algoritmos eficientes (polinomiales) para resolverlos de manera exacta.
- ▶ Se cree que  $P \neq NP$ , lo que implicaría que tales algoritmos no existen.
- ▶ En la práctica, se emplean estrategias alternativas:
  - ▶ Algoritmos exactos con poda (backtracking, branch and bound) para instancias pequeñas o medianas.
  - ▶ Algoritmos de aproximación con garantías de calidad.
  - ▶ Metaheurísticas (AG, PSO, recocido simulado) para instancias grandes.
  - ▶ Algoritmos probabilísticos (Monte Carlo, Las Vegas).

# Estrategias Algorítmicas para NP-completeness I

- ▶ **Backtracking:** Exploración sistemática del espacio de soluciones con mecanismos de retroceso ante soluciones parciales inviables. Construye soluciones incrementalmente y retrocede cuando una rama no puede conducir a una solución válida.
- ▶ **Branch and Bound (Ramificación y Poda):** Estrategia de exploración basada en árbol de decisiones con eliminación de subárboles no prometedores mediante funciones de acotamiento (cota inferior para minimización, cota superior para maximización).
- ▶ **Algoritmos Aproximados:** Construcción de soluciones subóptimas con garantías de rendimiento demostrable en tiempo polinomial (ej. algoritmo de Christofides para TSP métrico con ratio  $3/2$ ).
- ▶ **Heurísticas:** Estrategias prácticas sin garantías teóricas completas pero efectivas empíricamente en escenarios reales (ej. vecino más cercano para TSP).

# Estrategias Algorítmicas para NP-completeness II

## Desafío Computacional

El espacio de búsqueda asociado a estos problemas exhibe crecimiento exponencial con respecto al tamaño de la instancia:

$$|\mathcal{S}| = O(2^n) \quad \text{o} \quad O(n!)$$

Esta explosión combinatoria constituye el principal desafío computacional y motiva el empleo de estrategias metódicas para su exploración sistemática, así como técnicas de poda que reduzcan drásticamente el espacio efectivo de búsqueda.

# Ejemplo 1: Problema de las N-Reinas - Planteamiento I

## Problema

Colocar  $n$  reinas en un tablero de ajedrez de  $n \times n$  de manera que ninguna reina ataque a otra. En ajedrez, una reina ataca todas las casillas en su misma fila, columna y diagonales.

# Ejemplo 1: Problema de las N-Reinas - Planteamiento II

## Definición Formal

**Entrada:** Un entero  $n \geq 4$  (tamaño del tablero).

**Salida:** Una configuración de  $n$  reinas en el tablero  $n \times n$  tal que no haya dos reinas compartiendo fila, columna o diagonal, o una indicación de que no existe solución.

### Características:

- ▶ Cada fila debe contener exactamente una reina.
- ▶ Cada columna debe contener exactamente una reina.
- ▶ Las reinas no pueden compartir diagonales (pendiente  $\pm 1$ ).

# Ejemplo 1: N-Reinas - Algoritmo de Backtracking

---

## Algoritmo 33 N-Reinas con Backtracking

---

**Require:**  $n$ : tamaño del tablero

**Ensure:** Matriz  $n \times n$  con las reinas colocadas (1 indica reina)

```
1: procedure N_QUEENS(n)
2: $board \leftarrow$ matriz $n \times n$ inicializada con 0
3: return SOLVE_NQ_UTIL($board, 0, n$)
4: end procedure
5: procedure SOLVE_NQ_UTIL($board, col, n$)
6: if $col \geq n$ then //Todas las reinas colocadas
7: return true
8: end if
9: for $row = 0$ to $n - 1$ do
10: if IS_SAFE($board, row, col, n$) then
11: $board[row][col] \leftarrow 1$ //Colocar reina
12: if SOLVE_NQ_UTIL($board, col + 1, n$) then
13: return true
14: end if
15: $board[row][col] \leftarrow 0$ //Retroceder (backtrack)
16: end if
17: end for
18: return false
19: end procedure
20: procedure IS_SAFE($board, row, col, n$)
21: Verificar que no haya reina en misma fila a la izquierda
22: Verificar diagonal superior izquierda
23: Verificar diagonal inferior izquierda
24: return true si todas las verificaciones pasan
25: end procedure
```

---



# Ejemplo 1: Problema de las N-Reinas - Planteamiento I

## Problema

Colocar  $n$  reinas en un tablero de ajedrez de  $n \times n$  de manera que ninguna reina ataque a otra. En ajedrez, una reina ataca todas las casillas en su misma fila, columna y diagonales.

# Ejemplo 1: Problema de las N-Reinas - Planteamiento II

## Definición Formal

**Entrada:** Un entero  $n \geq 4$  (tamaño del tablero).

**Salida:** Una configuración de  $n$  reinas en el tablero  $n \times n$  tal que no haya dos reinas compartiendo fila, columna o diagonal, o una indicación de que no existe solución.

**Características:**

- ▶ Cada fila debe contener exactamente una reina.
- ▶ Cada columna debe contener exactamente una reina.
- ▶ Las reinas no pueden compartir diagonales (pendiente  $\pm 1$ ).

## Observación Inicial

Para  $n = 4$ , existen 2 soluciones posibles. El backtracking explorará sistemáticamente el espacio de búsqueda hasta encontrar una.

# Ejemplo 1: N-Reinas - Algoritmo de Backtracking

---

## Algoritmo 34 N-Reinas con Backtracking

---

**Require:**  $n$ : tamaño del tablero

**Ensure:** Matriz  $n \times n$  con las reinas colocadas (1 indica reina)

```
1: procedure N_QUEENS(n)
2: $board \leftarrow$ matriz $n \times n$ inicializada con 0
3: return SOLVE_NQ_UTIL($board, 0, n$)
4: end procedure
5: procedure SOLVE_NQ_UTIL($board, col, n$)
6: if $col \geq n$ then //Todas las reinas colocadas
7: return true
8: end if
9: for $row = 0$ to $n - 1$ do
10: if IS_SAFE($board, row, col, n$) then
11: $board[row][col] \leftarrow 1$ //Colocar reina
12: if SOLVE_NQ_UTIL($board, col + 1, n$) then
13: return true
14: end if
15: $board[row][col] \leftarrow 0$ //Retroceder (backtrack)
16: end if
17: end for
18: return false
19: end procedure
20: procedure IS_SAFE($board, row, col, n$)
21: Verificar que no haya reina en misma fila a la izquierda
22: Verificar diagonal superior izquierda
23: Verificar diagonal inferior izquierda
24: return true si todas las verificaciones pasan
25: end procedure
```

---

# Ejemplo 1: N-Reinas - Análisis de Complejidad I

## Complejidad Temporal

En el peor caso, sin poda, se explorarían todas las disposiciones posibles. Dado que cada reina debe estar en una columna diferente y en una fila diferente, el espacio de búsqueda son las permutaciones de filas para cada columna.

$$T(n) = O(n!)$$

### Justificación:

- ▶ Para la primera columna:  $n$  opciones de fila.
- ▶ Para la segunda columna:  $n - 1$  opciones (no puede coincidir fila).
- ▶ Para la tercera columna:  $n - 2$  opciones.
- ▶ Total:  $n \times (n - 1) \times \dots \times 1 = n!$

# Ejemplo 1: N-Reinas - Análisis de Complejidad II

## Resumen de Complejidades

| Temporal | Espacial |
|----------|----------|
| $O(n!)$  | $O(n^2)$ |

# Ejemplo 1: N-Reinas - Análisis de Complejidad III

## Complejidad Espacial

$$M(n) = O(n^2)$$

- ▶ Tablero  $n \times n$  para almacenar la configuración actual.
- ▶ Profundidad de recursión:  $O(n)$  en la pila de llamadas.
- ▶ Variables auxiliares por llamada: constantes.

# Ejemplo 1: N-Reinas - Análisis de Complejidad IV

## Mecanismo de Poda

La función `is_safe` verifica que la reina no sea atacada por otras ya colocadas:

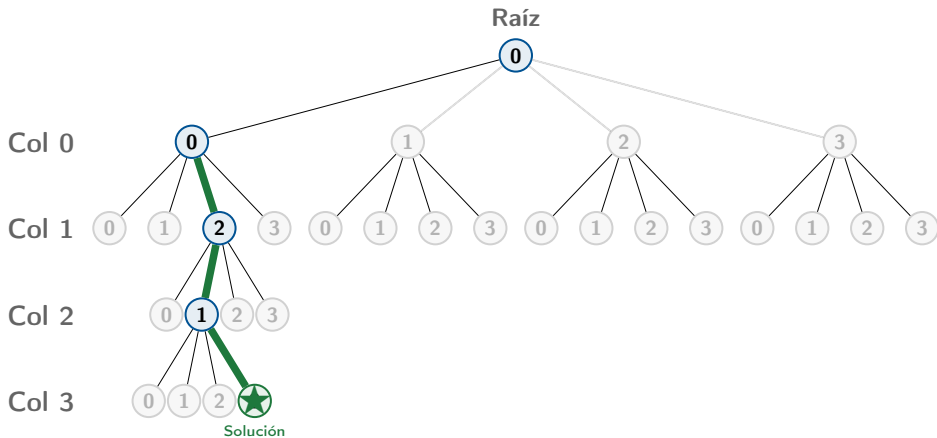
- ▶ Misma fila (a la izquierda, ya que colocamos por columnas).
- ▶ Diagonal principal (superior izquierda).
- ▶ Diagonal secundaria (inferior izquierda).

Esta verificación permite podar ramas inviables tempranamente, reduciendo drásticamente el espacio explorado en la práctica.

## Observación

Aunque la complejidad teórica es  $O(n!)$ , en la práctica el backtracking resuelve  $n = 8$  instantáneamente,  $n = 15$  en segundos, y  $n = 20$  en minutos, gracias a la poda efectiva.

## Ejemplo 1: N-Reinas - Árbol de Backtracking para $n = 4$ (Solución 1)





## Ejemplo 1: N-Reinas - Soluciones para $n = 4$ (Solución 1)

|   |   |   |   |
|---|---|---|---|
| Q |   |   |   |
|   |   | Q |   |
|   | Q |   |   |
|   |   |   | Q |

$(0, 0), (1, 2), (2, 1), (3, 3)$

# Recorrido del Backtracking para la Solución 1 I

1. **Col 0:** Prueba fila 0 (válida)  $\rightarrow$  coloca reina en  $(0, 0)$ .
2. **Col 1:** Prueba fila 0 (inviable - misma fila), fila 1 (inviable - diagonal), fila 2 (válida)  $\rightarrow$  coloca reina en  $(1, 2)$ .
3. **Col 2:** Prueba fila 0 (inviable - diagonal), fila 1 (válida)  $\rightarrow$  coloca reina en  $(2, 1)$ .
4. **Col 3:** Prueba fila 0 (inviable - diagonal), fila 1 (inviable - fila), fila 2 (inviable - diagonal), fila 3 (válida)  $\rightarrow$  coloca reina en  $(3, 3)$ .
5. **Solución encontrada:**  $(0, 0), (1, 2), (2, 1), (3, 3)$ .

Nota: Las coordenadas están en formato  $(fila, columna)$ .

## Ejemplo 1: N-Reinas - Soluciones para $n = 4$ (Solución 2)

|   |   |   |   |
|---|---|---|---|
|   | Q |   |   |
|   |   |   | Q |
| Q |   |   |   |
|   |   | Q |   |

$(0, 1), (1, 3), (2, 0), (3, 2)$

## Recorrido del Backtracking para la Solución 2 I

1. **Col 0:** Prueba fila 1 (válida)  $\rightarrow$  coloca reina en  $(0, 1)$ .
2. **Col 1:** Prueba fila 3 (válida)  $\rightarrow$  coloca reina en  $(1, 3)$ .
3. **Col 2:** Prueba fila 0 (válida)  $\rightarrow$  coloca reina en  $(2, 0)$ .
4. **Col 3:** Prueba fila 2 (válida)  $\rightarrow$  coloca reina en  $(3, 2)$ .
5. **Solución encontrada:**  $(0, 1), (1, 3), (2, 0), (3, 2)$ .

## Ejemplo 2: Mochila 0/1 - Planteamiento con Branch and Bound I

### Problema

Dados  $n$  objetos con peso  $w_i$  y valor  $v_i$ , y una mochila con capacidad  $W$ , seleccionar un subconjunto de objetos que maximice el valor total sin exceder la capacidad.

### Limitación del Backtracking Simple

El backtracking básico explora las  $2^n$  combinaciones, lo cual es inviable para  $n$  grande. Se necesita una función de cota (bound) para podar ramas que no pueden superar la mejor solución actual.

## Ejemplo 2: Mochila 0/1 - Planteamiento con Branch and Bound II

### Estrategia Branch and Bound

- ▶ Se explora un árbol binario donde cada nivel representa un objeto.
- ▶ Rama izquierda: incluir el objeto.
- ▶ Rama derecha: excluir el objeto.
- ▶ En cada nodo, se calcula una cota superior del valor alcanzable desde ese nodo.
- ▶ Si la cota es menor que la mejor solución encontrada, se poda la rama.

# Ejemplo 2: Mochila 0/1 - Algoritmo Branch and Bound

---

**Algoritmo 35** Mochila 0/1 con Ramificación y Poda (Branch and Bound)

---

**Require:**  $weights[0..n-1]$ ,  $values[0..n-1]$ , capacidad  $capacity$

**Ensure:** Valor máximo alcanzable

```
1: procedure KNAPSACK_BNB($weights, values, capacity$)
2: $n \leftarrow |weights|$
3: $best_value \leftarrow 0$
4: Ordenar objetos por ratio v_i/w_i descendente (para mejor cota)
5: KNAPSACK_UTIL($weights, values, capacity, 0, 0, best_value$)
6: return $best_value$
7: end procedure
8: procedure KNAPSACK_UTIL($weights, values, cap, idx, w, v, best$)
9: if $idx = |weights|$ then //Nodo hoja
10: $best \leftarrow \max(best, v)$
11: return
12: end if
13: if $w + weights[idx] \leq cap$ then //Incluir objeto
14: KNAPSACK_UTIL($weights, values, cap, idx + 1, w + weights[idx], v + values[idx], best$)
15: end if
16: $bound \leftarrow \text{CALCULATE_BOUND}(weights, values, cap, idx, w, v)$
17: if $bound > best$ then //Solo si la cota supera la mejor actual
18: KNAPSACK_UTIL($weights, values, cap, idx + 1, w, v, best$)
19: //Excluir objeto
20: end if
21: end procedure
22: procedure CALCULATE_BOUND($weights, values, cap, idx, w, v$)
23: $bound \leftarrow v$
24: $remaining_cap \leftarrow cap - w$
25: $j \leftarrow idx$
26: while $j < |weights|$ and $weights[j] \leq remaining_cap$ do
27: $bound \leftarrow bound + values[j]$
28: $remaining_cap \leftarrow remaining_cap - weights[j]$
29: $j \leftarrow j + 1$
30: end while
31: if $j < |weights|$ then
32: $bound \leftarrow bound + remaining_cap \times (values[j]/weights[j])$
33: //Fracción del siguiente
34: end if
35: return $bound$
36: end procedure
```

---

## Ejemplo 2: Mochila 0/1 - Análisis de Complejidad I

### Complejidad Temporal

En el peor caso, sin poda efectiva, se exploran los  $2^n$  nodos del árbol binario.

$$T(n) = O(2^n) \quad (\text{peor caso})$$

Con una buena función de cota, muchas ramas se podan, reduciendo drásticamente el número de nodos explorados en la práctica. El rendimiento depende de:

- ▶ Calidad de la función de cota.
- ▶ Orden de procesamiento de los objetos.
- ▶ Distribución de pesos y valores.



## Ejemplo 2: Mochila 0/1 - Análisis de Complejidad II

### Resumen de Complejidades

| Temporal             | Espacial |
|----------------------|----------|
| $O(2^n)$ (peor caso) | $O(n)$   |

## Ejemplo 2: Mochila 0/1 - Análisis de Complejidad III

### Complejidad Espacial

$$M(n) = O(n)$$

- ▶ Profundidad máxima de recursión:  $n$  (un nivel por objeto).
- ▶ Variables locales por llamada: constantes.
- ▶ No se almacena la matriz completa como en programación dinámica.

## Ejemplo 2: Mochila 0/1 - Análisis de Complejidad IV

### Función de Cota (Bound)

La función `calculate_bound` estima el valor máximo alcanzable desde el nodo actual:

1. Toma los objetos ya incluidos (valor  $v$ ).
2. Añade objetos completos mientras quepan en la capacidad restante.
3. Para el siguiente objeto que no cabe completo, añade una fracción (como en la mochila fraccionaria).

Esta cota es una relajación del problema (permite fracciones) y por tanto es una cota superior válida para el problema 0/1.

## Ejemplo 2: Mochila 0/1 - Análisis de Complejidad V

### Importancia del Ordenamiento

Ordenar los objetos por ratio  $v_i/w_i$  descendente mejora la cota y permite podar más ramas tempranamente, ya que los objetos más prometedores se consideran primero.

## Ejemplo 3: Sudoku - Planteamiento del Problema I

### Problema

Completar una cuadrícula de  $9 \times 9$  dividida en subcuadrículas de  $3 \times 3$  (regiones) con dígitos del 1 al 9, de manera que cada fila, cada columna y cada región contenga todos los dígitos exactamente una vez.

### Definición Formal

**Entrada:** Una cuadrícula parcialmente llenada de  $9 \times 9$ , donde algunas celdas contienen un número del 1 al 9 y otras están vacías (representadas por 0).

**Salida:** La cuadrícula completa que satisface las restricciones, o una indicación de que no existe solución.

## Ejemplo 3: Sudoku - Planteamiento del Problema II

### Restricciones

- ▶ **Filas:** Cada fila debe contener los números 1-9 sin repetición.
- ▶ **Columnas:** Cada columna debe contener los números 1-9 sin repetición.
- ▶ **Regiones:** Cada subcuadrícula  $3 \times 3$  debe contener los números 1-9 sin repetición.

## Ejemplo 3: Sudoku - Algoritmo de Backtracking

---

### Algoritmo 36 Sudoku con Backtracking

---

**Require:**  $grid[9][9]$ : tablero parcialmente lleno (0 indica vacío)

**Ensure:** true si se encuentra solución, false en otro caso

```
1: procedure SOLVE_SUDOKU($grid$)
2: $empty \leftarrow$ ENCONTRAR_POSICIÓN_VACÍA($grid$)
3: if $empty = \text{None}$ then //No hay celdas vacías
4: return true //Sudoku resuelto
5: end if
6: $row, col \leftarrow empty$
7: for $num = 1$ to 9 do
8: if IS_SAFE($grid, row, col, num$) then
9: $grid[row][col] \leftarrow num$
10: if SOLVE_SUDOKU($grid$) then
11: return true
12: end if
13: $grid[row][col] \leftarrow 0$ //Retroceder
14: end if
15: end for
16: return false
17: end procedure
18: procedure IS_SAFE($grid, row, col, num$)
19: Verificar que num no esté en la fila row
20: Verificar que num no esté en la columna col
21: Verificar que num no esté en la región 3×3 correspondiente
22: return true si todas las verificaciones pasan
23: end procedure
```

---

## Ejemplo 3: Sudoku - Análisis de Complejidad I

### Complejidad Temporal

En el peor caso, sin poda, cada celda vacía podría probar 9 valores.

$$T(n) = O(9^{n^2})$$

Para un Sudoku estándar de  $9 \times 9$  con  $n = 9$ , esto sería  $9^{81}$ , astronómicamente grande. Sin embargo:

- ▶ La poda por restricciones reduce drásticamente el espacio.
- ▶ En la práctica, los Sudokus bien formados tienen una única solución y se resuelven rápidamente.
- ▶ Para Sudokus vacíos ( $9 \times 9$  completamente vacío), el número de soluciones es enorme y la exploración sería impracticable sin heurísticas adicionales.



## Ejemplo 3: Sudoku - Análisis de Complejidad II

### Resumen de Complejidades

| Temporal                 | Espacial |
|--------------------------|----------|
| $O(9^{n^2})$ (peor caso) | $O(n^2)$ |

## Ejemplo 3: Sudoku - Análisis de Complejidad III

### Complejidad Espacial

$$M(n) = O(n^2)$$

- ▶ Tablero  $n \times n$  para almacenar la configuración actual.
- ▶ Profundidad de recursión proporcional al número de celdas vacías (hasta  $n^2$  en el peor caso).

## Ejemplo 3: Sudoku - Análisis de Complejidad IV

### Optimizaciones Prácticas

- ▶ **Heurística de mínimos valores restantes (MRV):** Seleccionar la celda con menos opciones válidas primero, reduciendo el factor de ramificación.
- ▶ **Forward checking:** Después de asignar un valor, eliminar ese valor de las opciones de las celdas afectadas en la misma fila, columna y región.
- ▶ **Máscaras de bits:** Representar opciones disponibles con bits para verificación  $O(1)$ .
- ▶ **Propagación de restricciones:** Aplicar reglas lógicas (singleton, única posibilidad) antes de la bifurcación.

## Ejemplo 4: Subset Sum - Planteamiento del Problema I

### Problema

Dado un conjunto de números enteros  $arr$  y un valor objetivo  $target$ , determinar si existe un subconjunto cuya suma sea exactamente igual a  $target$ .

### Definición Formal

**Entrada:** Un conjunto  $arr = \{a_1, a_2, \dots, a_n\}$  de enteros no negativos y un entero  $target \geq 0$ .

**Salida:** true si existe un subconjunto  $S \subseteq arr$  tal que  $\sum_{x \in S} x = target$ , false en caso contrario.

## Ejemplo 4: Subset Sum - Planteamiento del Problema II

### Aplicaciones

- ▶ Criptografía (problema de la mochila de Merkle-Hellman).
- ▶ Problemas de partición y balanceo de carga.
- ▶ Algoritmos de aproximación para problemas NP-completeness.

# Ejemplo 4: Subset Sum - Algoritmo Branch and Bound

---

## Algoritmo 37 Subset Sum con Ramificación y Poda

---

**Require:**  $arr[0..n-1]$ : conjunto de números,  $target$ : suma objetivo

**Ensure:** true si existe subconjunto con suma  $target$

```
1: procedure SUBSET_SUM_BNB($arr, target$)
2: Ordenar arr en orden descendente (para mejor poda)
3: return SUBSET_SUM_UTIL($arr, target, 0, 0, n$)
4: end procedure
5: procedure SUBSET_SUM_UTIL($arr, target, sum, idx, n$)
6: if $sum = target$ then
7: return true
8: end if
9: if $idx \geq n$ or $sum > target$ then
10: return false
11: end if
12: $remaining_sum \leftarrow SUM_REMAINING(arr, idx, n)$
13: if $sum + remaining_sum < target$ then //Cota: no se puede
14: alcanzar
15: return false
16: end if
17: $include \leftarrow SUBSET_SUM_UTIL(arr, target, sum + arr[idx], idx +$
18: 1, n)
19: if $include$ then
20: return true
21: end if
22: return SUBSET_SUM_UTIL($arr, target, sum, idx + 1, n$)
23: end procedure
24: procedure SUM_REMAINING($arr, start, n$)
25: $s \leftarrow 0$
26: for $i = start$ to $n - 1$ do
27: $s \leftarrow s + arr[i]$
28: end for
29: return s
30: end procedure
```

---

## Ejemplo 4: Subset Sum - Análisis de Complejidad I

### Complejidad Temporal

En el peor caso, sin poda, se exploran las  $2^n$  combinaciones posibles.

$$T(n) = O(2^n) \quad (\text{peor caso})$$

### Justificación:

- ▶ Cada elemento puede incluirse o excluirse del subconjunto.
- ▶ Árbol de decisiones binario con profundidad  $n$ .
- ▶ Número de nodos en el árbol completo:  $2^{n+1} - 1$ .

## Ejemplo 4: Subset Sum - Análisis de Complejidad II

### Resumen de Complejidades

| Temporal             | Espacial |
|----------------------|----------|
| $O(2^n)$ (peor caso) | $O(n)$   |



## Ejemplo 4: Subset Sum - Análisis de Complejidad III

### Complejidad Espacial

$$M(n) = O(n)$$

- ▶ Profundidad máxima de recursión:  $n$ .
- ▶ Variables locales por llamada: constantes.

## Ejemplo 4: Subset Sum - Análisis de Complejidad IV

### Mecanismos de Poda

- ▶ **Poda por exceso:** Si  $sum > target$ , la rama no puede llevar a solución (todos los números son no negativos).
- ▶ **Poda por cota superior:** Si  $sum + suma\_restante < target$ , incluso incluyendo todos los elementos restantes no se alcanza el objetivo.
- ▶ **Ordenamiento descendente:** Colocar los números grandes primero permite podar más rápidamente por exceso.

### Relación con Otros Problemas

- ▶ Caso especial del problema de la mochila (cuando pesos = valores).
- ▶ Relacionado con el problema de partición (si  $target = \frac{suma\_total}{2}$ ).
- ▶ Fundamental en criptografía de clave pública (criptosistema de la mochila).

# Problemas de Análisis y Métodos Numéricos

# Grupo A: Ecuaciones y Optimización

# Grupo A - Ecuaciones y Optimización I

## Objetivo General

Encontrar valores que satisfacen condiciones algebraicas y de optimización en espacios de dimensión arbitraria  $\mathbb{R}^n$ .

## Problemas Fundamentales

- ▶ Ceros en  $\mathbb{R}^1$ :  $f(x) = 0$  (ej:  $x^3 - 2x - 5 = 0$ )
- ▶ Ceros en  $\mathbb{R}^n$ :  $\mathbf{F}(\mathbf{x}) = \mathbf{0}$  (sistemas no lineales)
- ▶ Optimización en  $\mathbb{R}^n$ :  $\min f(\mathbf{x})$  con  $\mathbf{x} \in \mathbb{R}^n$
- ▶ Álgebra Lineal:  $\mathbf{Ax} = \mathbf{b}$ , autovalores  $\mathbf{Av} = \lambda \mathbf{v}$

## Métodos Principales

- ▶ Newton-Raphson (1D y multidimensional)
- ▶ Broyden (cuasi-Newton para sistemas)
- ▶ Gradiente descendiente (optimización)
- ▶ Eliminación gaussiana, QR, gradiente conjugado

# Newton-Raphson en $\mathbb{R}^1$ - Algoritmo

---

## Algoritmo 38 Newton-Raphson Unidimensional

---

**Require:**  $f, f', x_0, \epsilon, max\_iter$

**Ensure:** Aproximación de  $x$  tal que  $f(x) = 0$

```
1: procedure NEWTON_1D(f, f', x_0, ϵ)
2: $x \leftarrow x_0$
3: for $k \leftarrow 1$ to max_iter do
4: $x_{new} \leftarrow x - f(x)/f'(x)$
5: if $|x_{new} - x| < \epsilon$ then
6: return x_{new}
7: end if
8: $x \leftarrow x_{new}$
9: end for
10: return x
11: end procedure
```

---

# Newton-Raphson - Análisis de Complejidad I

## Complejidad Temporal

- ▶ **Convergencia:** Cuadrática  $O(\log(\log(1/\epsilon)))$  cerca de la raíz
- ▶ **Costo por iteración:**  $O(1)$  evaluaciones de  $f$  y  $f'$
- ▶ **Total:**  $O(k)$  donde  $k$  es número de iteraciones (típicamente 5-10)

## Resumen de Complejidades

| Temporal                   | Espacial |
|----------------------------|----------|
| $O(k)$ con $k$ iteraciones | $O(1)$   |



# Newton-Raphson - Análisis de Complejidad II

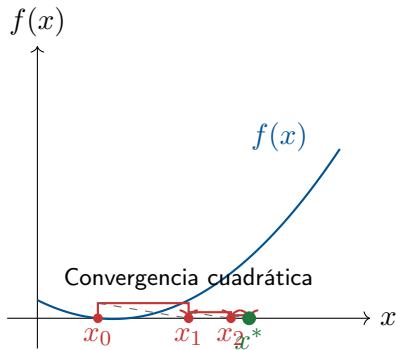
## Complejidad Espacial

- ▶ **Espacial:**  $O(1)$  (solo variables temporales)
- ▶ No requiere almacenamiento de estructuras auxiliares
- ▶ Solo mantiene el valor actual de  $x$  y el nuevo valor

## Limitaciones

- ▶ Requiere derivada analítica o numérica (puede ser costosa)
- ▶ Puede divergir si  $x_0$  está lejos de la raíz
- ▶ Falla cuando  $f'(x) \approx 0$  (puntos críticos o raíces múltiples)
- ▶ Convergencia solo local (no global)

# Newton-Raphson en $\mathbb{R}^1$ - Ilustración I



# Newton-Raphson en $\mathbb{R}^1$ - Ilustración II

## Interpretación

Cada iteración usa la tangente a la curva en el punto actual para aproximar la raíz. La nueva aproximación es la intersección de la tangente con el eje  $x$ . La convergencia es rápida cuando el punto inicial está cerca de la solución.

# Newton-Raphson en $\mathbb{R}^n$ - Algoritmo

---

## Algoritmo 39 Newton-Raphson Multidimensional

---

**Require:**  $\mathbf{F} : \mathbb{R}^n \rightarrow \mathbb{R}^n$ ,  $\mathbf{J}$  (Jacobiano),  $\mathbf{x}_0 \in \mathbb{R}^n$ ,  $\epsilon$

**Ensure:**  $\mathbf{x} \in \mathbb{R}^n$  tal que  $\mathbf{F}(\mathbf{x}) = \mathbf{0}$

```
1: procedure NEWTON_ND($\mathbf{F}, \mathbf{J}, \mathbf{x}_0, \epsilon$)
2: $\mathbf{x} \leftarrow \mathbf{x}_0$
3: for $k \leftarrow 1$ to max_iter do
4: Resolver $\mathbf{J}(\mathbf{x})\Delta\mathbf{x} = -\mathbf{F}(\mathbf{x})$
5: $\mathbf{x} \leftarrow \mathbf{x} + \Delta\mathbf{x}$
6: if $\|\Delta\mathbf{x}\| < \epsilon$ then
7: return $\mathbf{x}$
8: end if
9: end for
10: return $\mathbf{x}$
11: end procedure
```

---

Cada iteración requiere resolver un sistema lineal de  $n \times n$ .

# Newton en $\mathbb{R}^n$ - Análisis de Complejidad I

## Complejidad Temporal

- ▶ **Por iteración:**  $O(n^3)$  para resolver sistema lineal por eliminación gaussiana
- ▶ **Evaluación Jacobiano:**  $O(n^2)$  derivadas parciales (analíticas o numéricas)
- ▶ **Total:**  $O(k \cdot n^3)$  donde  $k$  es número de iteraciones

## Resumen de Complejidades

| Temporal         | Espacial |
|------------------|----------|
| $O(k \cdot n^3)$ | $O(n^2)$ |

# Newton en $\mathbb{R}^n$ - Análisis de Complejidad II

## Complejidad Espacial

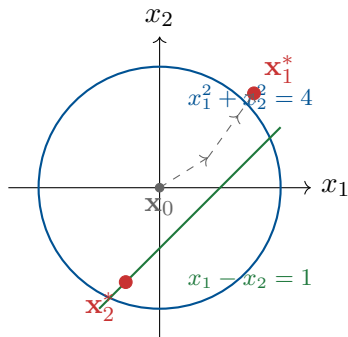
- ▶ **Jacobiano:**  $O(n^2)$  para almacenar matriz  $n \times n$
- ▶ **Vectores:**  $O(n)$  para  $\mathbf{x}$ ,  $\mathbf{F}(\mathbf{x})$ ,  $\Delta\mathbf{x}$
- ▶ **Total:**  $O(n^2)$  dominado por el Jacobiano

## Ejemplo: Sistema 2D

$$\mathbf{F}(\mathbf{x}) = \begin{bmatrix} x_1^2 + x_2^2 - 4 \\ x_1 - x_2 - 1 \end{bmatrix} = \mathbf{0}$$

Geométricamente: intersección de un círculo y una recta en  $\mathbb{R}^2$ .

# Newton en $\mathbb{R}^2$ - Ilustración I



Convergencia a solución del sistema

# Newton en $\mathbb{R}^2$ - Ilustración II

## Interpretación

En  $\mathbb{R}^n$ , Newton encuentra la intersección de múltiples superficies. Cada iteración resuelve un sistema lineal aproximando las funciones por sus planos tangentes. La convergencia sigue siendo cuadrática cerca de la solución.



## Grupo B: Aproximación y Representación

# Grupo B - Aproximación y Representación I

## Objetivo

Representar funciones, datos e integrales de forma aproximada en espacios de dimensión arbitraria  $\mathbb{R}^d$ .

## Problemas Fundamentales

- ▶ **Interpolación en  $\mathbb{R}^d$ :** Lagrange, Newton, splines
- ▶ **Aproximación:** Mínimos cuadrados, Fourier, wavelets
- ▶ **Integración:** Newton-Cotes, Gaussiana, cubatura ( $d > 1$ )
- ▶ **Ecuaciones integrales:** Fredholm, Volterra

## Métodos Principales

- ▶ Diferencias divididas
- ▶ FFT (Transformada Rápida de Fourier)
- ▶ Cuadratura adaptativa
- ▶ Polinomios ortogonales

# Interpolación de Lagrange en $\mathbb{R}^n$ - Algoritmo

---

## Algoritmo 40 Lagrange Multidimensional

---

**Require:**  $\{(\mathbf{x}_i, y_i)\}_{i=0}^m, \mathbf{x}^* \in \mathbb{R}^d$

**Ensure:**  $P(\mathbf{x}^*)$ : valor interpolado

```
1: procedure LAGRANGE_ND(points, $\mathbf{x}^*$)
2: result $\leftarrow 0$
3: for $i \leftarrow 0$ to m do
4: $L_i \leftarrow 1$
5: for $j \leftarrow 0$ to m do
6: if $i \neq j$ then
7: $L_i \leftarrow L_i \cdot \frac{\|\mathbf{x}^* - \mathbf{x}_j\|}{\|\mathbf{x}_i - \mathbf{x}_j\|}$
8: end if
9: end for
10: result $\leftarrow$ result + $y_i \cdot L_i$
11: end for
12: return result
13: end procedure
```

---

# Lagrange en $\mathbb{R}^n$ - Análisis de Complejidad I

## Complejidad Temporal

- ▶ **Temporal:**  $O(m^2)$  para  $m + 1$  puntos
- ▶ **En  $\mathbb{R}^d$ :**  $O(m^2 \cdot d)$  por cálculo de distancias
- ▶ Cada evaluación requiere recalcular todos los polinomios base  $L_i$
- ▶ Ineficiente para múltiples evaluaciones (mejor usar splines)

## Resumen de Complejidades

| Temporal         | Espacial       |
|------------------|----------------|
| $O(m^2 \cdot d)$ | $O(m \cdot d)$ |

# Lagrange en $\mathbb{R}^n$ - Análisis de Complejidad II

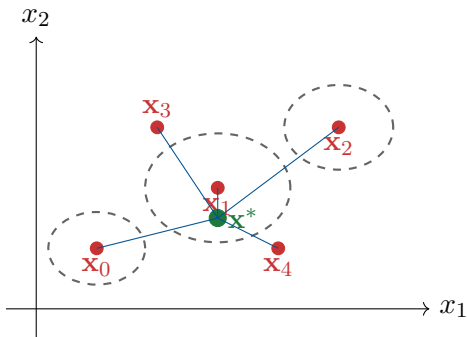
## Complejidad Espacial

- ▶ **Puntos de datos:**  $O(m \cdot d)$  para almacenar  $\mathbf{x}_i$  en  $\mathbb{R}^d$
- ▶ **Valores:**  $O(m)$  para  $y_i$
- ▶ **Auxiliar:**  $O(1)$  para cálculos temporales

## Limitaciones

- ▶ Fenómeno de Runge para puntos equidistantes (oscilaciones en bordes)
- ▶ Ineficiente para evaluaciones múltiples (costo  $O(m^2)$  por evaluación)
- ▶ Alternativa: Splines cúbicas ( $O(m)$  por evaluación)
- ▶ En altas dimensiones, sufre la maldición de la dimensionalidad

# Interpolación en $\mathbb{R}^2$ - Ilustración I



Interpolación en  $\mathbb{R}^2$

# Interpolación en $\mathbb{R}^2$ - Ilustración II

## Interpretación

En  $\mathbb{R}^d$ , la interpolación construye una superficie que pasa por todos los puntos dados. El valor en  $\mathbf{x}^*$  se calcula como combinación ponderada de los valores conocidos, con pesos  $L_i(\mathbf{x}^*)$  que dependen de las distancias a los puntos de interpolación.



## Grupo C: Evolución y Dinámica

## Objetivo

Modelar evolución temporal y problemas de frontera en espacios de dimensión arbitraria  $\mathbb{R}^n$ .

## Problemas Fundamentales

- ▶ ODEs en  $\mathbb{R}^n$ :  $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, t)$
- ▶ PDEs en  $\mathbb{R}^d$ : Laplace, calor, onda, Navier-Stokes
- ▶ Problemas de frontera: Dirichlet, Neumann, mixtos
- ▶ Estocásticos: SDEs, Monte Carlo en dimensión alta

## Métodos Principales

- ▶ Runge-Kutta (RK4, RK45) para ODEs
- ▶ Diferencias finitas para PDEs
- ▶ Elementos finitos para dominios complejos
- ▶ Métodos espectrales para alta precisión

# Runge-Kutta en $\mathbb{R}^n$ - Algoritmo RK4

---

## Algoritmo 41 RK4 para Sistemas de ODEs en $\mathbb{R}^n$

---

**Require:**  $\mathbf{f} : \mathbb{R}^n \times \mathbb{R} \rightarrow \mathbb{R}^n$ ,  $\mathbf{y}_0 \in \mathbb{R}^n$ ,  $t_0$ ,  $t_f$ ,  $h$

**Ensure:** Aproximación de  $\mathbf{y}(t)$  en  $\mathbb{R}^n$

```
1: procedure RK4_ND($\mathbf{f}, \mathbf{y}_0, t_0, t_f, h$)
2: $\mathbf{y} \leftarrow \mathbf{y}_0$, $t \leftarrow t_0$
3: while $t < t_f$ do
4: $\mathbf{k}_1 \leftarrow h \cdot \mathbf{f}(t, \mathbf{y})$
5: $\mathbf{k}_2 \leftarrow h \cdot \mathbf{f}(t + h/2, \mathbf{y} + \mathbf{k}_1/2)$
6: $\mathbf{k}_3 \leftarrow h \cdot \mathbf{f}(t + h/2, \mathbf{y} + \mathbf{k}_2/2)$
7: $\mathbf{k}_4 \leftarrow h \cdot \mathbf{f}(t + h, \mathbf{y} + \mathbf{k}_3)$
8: $\mathbf{y} \leftarrow \mathbf{y} + (\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4)/6$
9: $t \leftarrow t + h$
10: end while
11: return $\mathbf{y}$
12: end procedure
```

---

# RK4 en $\mathbb{R}^n$ - Análisis de Complejidad I

## Complejidad Temporal

- ▶ **Temporal:**  $O(N \cdot n \cdot C_f)$  donde  $N = (t_f - t_0)/h$  es número de pasos
- ▶ **Evaluaciones por paso:** 4 evaluaciones de  $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^n$
- ▶ **Precisión:** Error local  $O(h^4)$ , error global  $O(h^3)$

## Resumen de Complejidades

| Temporal                 | Espacial |
|--------------------------|----------|
| $O(N \cdot n \cdot C_f)$ | $O(n)$   |

## Complejidad Espacial

- ▶ **Espacial:**  $O(n)$  para vectores  $\mathbf{y}$ ,  $\mathbf{k}_1, \dots, \mathbf{k}_4$
- ▶ No requiere almacenamiento de pasos anteriores
- ▶ Adecuado para sistemas grandes en  $\mathbb{R}^n$

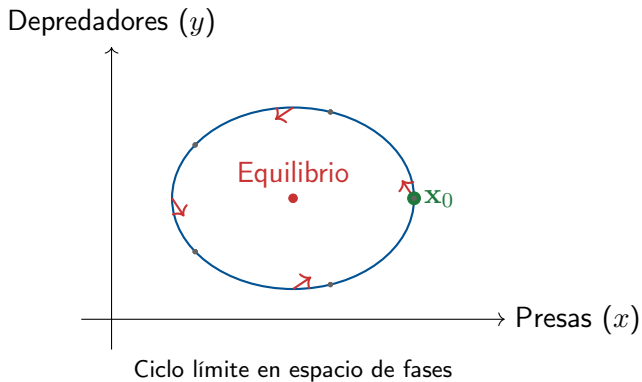
## Ejemplo: Sistema Depredador-Presa (Lotka-Volterra)

$$\dot{x} = \alpha x - \beta xy$$

$$\dot{y} = \delta xy - \gamma y$$

$\mathbf{x} = [x, y]^T \in \mathbb{R}^2$  modela la dinámica poblacional de dos especies.

# Trayectoria en $\mathbb{R}^2$ - Ilustración I



# Trayectoria en $\mathbb{R}^2$ - Ilustración II

## Interpretación

En  $\mathbb{R}^n$ , las soluciones de ODEs trazan trayectorias en el espacio de fases. Los ciclos límite representan comportamientos periódicos estables del sistema dinámico. RK4 aproxima estas trayectorias con alta precisión.



# Métodos Avanzados de Discretización

# Diferencias Finitas en $\mathbb{R}^d$ I

## Descripción

Aproximan derivadas parciales mediante cocientes de diferencias en mallas estructuradas de  $\mathbb{R}^d$ .

## Aproximaciones en $\mathbb{R}^d$

- ▶ **1D:**  $f'(x) \approx \frac{f(x+h)-f(x-h)}{2h}$  (centrada, orden  $O(h^2)$ )
- ▶ **2D:** Laplaciano  $\nabla^2 u \approx \frac{u_{i+1,j}+u_{i-1,j}+u_{i,j+1}+u_{i,j-1}-4u_{i,j}}{h^2}$
- ▶ **3D:** Extensión natural con 6 vecinos (estel de 7 puntos)
- ▶ **Alto orden:** Esquemas compactos de Padé

# Diferencias Finitas en $\mathbb{R}^d$ II

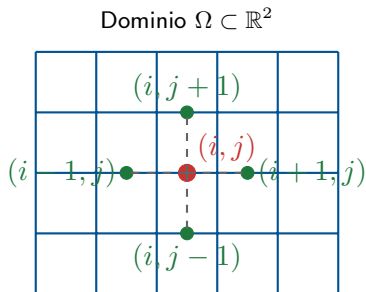
## Complejidad

- ▶ **Temporal:**  $O(N)$  donde  $N$  es número de puntos de malla
- ▶ **En  $\mathbb{R}^d$ :**  $N = n_1 \times n_2 \times \cdots \times n_d$  (producto cartesiano)
- ▶ **Espacial:**  $O(N)$  para almacenar solución discreta
- ▶ **Precisión:**  $O(h^p)$  donde  $p$  es orden del esquema

## Ventajas y Limitaciones

- ▶ **Ventajas:** Implementación simple, paralelización eficiente
- ▶ **Limitaciones:** Requiere mallas estructuradas, dificultad con geometrías complejas

# Malla 2D para Diferencias Finitas - Ilustración I



Estencil de 5 puntos para  $\nabla^2$

# Malla 2D para Diferencias Finitas - Ilustración II

## Interpretación

En  $\mathbb{R}^d$ , cada punto de la malla se relaciona con sus vecinos inmediatos. El estencil define qué vecinos contribuyen a la aproximación de derivadas en ese punto. El laplaciano en 2D usa los 4 vecinos ortogonales.

# Elementos Finitos en $\mathbb{R}^d$ (FEM) I

## Descripción

Discretiza el dominio  $\Omega \subset \mathbb{R}^d$  en elementos geométricos y aproxima la solución mediante funciones base locales definidas en cada elemento.

## Elementos por Dimensión

- ▶ **1D:** Segmentos de línea (2 nodos por elemento)
- ▶ **2D:** Triángulos (3 nodos) o cuadriláteros (4 nodos)
- ▶ **3D:** Tetraedros (4 nodos) o hexaedros (8 nodos)
- ▶ **Orden superior:** Elementos con nodos en aristas, caras o interiores

# Elementos Finitos en $\mathbb{R}^d$ (FEM) II

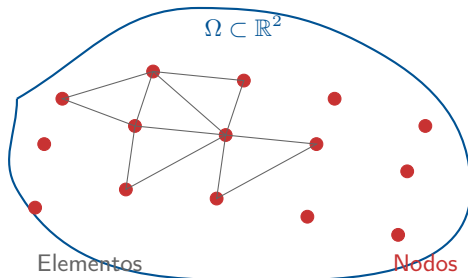
## Ventajas sobre Diferencias Finitas

- ▶ Maneja geometrías complejas e irregulares (dominios con fronteras curvas)
- ▶ Condiciones de frontera naturales incorporadas en la formulación débil
- ▶ Refinamiento adaptativo de malla localizado ( $h$ -refinement)
- ▶ Mayor orden de precisión con elementos de alto grado ( $p$ -refinement)

## Complejidad

- ▶ **Temporal:**  $O(N)$  a  $O(N \log N)$  según solver lineal
- ▶ **Espacial:**  $O(N)$  para matriz de rigidez dispersa
- ▶ **Precisión:**  $O(h^p)$  según orden de funciones base

# Malla de Elementos Finitos 2D - Ilustración I



## Interpretación

El dominio continuo  $\Omega \subset \mathbb{R}^d$  se discretiza en elementos finitos conectados en nodos. La solución se aproxima como combinación lineal de funciones base definidas por elemento. La malla puede adaptarse a geometrías complejas.



# Método Multigrid I

## Descripción

Método iterativo óptimo  $O(N)$  para sistemas lineales grandes que surgen de la discretización de PDEs. Usa múltiples niveles de malla para acelerar la convergencia eliminando componentes de error en todas las escalas.

## Principio Fundamental

- ▶ **Alta frecuencia:** Se elimina rápidamente mediante suavizado en mallas finas
- ▶ **Baja frecuencia:** Se elimina eficientemente resolviendo en mallas gruesas
- ▶ **Ciclo V:** Alterna entre niveles de malla  $h, 2h, 4h, \dots$
- ▶ **Corrección:** El error en malla fina se corrige con solución de malla gruesa

# Método Multigrid II

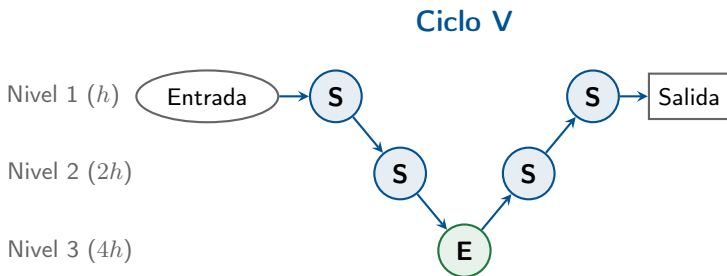
## Complejidad

- ▶ **Temporal:**  $O(N)$  (óptimo para sistemas elípticos)
- ▶ **Espacial:**  $O(N)$  para almacenamiento de múltiples niveles
- ▶ **Convergencia:** Independiente del tamaño de malla  $h$  (propiedad clave)
- ▶ **Total:**  $O(N \log N)$  para problemas en 3D con refinamiento geométrico

## Aplicaciones

- ▶ Ecuaciones de Poisson y Laplace en  $\mathbb{R}^d$
- ▶ Elasticidad lineal y no lineal
- ▶ Dinámica de fluidos computacional (CFD)
- ▶ Problemas de autovalores

# Ciclos Multigrid: V y W I



# Ciclos Multigrid: V y W II

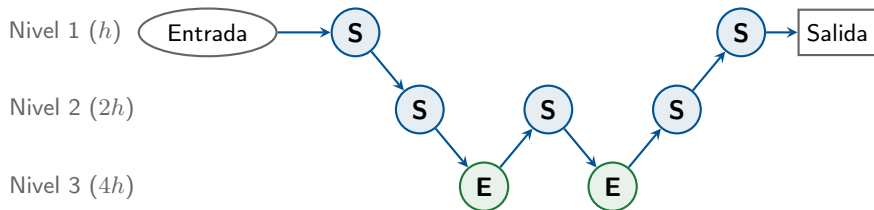
## Ciclo V

- ▶  $\gamma = 1$ : una visita por nivel grueso
- ▶ Forma de **V** simple
- ▶ Más rápido, menos robusto

*S: Suavizado (Jacobi/Gauss-Seidel)    E: Solución Exacta (Directa)*

# Ciclos Multigrid: V y W III

Ciclo W ( $\gamma = 2$ )



# Ciclos Multigrid: V y W IV

## Ciclo W

- ▶  $\gamma = 2$ : dos visitas por nivel grueso
- ▶ Forma de **W** doble
- ▶ Más lento, más robusto

## Interpretación

Un ciclo desciende por restricción a mallas más gruesas, resuelve en el nivel más grueso, y asciende por interpolación corrigiendo el error en cada nivel. Complejidad óptima  $O(N)$  independiente del tamaño de malla.

# Otros Métodos Avanzados I

## Métodos Espectrales

- ▶ Funciones base globales (Fourier, Chebyshev, Legendre)
- ▶ Precisión exponencial para soluciones suaves
- ▶ Complejidad:  $O(N \log N)$  con FFT
- ▶ Aplicaciones: Turbulencia, mecánica cuántica, modelos climáticos

## Métodos sin Malla (Meshless)

- ▶ Funciones de base radial (RBF) en  $\mathbb{R}^d$
- ▶ Sin generación de malla estructurada
- ▶ Ideal para fronteras móviles y dominios complejos
- ▶ Aplicaciones: Fractura, deformación grande, fluidos con superficie libre

# Otros Métodos Avanzados II

## Descomposición de Dominios

- ▶ Divide  $\Omega$  en subdominios  $\Omega_i$  más pequeños
- ▶ Paralelización natural para HPC (High Performance Computing)
- ▶ Métodos: Schwarz alternante, FETI, BDDC
- ▶ Aplicaciones: Problemas a gran escala, multifísica acoplada

## Métodos de Galerkin Discontinuo (DG)

- ▶ Combina ventajas de elementos finitos y volúmenes finitos
- ▶ Conservación local, alta precisión, paralelización eficiente
- ▶ Mayor número de grados de libertad que FEM continuo
- ▶ Aplicaciones: CFD compresible, transporte de radiación, ondas



# Interconexión de Grupos

# Interconexión de Grupos A, B, C I

## Relaciones entre Grupos

**A: Ecuaciones**

$\leftrightarrow$

**B: Aproximación**

$\updownarrow$

$\updownarrow$

**C: Evolución**

$\leftrightarrow$

*Todos interconectados*

## Ejemplos de Interconexión en $\mathbb{R}^n$

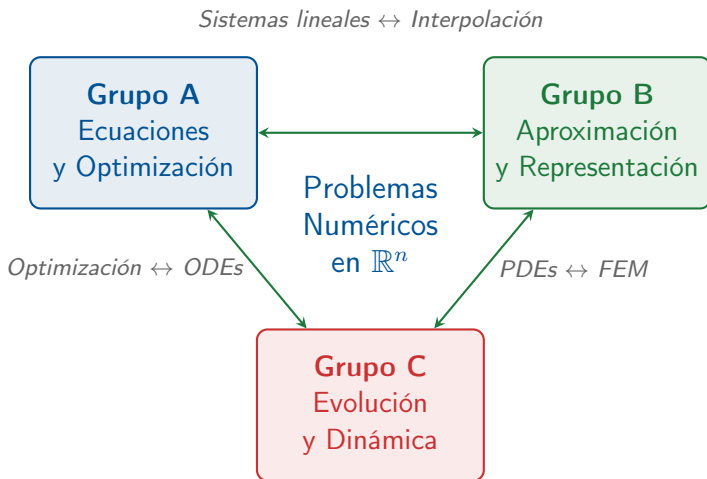
- ▶ **A**  $\leftrightarrow$  **B**: Resolver sistemas lineales para coeficientes de interpolación en  $\mathbb{R}^d$
- ▶ **B**  $\leftrightarrow$  **C**: Aproximar soluciones de PDEs con elementos finitos
- ▶ **C**  $\leftrightarrow$  **A**: Discretizar ODEs en  $\mathbb{R}^n$  genera sistemas algebraicos
- ▶ **A**  $\leftrightarrow$  **C**: Optimización de parámetros en modelos dinámicos

# Interconexión de Grupos A, B, C II

## Importancia en Aplicaciones

- ▶ **Ingeniería:** Análisis estructural (A+B), dinámica de fluidos (B+C)
- ▶ **Ciencia de Datos:** Ajuste de modelos por mínimos cuadrados (A+B), series temporales (B+C)
- ▶ **Física:** Simulación de N-cuerpos en  $\mathbb{R}^3$  (A+C), mecánica cuántica (B+C)
- ▶ **Bioingeniería:** Modelado de flujo sanguíneo (C) con elementos finitos (B) y solvers lineales (A)

# Interconexión de Métodos - Diagrama I



# Interconexión de Métodos - Diagrama II

## Interpretación

Los tres grupos están interconectados. Un problema práctico en  $\mathbb{R}^n$  típicamente requiere métodos de múltiples grupos. Por ejemplo, resolver una PDE (C) requiere discretización (B) y resolver sistemas lineales (A). La optimización de parámetros (A) puede requerir simulación dinámica (C).

# Algoritmos Paralelos y Computación GPU

# Fundamentos de Computación Paralela

# Introducción a la Computación Paralela I

## Definición y Motivación

La computación paralela es un paradigma donde múltiples cálculos se ejecutan simultáneamente para reducir el tiempo total de procesamiento. Surge como respuesta a las limitaciones físicas en el aumento de frecuencias de reloj y la necesidad de procesar volúmenes crecientes de datos.

El speedup se define como  $S_p = T_1/T_p$ , donde  $T_1$  es tiempo secuencial y  $T_p$  tiempo con  $p$  procesadores.



# Introducción a la Computación Paralela II

## Ley de Amdahl

El speedup máximo está limitado por la fracción no paralelizable del código:

$$S_p \leq \frac{1}{(1 - \alpha) + \alpha/p}$$

donde  $\alpha$  es la fracción paralelizable. Esta ley establece que incluso con infinitos procesadores, el speedup máximo es  $1/(1 - \alpha)$ .

# Introducción a la Computación Paralela III

## Taxonomía de Flynn

Clasificación de arquitecturas paralelas:

- ▶ **SISD**: Computador secuencial tradicional
- ▶ **SIMD**: Una instrucción, múltiples datos (GPUs, procesadores vectoriales)
- ▶ **MISD**: Poco común, usado en sistemas tolerantes a fallos
- ▶ **MIMD**: Múltiples instrucciones, múltiples datos (clusters, multinúcleo)

# Modelos de Programación Paralela I

## Memoria Compartida

Los procesadores comparten un espacio de direcciones común. La comunicación es implícita mediante variables compartidas. Ejemplos: OpenMP, pthreads. Ventaja: programación más intuitiva. Limitación: escalabilidad limitada por contención de memoria.

## Memoria Distribuida

Cada procesador tiene memoria privada. La comunicación es explícita mediante paso de mensajes. Ejemplo: MPI. Ventaja: alta escalabilidad. Limitación: overhead de comunicación y complejidad de programación.

# Modelos de Programación Paralela II

## Modelo Híbrido CPU-GPU

La CPU maneja la lógica de control y tareas secuenciales, mientras que la GPU ejecuta kernels masivamente paralelos. La comunicación entre ambos se realiza a través del bus PCIe, que puede convertirse en cuello de botella.

# Arquitectura CUDA

## ¿Qué es CUDA?

CUDA (Compute Unified Device Architecture) es una plataforma de computación paralela y modelo de programación de NVIDIA que permite utilizar GPUs para propósito general (GPGPU). Proporciona una extensión del lenguaje C/C++ para escribir código que se ejecuta en la GPU.

# Fundamentos de CUDA II

## Jerarquía de Memoria CUDA

- ▶ **Memoria Global:** Accesible por todos los threads, alta latencia (cientos de ciclos), gran capacidad.
- ▶ **Memoria Compartida:** Por bloque de threads, baja latencia, tamaño limitado (decenas de KB).
- ▶ **Registros:** Por thread, acceso inmediato, muy limitados (255 por thread máximo).
- ▶ **Memoria Constante:** Solo lectura, cacheada, optimizada para datos constantes.
- ▶ **Memoria Local:** Por thread, en realidad reside en global cuando no caben registros.

## Organización de Threads

- ▶ **Thread:** Unidad básica de ejecución, cada uno ejecuta el mismo kernel.
- ▶ **Warp:** Grupo de 32 threads que se ejecutan simultáneamente en el mismo SM.
- ▶ **Block:** Grupo de threads (hasta 1024) que pueden comunicarse vía memoria compartida.
- ▶ **Grid:** Conjunto de blocks que ejecutan un kernel.



## Estructura de un Kernel

Un kernel es una función marcada con `__global__` que se ejecuta en la GPU. Cada thread ejecuta una instancia del kernel y se identifica mediante índices predefinidos:

- ▶ `threadIdx.x`, `threadIdx.y`, `threadIdx.z` (dentro del block)
- ▶ `blockIdx.x`, `blockIdx.y`, `blockIdx.z` (dentro del grid)
- ▶ `blockDim.x` (tamaño del block)
- ▶ `gridDim.x` (tamaño del grid)

# Modelo de Programación CUDA II

## Configuración de Lanzamiento

El lanzamiento de un kernel se realiza con la sintaxis:

```
kernel«<numBlocks, threadsPerBlock»>(args);
```

Los parámetros de configuración determinan el número total de threads y su organización, lo que impacta directamente en el rendimiento.

# Modelo de Programación CUDA III

## Flujo Típico de Ejecución CUDA

1. Reservar memoria en la GPU (`cudaMalloc`)
2. Transferir datos de CPU a GPU (`cudaMemcpy`)
3. Lanzar kernel con configuración adecuada
4. Sincronizar si es necesario (`cudaDeviceSynchronize`)
5. Transferir resultados de GPU a CPU
6. Liberar memoria en GPU (`cudaFree`)

# Patrones de Diseño Paralelo

# Patrones de Diseño en GPU I

## Patrón Map

Aplica una función independiente a cada elemento de un conjunto de datos. Es el patrón más simple y eficiente en GPU porque no hay dependencias entre threads. Cada thread procesa uno o varios elementos. Ejemplos: suma de vectores, transformaciones elemento a elemento, filtrado de imágenes.

## Patrón Reduce

Combina todos los elementos en un único valor mediante una operación asociativa (suma, máximo, producto). Se implementa mediante un árbol binario paralelo con complejidad  $O(\log n)$  en tiempo y  $O(n)$  en trabajo total. Requiere sincronización dentro de cada block.

# Patrones de Diseño en GPU II

## Patrón Scan (Prefix Sum)

Calcula todas las sumas acumuladas  $\text{output}[i] = \sum_{j=0}^i \text{input}[j]$ . Fundamental para algoritmos como compactación, ordenamiento radix, y asignación dinámica de índices. El algoritmo de Blelloch logra  $O(\log n)$  tiempo con dos fases: ascendente (reduce) y descendente (down-sweep).

## Patrón Stencil

Actualiza cada elemento basándose en una vecindad fija de elementos adyacentes. Común en simulaciones físicas (difusión de calor, Jacobi), autómatas celulares (Game of Life), y procesamiento de imágenes (filtros gaussianos). Se optimiza usando memoria compartida para cachear la vecindad.

# Optimizaciones en CUDA

# Optimización de Memoria I

## Coalescencia de Accesos

Los accesos a memoria global son más eficientes cuando threads consecutivos acceden a direcciones consecutivas. Esto permite que la GPU combine múltiples accesos en una sola transacción. El patrón ideal es que `thread[i]` acceda a `address[i]`.

## Memoria Compartida

Aproximadamente  $100\times$  más rápida que la memoria global, pero limitada en tamaño (48-96 KB por SM). Ideal para datos reutilizados por múltiples threads en un mismo block. Requiere sincronización explícita con `__syncthreads()` para evitar condiciones de carrera.



## Bank Conflicts

La memoria compartida está dividida en 32 bancos. Si múltiples threads acceden al mismo banco en la misma instrucción, se producen conflictos que serializan los accesos. Se mitigan con padding o reorganización de datos.

# Optimización de Occupancy I

## Concepto de Occupancy

La occupancy es la relación entre warps activos por SM y el máximo posible. Una alta occupancy permite ocultar la latencia de memoria y mejorar el rendimiento general.

## Factores Limitantes de Occupancy

- ▶ **Registros por thread:** Máximo 255, pero a mayor uso, menos threads por SM.
- ▶ **Memoria compartida por block:** Consumo elevado limita blocks activos.
- ▶ **Tamaño de block:** Mínimo para ocupación eficiente suele ser 128-256 threads.

# Optimización de Occupancy II

## Estrategias de Mejora

Reducir uso de registros, ajustar tamaño de block al múltiplo de 32, minimizar memoria compartida por block, y utilizar herramientas como `cudaOccupancyMaxPotentialBlockSize` para encontrar configuración óptima.

# Optimización de Transferencias CPU-GPU I

## Cuello de Botella PCIe

El bus PCIe (8-16 GB/s) es significativamente más lento que la memoria GPU (400-900 GB/s). Las transferencias frecuentes pueden dominar el tiempo de ejecución.

## Buenas Prácticas

- ▶ Mantener datos en GPU el mayor tiempo posible.
- ▶ Agrupar transferencias pequeñas en una grande.
- ▶ Usar memoria page-locked (pinned) para transferencias más rápidas.
- ▶ Emplear streams y `cudaMemcpyAsync` para superponer cómputo y transferencia.

# Optimización de Transferencias CPU-GPU II

## Intensidad Aritmética

$$\text{Intensidad} = \frac{\text{Operaciones de punto flotante}}{\text{Bytes transferidos}}$$

Alta intensidad ( $>10$ ) indica que el kernel está limitado por cómputo (compute-bound) y es ideal para GPU. Baja intensidad ( $<1$ ) indica limitación por memoria (memory-bound) y puede no beneficiarse tanto de la GPU.

# Comparativa y Consideraciones

# CPU vs GPU: Características Distintivas I

## Comparativa General

| Característica | CPU                      | GPU                   |
|----------------|--------------------------|-----------------------|
| Núcleos        | 4-64                     | 1000-10000+           |
| Propósito      | Baja latencia            | Alto throughput       |
| Caché          | Grande (MBs)             | Pequeña (KBs por SM)  |
| Ancho de banda | 50-100 GB/s              | 400-900 GB/s          |
| Optimización   | Latencia individual      | Throughput masivo     |
| Ideal para     | Control, lógica compleja | Cómputo data-parallel |

# CPU vs GPU: Características Distintivas II

## Cuándo Usar GPU

**Sí:** Operaciones SIMD, alta intensidad aritmética, datos masivos, independencia entre elementos.

**No:** Lógica compleja con ramificación divergente, dependencias fuertes, datos pequeños donde el overhead de transferencia domina.



# Herramientas de Profiling CUDA I

## NVIDIA Nsight Systems

Proporciona una línea de tiempo de la ejecución CPU-GPU, permitiendo identificar cuellos de botella como transferencias PCIe, sincronizaciones y lanzamiento de kernels.

## NVIDIA Nsight Compute

Ofrece profiling detallado a nivel de kernel, con métricas de occupancy, uso de memoria, divergencia de warps, y sugerencias de optimización automáticas.

# Herramientas de Profiling CUDA II

## Métricas Clave a Monitorear

- ▶ **Occupancy:** Porcentaje de warps activos.
- ▶ **Memory Throughput:** Ancho de banda efectivo alcanzado.
- ▶ **Compute Throughput:** GFLOPS efectivos.
- ▶ **Divergence:** Porcentaje de warps con ramificación divergente.

# Limitaciones y Consideraciones Prácticas I

## Desafíos de Programación GPU

- ▶ **Curva de aprendizaje:** Modelo mental diferente a programación secuencial.
- ▶ **Depuración:** Más compleja que código CPU, requiere herramientas especializadas.
- ▶ **Portabilidad:** Código CUDA específico para NVIDIA; alternativas como OpenCL, HIP o SYCL ofrecen portabilidad entre vendedores.
- ▶ **Mantenimiento:** Optimizaciones pueden requerir ajustes por generación de hardware.

# Limitaciones y Consideraciones Prácticas II

## Consideraciones Energéticas

Las GPUs consumen 150-400W bajo carga, pero ofrecen mejor eficiencia energética (FLOPS/Watt) que CPUs para cargas de trabajo adecuadas. Importante en centros de datos y sistemas embebidos.

## Alternativas a CUDA

- ▶ **OpenCL:** Estándar abierto, soporte multi-vendor.
- ▶ **HIP:** Capa de portabilidad AMD/NVIDIA, código similar a CUDA.
- ▶ **SYCL:** C++ single-source con múltiples backends.
- ▶ **OpenACC:** Directivas para aceleración, similar a OpenMP.

# Análisis Comparativo

# Tablas Comparativas

# Análisis Comparativo de Paradigmas I

## Evaluación Cualitativa de Estrategias

| Paradigma             | Naturaleza             | Reutiliza      | Complejidad         | Optimalidad        |
|-----------------------|------------------------|----------------|---------------------|--------------------|
| Divide y Vencerás     | Recursiva              | Parcialmente   | $O(n \log n)$       | No garantizada     |
| Programación Dinámica | Recursiva + memoria    | Sí             | $O(n^2)$ a $O(n^3)$ | Óptima global      |
| Algoritmos Voraces    | Iterativa              | No             | $O(n \log n)$       | Óptima local       |
| Backtracking          | Recursiva exploratoria | Parcialmente   | Exponencial         | Óptima (con poda)  |
| Branch & Bound        | Recursiva + heurística | Sí             | Exponencial         | Óptima garantizada |
| Algoritmos Genéticos  | Poblacional evolutivo  | Implícitamente | Variable            | Aproximada         |

# Análisis de Ventajas y Limitaciones I

## Divide y Vencerás

### Ventajas

- Implementación intuitiva
- Paralelización natural
- Análisis bien establecido
- Estructura modular

### Limitaciones

- Overhead de recursión
- No reutiliza soluciones
- Subproblemas independientes
- Stack overflow



# Análisis de Ventajas y Limitaciones II

## Programación Dinámica

### Ventajas

- Evita recomputación
- Optimalidad garantizada
- Aplica a problemas complejos
- Solución bottom-up

### Limitaciones

- Alto consumo de memoria
- Análisis no trivial
- Diseño de tabla complejo
- Dificultad multidimensional

# Dominios de Aplicación Paradigmática I

## Contextos de Aplicación Característicos

- ▶ **Divide y Vencerás:** Ordenamiento (Quicksort, Mergesort), FFT, Strassen.
- ▶ **Programación Dinámica:** Alineamiento de secuencias, problemas de optimización combinatoria.
- ▶ **Algoritmos Voraces:** Árboles de expansión mínima, codificación de Huffman.
- ▶ **Backtracking:** N-Reinas, Sudoku, generación de permutaciones.
- ▶ **Branch and Bound:** Optimización exacta en programación entera, TSP.

# Criterios de Selección Paradigmática I

## Heurísticas para la Elección Estratégica

- ▶ Presencia de **subestructura óptima** y **subproblemas solapados** → **Programación Dinámica**.
- ▶ Problema **inherentemente recursivo** con subproblemas **independientes** → **Divide y Vencerás**.
- ▶ Existencia de **propiedad de elección voraz** demostrable → **Algoritmos Voraces**.
- ▶ Necesidad de **exploración exhaustiva** → **Backtracking**.
- ▶ Optimización **combinatoria exacta** con espacio reducible → **Branch and Bound**.

# Criterios de Selección Paradigmática II

## Principio Metodológico

El análisis preliminar de la *estructura combinatoria* del problema y la *naturaleza del espacio de búsqueda* constituye una etapa fundamental en el diseño algorítmico formal.

# Ejercicios propuestos

# NIVEL BÁSICO (30 Problemas)

## Nivel Básico: Búsqueda de Ceros y Raíces I

1. Diseñe un algoritmo que encuentre un cero de la función  $f(x) = x^3 - 2x - 5$  en el intervalo  $[2, 3]$  utilizando el método de bisección con una tolerancia  $\epsilon = 10^{-6}$ . Especifique el número máximo de iteraciones necesarias.
2. Implemente un algoritmo de iteración de punto fijo para encontrar la raíz de  $g(x) = \cos(x) - x = 0$ . Reformule la ecuación como  $x = \cos(x)$  y analice la convergencia del esquema.
3. Desarrolle el algoritmo de Newton-Raphson para aproximar la raíz de  $f(x) = e^x - 2$  partiendo de  $x_0 = 1$ . Incluya criterios de parada basados en el valor absoluto de  $f(x)$  y en la diferencia  $|x_{k+1} - x_k|$ .
4. Diseñe una variante del método de la secante que no requiera el cálculo explícito de derivadas, empleando la aproximación  $f'(x_n) \approx \frac{f(x_n+h) - f(x_n)}{h}$  con  $h = 10^{-5}$ .

## Nivel Básico: Búsqueda de Ceros y Raíces II

5. Cree un algoritmo numéricamente estable para resolver la ecuación cuadrática  $ax^2 + bx + c = 0$ , de modo que se evite la cancelación catastrófica cuando  $|b| \approx \sqrt{b^2 - 4ac}$ .



# Nivel Básico: Optimización Unidimensional I

6. Implemente el método de la sección dorada para minimizar la función  $f(x) = x^4 - 3x^3 + 2$  en el intervalo  $[0, 3]$ . Determine el intervalo final de incertidumbre, el cual debe ser menor a  $10^{-4}$ .
7. Diseñe un algoritmo que utilice interpolación cuadrática sucesiva para aproximar el mínimo de  $f(x) = x^2 + \sin(5x)$  en el intervalo  $[0, 2]$ .
8. Combine los métodos de bisección, secante e interpolación cuadrática en un algoritmo híbrido robusto para encontrar mínimos de funciones unimodales sin requerir derivadas (método de Brent).

## Nivel Básico: Ajuste de Datos - Mínimos Cuadrados I

9. Dado un conjunto de puntos  $\{(x_i, y_i)\}_{i=1}^n$ , diseñe un algoritmo de complejidad  $O(n)$  que calcule los parámetros  $a$  y  $b$  de la recta  $y = ax + b$  minimizando la suma de cuadrados  $\sum_{i=1}^n (y_i - ax_i - b)^2$ .
10. Desarrolle un algoritmo que ajuste un polinomio cuadrático  $y = ax^2 + bx + c$  mediante el método de las ecuaciones normales. Resuelva el sistema lineal resultante utilizando descomposición LU.
11. Diseñe un algoritmo para ajustar un modelo exponencial  $y = ae^{bx}$  transformando el problema a uno lineal mediante logaritmos; a continuación, calcule los parámetros óptimos.
12. Implemente un algoritmo eficiente para calcular el promedio móvil de orden  $k$  sobre una serie temporal de  $n$  datos, optimizando para que su tiempo de ejecución sea  $O(n)$ .

## Nivel Básico: Programación Lineal Básica I

13. Diseñe un algoritmo que resuelva problemas de programación lineal con dos variables mediante el método gráfico: debe encontrar los vértices factibles de la región poliedral y evaluar la función objetivo en ellos.
14. Implemente el método Simplex para maximizar la función  $z = 3x_1 + 2x_2$  sujeta a las restricciones  $x_1 + x_2 \leq 4$ ,  $x_1 - x_2 \leq 2$ ,  $x_1, x_2 \geq 0$ . Muestre todas las tablas del proceso.
15. Diseñe el procedimiento de las dos fases del método Simplex para obtener una solución factible inicial cuando el origen no es factible.

# Nivel Básico: Interpolación y Aproximación I

16. Cree un algoritmo que construya el polinomio interpolante de Lagrange para un conjunto de  $n$  puntos dados y que evalúe dicho polinomio en un punto  $x$  cualquiera con complejidad  $O(n^2)$ .
17. Implemente el esquema de diferencias divididas de Newton para la interpolación polinómica, de modo que sea posible añadir nuevos puntos sin recalcular todo desde el principio.
18. Diseñe un algoritmo que construya una función spline lineal continua por tramos que interpole  $n$  puntos dados.
19. Para una función  $f$  conocida en un conjunto de puntos  $\{x_i\}$ , aproxímela mediante una combinación lineal  $p(x) = \sum_{j=0}^m c_j \phi_j(x)$ , donde  $\{\phi_j\}$  es una base ortogonal dada, utilizando el método de mínimos cuadrados discretos.

## Nivel Básico: Derivación e Integración Numérica I

20. Diseñe un algoritmo que estime la derivada  $f'(x)$  utilizando diferencias centrales con un paso  $h$  adaptativo, de manera que se equilibren los errores de truncamiento y redondeo.
21. Implemente el método de integración de Romberg, que acelera la regla del trapecio mediante extrapolación de Richardson hasta alcanzar un orden  $O(h^8)$ .
22. Diseñe un algoritmo de cuadratura gaussiana con tres puntos para aproximar la integral  $\int_{-1}^1 f(x) dx$  y extiéndalo a intervalos arbitrarios mediante un cambio de variable adecuado.

## Nivel Básico: Ecuaciones No Lineales Simples I

23. Resuelva el siguiente sistema no lineal utilizando el método de Newton multivariante, partiendo del punto  $(1.5, 0.5)$ :

$$\begin{cases} x^2 + y^2 = 4, \\ xy = 1. \end{cases}$$

24. Diseñe un algoritmo de iteración de punto fijo para sistemas lineales, aplicado a la reformulación de  $Ax = b$  como  $x = x - \omega(Ax - b)$ . Analice la elección del parámetro  $\omega$  para garantizar la convergencia.

# Nivel Básico: Optimización con Restricciones Simples I

25. Implemente el método de descenso coordinado (cíclico) para minimizar la función  $f(x, y) = x^2 + 2y^2 + xy$  partiendo del punto  $(1, 1)$ .
26. Diseñe un algoritmo de gradiente descendente con búsqueda lineal exacta para funciones cuadráticas, donde el paso  $\alpha$  en cada iteración se determina analíticamente resolviendo  $\min_{\alpha} f(x_k + \alpha d_k)$ .
27. Para minimizar la función  $f(x) = \|Ax - b\|^2$  sujeta a la restricción  $x \geq 0$ , diseñe el método de proyección del gradiente con paso fijo.

# Nivel Básico: Algoritmos de Aleatorización Básicos I

28. Estime el valor de la integral  $\int_0^1 e^{-x^2} dx$  mediante el método de Monte Carlo con  $N$  muestras. Diseñe el algoritmo y calcule el error estándar de la estimación.
29. Implemente el algoritmo de recocido simulado para minimizar la función  $f(x) = x^2$  en el intervalo  $[-10, 10]$ , utilizando un esquema de temperatura  $T_k = T_0 / \log(1 + k)$ .
30. Diseñe un algoritmo de búsqueda local que, dado un punto inicial, explore vecindarios aleatorios y acepte únicamente mejoras, con el fin de optimizar funciones multimodales.



# NIVEL MEDIO (40 Problemas)

## Nivel Medio: Métodos Avanzados para Ceros I

1. Extienda el método de la secante mediante interpolación cuadrática inversa (método de Müller) para encontrar raíces complejas de polinomios con coeficientes reales.
2. Diseñe una variante del método de Newton que mantenga convergencia cuadrática para raíces de multiplicidad  $m > 1$  de una función  $f$ .
3. Modifique el método de la falsa posición para evitar el estancamiento unilateral, combinando características de la bisección y del propio método de regula falsi.
4. Diseñe un algoritmo de continuación (homotopía) para resolver sistemas no lineales  $F(x) = 0$ , trazando una curva desde un problema conocido  $G(x) = 0$  hasta la solución deseada.
5. Implemente el método de Broyden para sistemas no lineales, actualizando la aproximación del jacobiano mediante una modificación de rango uno sin recalcular derivadas.

## Nivel Medio: Optimización Multidimensional Sin Restricciones I

6. Diseñe un algoritmo de Newton que garantice direcciones de descenso modificando la matriz hessiana cuando no sea definida positiva (por ejemplo, mediante el ajuste de sus valores propios).
7. Implemente el método del gradiente conjugado para funciones no lineales generales (versión de Fletcher-Reeves), con reinicio cada  $n$  iteraciones.
8. Desarrolle el algoritmo DFP (Davidon-Fletcher-Powell) manteniendo matrices  $H_k$  definidas positivas mediante actualizaciones de rango dos.
9. Implemente el algoritmo BFGS, que constituye la actualización dual de DFP, demostrando su superioridad numérica y su propiedad de herencia de curvatura.

## Nivel Medio: Optimización Multidimensional Sin Restricciones II

10. Diseñe un método de región de confianza que utilice el paso dogleg para resolver de forma aproximada el subproblema  $\min_{\|d\| \leq \Delta} m_k(d)$ , donde  $m_k$  es un modelo cuadrático de la función objetivo.
11. Implemente el algoritmo de Levenberg-Marquardt para problemas de mínimos cuadrados no lineales, es decir, para minimizar  $\sum_{i=1}^m r_i(x)^2$ , donde  $r_i$  son los residuos.
12. Diseñe un algoritmo de búsqueda lineal inexacta que satisfaga las condiciones fuertes de Wolfe, utilizando una estrategia de retroceso (backtracking).

## Nivel Medio: Optimización con Restricciones I

13. Minimice una función  $f(x)$  sujeta a restricciones de igualdad  $c_i(x) = 0$  mediante el método de penalización cuadrática, resolviendo  $\min f(x) + \frac{\mu}{2} \sum c_i(x)^2$  con una secuencia creciente de parámetros  $\mu$ .
14. Para restricciones de desigualdad  $c_i(x) \geq 0$ , diseñe el método de barrera logarítmica, que resuelve  $\min f(x) - \mu \sum \log(c_i(x))$  haciendo tender  $\mu$  a cero.
15. Implemente el método del gradiente proyectado para el problema  $\min_{l \leq x \leq u} f$  utilizando una proyección euclidiana eficiente sobre la caja.
16. Diseñe el método del gradiente reducido generalizado (GRG) para optimización no lineal con restricciones de igualdad, empleando factorización QR para determinar las variables básicas y no básicas.
17. Diseñe un algoritmo que verifique las condiciones KKT para un punto dado y estime los multiplicadores de Lagrange mediante mínimos cuadrados.

## Nivel Medio: Optimización con Restricciones II

18. Implemente un paso de programación cuadrática secuencial (SQP) resolviendo el subproblema cuadrático:

$$\begin{aligned} \min_d \quad & \nabla f(x_k)^T d + \frac{1}{2} d^T B_k d \\ \text{s.a.} \quad & \nabla c_i(x_k)^T d + c_i(x_k) = 0. \end{aligned}$$

## Nivel Medio: Ajuste de Datos Avanzado I

19. Diseñe un algoritmo de regresión ponderada donde cada observación  $(x_i, y_i)$  tiene un peso  $w_i$ , resolviendo el problema  $\min \sum w_i(y_i - f(x_i))^2$ .
20. Implemente la regresión robusta utilizando la función de pérdida de Huber, que combina el comportamiento cuadrático para residuos pequeños con el lineal para residuos grandes.
21. Para ajustar un modelo no lineal de la forma  $y = ae^{bx} + ce^{dx}$  a un conjunto de datos, diseñe el algoritmo de Gauss-Newton, linealizando los residuos respecto a los parámetros.
22. Diseñe un algoritmo de suavizado mediante splines cúbicos que resuelva  $\min \sum (y_i - f(x_i))^2 + \lambda \int (f''(x))^2 dx$ , donde  $f$  es un spline cúbico natural.
23. Cree un algoritmo de interpolación de Hermite que coincida con los valores  $f(x_i)$  y las derivadas  $f'(x_i)$  en un conjunto de puntos, utilizando polinomios de grado  $2n - 1$ .

24. Diseñe el algoritmo de Remez para la aproximación uniforme (minimax) de funciones continuas mediante polinomios, de modo que se minimice el máximo error absoluto.



## Nivel Medio: Programación Lineal Avanzada I

25. Implemente un método de punto interior primal-dual para programación lineal, resolviendo en cada iteración el sistema lineal de KKT perturbado.
26. Para problemas con estructura de bloque angular, diseñe el método de descomposición de Dantzig-Wolfe, generando variables extremas de forma dinámica (generación de columnas).
27. Aplique la generación de columnas al problema de corte unidimensional (cutting stock), cuyo objetivo es minimizar el número de rollos utilizados para satisfacer demandas de longitudes menores.
28. Diseñe un esquema de branch-and-bound para programación lineal entera mixta, utilizando la relajación lineal y reglas de branching sobre variables fraccionarias.

## Nivel Medio: Programación Lineal Entera y Combinatoria I

29. Implemente los planos de corte de Gomory (cortes fraccionarios) para eliminar soluciones fraccionarias obtenidas mediante el método simplex, añadiendo restricciones válidas que mejoren la relajación.
30. Combine branch-and-bound con la generación dinámica de cortes (por ejemplo, cover cuts o flow cover cuts) en un esquema de branch-and-cut para mejorar las cotas de las relajaciones.
31. Para problemas de optimización con restricciones complicadas, diseñe una relajación lagrangiana que dualice dichas restricciones y resuelva el dual lagrangiano mediante el método de subgradiente.
32. Resuelva el problema de la mochila 0-1:  $\max \sum v_i x_i$  sujeto a  $\sum w_i x_i \leq W$ ,  $x_i \in \{0, 1\}$ , utilizando programación dinámica con complejidad  $O(nW)$ .

## Nivel Medio: Optimización Multiobjetivo I

33. Diseñe un algoritmo que genere puntos de la frontera de Pareto para un problema bicriterio variando sistemáticamente los pesos  $\lambda \in [0, 1]$  en la combinación lineal  $\min \lambda f_1 + (1 - \lambda)f_2$ .
34. Para dos objetivos, diseñe un esquema basado en el método de las  $\epsilon$ -restricciones, que optimice  $f_1$  sujeto a  $f_2 \leq \epsilon$  para distintos valores de  $\epsilon$ , obteniendo así el frente de Pareto.
35. Formule y resuelva un problema de programación por metas (goal programming) que minimice las desviaciones absolutas respecto a metas  $g_i$  establecidas para múltiples objetivos, utilizando variables de desviación.
36. Implemente una versión básica del algoritmo NSGA-II (Non-dominated Sorting Genetic Algorithm II) que incluya ordenamiento no dominado, distancia de crowding y elitismo, para aproximar el frente de Pareto de un problema multiobjetivo.

## Nivel Medio: Algoritmos Bioinspirados y Metaheurísticas I

37. Diseñe un algoritmo de optimización por enjambre de partículas (PSO) para problemas continuos, actualizando la velocidad de cada partícula según la expresión  $v_i \leftarrow wv_i + c_1r_1(pbest_i - x_i) + c_2r_2(gbest - x_i)$ .
38. Implemente un algoritmo genético con codificación real para la optimización multimodal de funciones continuas, utilizando operadores de cruce BLX- $\alpha$  y mutación gaussiana.
39. Extienda el algoritmo de optimización por colonia de hormigas (ACO) a dominios continuos, reemplazando las feromonas discretas por distribuciones de probabilidad basadas en kernels.
40. Diseñe el algoritmo de evolución diferencial (DE) con la estrategia DE/rand/ para cada vector objetivo, genere un vector mutante  $v = x_{r1} + F(x_{r2} - x_{r3})$  y aplique un cruce binomial.

# Bibliografía recomendada

# Bibliografía recomendada I

## Textos Canónicos en Diseño Algorítmico

- ▶ **Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009).** *Introduction to Algorithms* (3ª ed.). MIT Press.  
[▶ Acceso al texto](#)
- ▶ **Brassard, G., & Bratley, P. (1996).** *Fundamentals of Algorithmics*. Prentice Hall.
- ▶ **Kleinberg, J., & Tardos, É. (2006).** *Algorithm Design*. Pearson Education.
- ▶ **Skiena, S. S. (2008).** *The Algorithm Design Manual* (2ª ed.). Springer.

# Bibliografía recomendada II

## Recursos Digitales Especializados

- ▶ **Python Algorithms & Data Structures:** Documentación y tutoriales oficiales.  
▶ Explorar recurso
- ▶ **GeeksforGeeks - Algorithm Fundamentals:** Plataforma con explicaciones y ejercicios.  
▶ Visitar sitio
- ▶ **ACM Journal of Experimental Algorithmics (JEA):** Publicaciones académicas de vanguardia.  
▶ Acceder al journal
- ▶ **Algorithm Visualization Portal:** Herramienta interactiva para comprender algoritmos.  
▶ Visualizar algoritmos



universidad  
de león

<https://www.unileon.es/>